Vietnam General Confederation of Labor

Ton Duc Thang University

Faculty of Information Technology



Dinh Quoc Cuong – 523H0008

Vo Huynh Minh Duc – 523H0014

Building a Video Sharing Platform with an Automated HLS Transcoding Pipeline on a Serverless Container and Event-Driven Architecture

INFORMATION TECHNOLOGY PROJECT

DEVOPS, CONTAINERIZATION & CLOUD SYSTEMS

Supervisor

MSc. Mai Van Manh

Ho Chi Minh City, 2026

Acknowledgment

First and foremost, we would like to express our sincere gratitude to our supervisor, **MSc. Mai Van Manh**, for his invaluable guidance, constructive feedback, and continuous encouragement throughout the duration of this project. His expertise, patience, and dedication have played a crucial role in shaping both the technical direction and the academic rigour of this work.

We are also deeply thankful to the lecturers and staff of the Faculty of Information Technology at Ton Duc Thang University, whose support and resources provided a strong foundation for our work. Their commitment to creating an inspiring and intellectually stimulating environment has greatly contributed to our growth.

Our gratitude further extends to our classmates and friends who offered helpful discussions, motivation, and meaningful insights during the development and evaluation of this project. Their companionship made the challenging moments far more manageable.

Finally, we are profoundly grateful to our families for their unconditional support, patience, and encouragement. Their belief in us has always been our greatest source of strength and determination.

To all of you, we extend our heartfelt thanks.

Certification

This thesis was carried out at **Ton Duc Thang University**.

Advisor: MSc. Mai Van Manh

.....
(Title, Full Name and Signature)

This thesis was defended at the Undergraduate Thesis Examination Committee held at Ton Duc Thang University on . . . . . / . . . . . / . . . . . .

Confirmation of the Chairman of the Undergraduate Thesis Examination Committee and the Dean of the Faculty after receiving the revised thesis (if any).

CHAIRMAN **DEAN OF FACULTY**

. . . . . .

Declaration

This thesis was completed at **Ton Duc Thang University**. We hereby declare that this is our own research work, conducted under the academic supervision of **MSc. Mai Van Manh**. All analyses, discussions, and results presented in this thesis are truthful, original, and have not been published in any form prior to this submission.

All data used for analysis, evaluation, and comparison were collected directly by the author(s) from various reliable sources, which are fully cited in the reference section of this thesis. In addition, certain evaluations, comments, and figures quoted from other authors, organizations, or research works are clearly referenced and properly acknowledged.

If any form of academic dishonesty, plagiarism, or copyright infringement is discovered, we accept full responsibility for the content of this thesis. Ton Duc Thang University shall bear no liability for any copyright or authorship violations committed by the author(s) during the execution of this research.

Ho Chi Minh City, day month year

Author(s)

(Signature and full name)

Abstract

Modern video sharing platforms must solve a common core problem: when a user uploads a source file in an arbitrary format, codec, or resolution, the system has to transcode it automatically into HTTP Live Streaming (HLS) so that browsers can play it with adaptive quality. Provisioning a dedicated server cluster or a Kubernetes system that runs continuously purely to wait for transcoding work is wasteful, because upload traffic is inherently bursty and the infrastructure sits idle most of the time. Conversely, when many users upload simultaneously, the system must scale out to process hundreds of videos in parallel.

This project presents the design, implementation, and evaluation of a complete video sharing platform whose transcoding subsystem is built on a Serverless Container and Event-Driven architecture. Uploading a file to Amazon S3 emits an event that is buffered in Amazon SQS and dispatched by an AWS Lambda job submitter to AWS Batch running on AWS Fargate. A container packaging FFmpeg is created only when there is work to do, produces up to six renditions from 144p to 1080p together with a master playlist and a thumbnail, uploads the result to a processed bucket, updates the metadata in MongoDB Atlas, and then terminates. Videos of five minutes or more are encoded as parallel chunks, and four hours of video was ready in 135 minutes at 8 vCPU. Content is delivered through Amazon CloudFront with Origin Access Control, and every stream is gated at the edge by CloudFront Signed Cookies, so that a viewer without a valid signature is rejected even when they know the exact manifest URL; a video's visibility setting governs who the backend will issue those cookies to, not whether the gate applies. Content is screened automatically and reviewed in a console that records each decision.

The entire AWS infrastructure is described as code using thirteen independent Terraform modules and can be provisioned with a single command. Seven GitHub Actions workflows automate linting, unit testing, static security analysis, container image scanning, infrastructure validation, and deployment, with a two-tier quality gate that blocks a pull request whenever a fixable critical vulnerability is detected.

The system is evaluated along four dimensions: quality of experience, measured

both as Time-to-First-Frame and, by driving the deployed player in a headless browser under emulated network conditions, as rebuffering ratio, bitrate switching and the bitrate a viewer actually receives; transcoding pipeline latency across 100 MB, 500 MB, and 1 GB inputs, scalability under a concurrent upload stress test, and operating cost compared against a traditional always-on Amazon EC2 deployment. The cost analysis shows that, for the architecture-dependent portion of the bill, the serverless approach removes almost all idle resource waste, although this advantage narrows as sustained utilisation increases.

List of Abbreviations

ABR	Adaptive Bitrate Streaming
ACM	AWS Certificate Manager
API	Application Programming Interface
ARN	Amazon Resource Name
AWS	Amazon Web Services
CDN	Content Delivery Network
CI/CD	Continuous Integration / Continuous Deployment
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
DLQ	Dead Letter Queue
DNS	Domain Name System
DRM	Digital Rights Management
EBS	Amazon Elastic Block Store
ECR	Amazon Elastic Container Registry
ECS	Amazon Elastic Container Service
ERD	Entity Relationship Diagram
FinOps	Cloud Financial Operations
FR	Functional Requirement
GPU	Graphics Processing Unit
HEVC	High Efficiency Video Coding
HLS	HTTP Live Streaming
IaC	Infrastructure as Code
IAM	AWS Identity and Access Management
JWT	JSON Web Token
MIME	Multipurpose Internet Mail Extensions
NFR	Non-Functional Requirement
OAC	Origin Access Control

QoE	Quality of Experience
SAST	Static Application Security Testing
SCA	Software Composition Analysis
SPA	Single Page Application
SQS	Amazon Simple Queue Service
S3	Amazon Simple Storage Service
TLS	Transport Layer Security
TTFF	Time-to-First-Frame
TTL	Time To Live
UC	Use Case
vCPU	Virtual Central Processing Unit
VMAF	Video Multi-Method Assessment Fusion
VOD	Video on Demand
VPC	Amazon Virtual Private Cloud

Contents

Acknowledgment **i**

Certification **ii**

Declaration **iii**

Abstract **iv**

List of Abbreviations **vi**

1 Introduction **1**

1.1 Background and Rationale 1

1.2 Project Objectives 2

1.3 Scope and Deliverables 2

1.4 Research Methodology 3

1.5 Practical Significance 4

1.6 Main Contributions 4

1.7 Structure of the Report 5

2 Literature Review and Technologies **6**

2.1 Related Platforms and Comparison 6

2.2 Core Models and Principles 8

2.2.1 Serverless Containers 8

2.2.2 Event-Driven Architecture 10

2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder 10

2.3 Technology Stack Justification 12

3 Requirements Analysis **13**

3.1 Stakeholders and User Requirements 13

3.2 Functional Requirements 13

3.3 Non-Functional Requirements . . . 15
3.4 Use Case Model . . . 16

4 System Design **20**
4.1 High-Level Architecture . . . 20
4.2 Transcoding Pipeline Design . . . 20
4.2.1 Upload of Large Files . . . 24
4.2.2 Chunk-Parallel Transcoding of Long Videos . . . 24
4.3 Database Design . . . 25
4.4 Content Protection Design . . . 29
4.5 Content Moderation and Operations Design . . . 30
4.5.1 Automatic Screening . . . 31
4.5.2 Moderation States and Visibility . . . 32
4.5.3 Review, Appeals and Audit . . . 32
4.5.4 Measuring the Screening . . . 33
4.5.5 Roles and Permissions . . . 34
4.5.6 Verifying the Owner of an Account . . . 34
4.6 Interface Design . . . 35

5 Development and Testing **39**
5.1 Backend Implementation . . . 39
5.2 Transcoder Implementation . . . 41
5.3 Infrastructure as Code . . . 43
5.4 CI/CD Pipeline and DevSecOps . . . 45
5.5 Challenges During Development . . . 47
5.5.1 Provisioning Blocked by Account Policy . . . 47
5.5.2 A Signed Cookie Policy That Matched Nothing . . . 48
5.5.3 Two Consequences of Gating Every Object . . . 48
5.5.4 Playback Across the Cookie Expiry Boundary . . . 48
5.5.5 An Empty Secret That Stopped the Pipeline . . . 49
5.5.6 Retry and Alerting on the Deployed Path . . . 49
5.5.7 Monitoring the Service Rather Than the Server . . . 50
5.5.8 Infrastructure Drift in the Instance Definition . . . 50
5.6 Testing . . . 51

6 Deployment and Evaluation **55**
6.1 The Deployed Environment . . . 55

6.2 User Interface Showcase . . . 56
6.3 Quality of Experience Evaluation . . . 59
6.3.1 Time to First Frame Through the CDN . . . 60
6.3.2 Delivery Latency by Client Location . . . 61
6.3.3 Playback Under Constrained Bandwidth . . . 62
6.3.4 Effect of Content Complexity . . . 64
6.4 Transcoding Pipeline Performance . . . 66
6.5 Chunked Transcoding of Long Videos . . . 67
6.6 Load Testing . . . 72
6.6.1 Queue and Admission Under Burst . . . 72
6.6.2 Throughput With Real Video . . . 73
6.7 Cost Analysis . . . 74
6.8 Requirements Against the Evidence . . . 75

7 Conclusion and Future Work **77**
7.1 Summary of Achievements . . . 77
7.2 Strengths of the System . . . 78
7.3 Limitations . . . 79
7.4 Future Work . . . 80

A Supplementary Material **85**
A.1 Repository Structure . . . 85
A.2 FFmpeg Transcoding Command . . . 85
A.3 Signed Cookie Custom Policy . . . 87
A.4 Reproducing the Measurements . . . 88
A.5 Deploying the Infrastructure . . . 89

B Measurement Tables **91**
B.1 Time-to-First-Frame . . . 91
B.2 Playback Quality . . . 93
B.3 Transcoding Latency and Capacity . . . 93

List of Figures

2.1 *Compute models for a bursty transcoding workload: an always-on cluster compared with on-demand serverless containers* 9
2.2 *HLS artefacts produced from one source file and the order in which a player retrieves them* 11

3.1 *Use case diagram of the video sharing platform* 17
3.2 *Use case diagram of moderation and operation* 18

4.1 *Overall system architecture, showing the deployment boundaries and the numbered data flow from an anonymous visit through upload, transcoding and signed delivery; the key at the lower left names each step* 21
4.2 *Sequence diagram of the event-driven transcoding pipeline for one container; a video of five minutes or more takes the chunked path of Table 4.1* 22
4.3 *Entity relationship diagram of the core MongoDB collections* 26
4.4 *Entity relationship diagram of the collections that support moderation and operation; every video field refers to* `Video` *and every user field to* `User` 28
4.5 *User flow through the application, from an anonymous visit to a published video* 37

5.1 *The seven GitHub Actions workflows, the file paths that trigger each of them, and the two-tier DevSecOps quality gate* 46

6.1 *Catalogue page, the entry point for a visitor* 56
6.2 *Watch page with custom player, rendition selector and related videos sidebar* 57
6.3 *Upload page, showing direct-to-storage transfer with live telemetry* . . . 57
6.4 *Channel management view, including videos still being processed* . . . 58
6.5 *Operations console, Removed tab, with the labels the machine detected and who removed each video* 58

6.6 *Operations console, Accuracy tab, each rate shown with its interval and sample size* 59
6.7 *Median time to first frame in three delivery configurations, and the median of each stage where it was measured* 60
6.8 *Median time to first frame by client location, before and after* `PriceClass_All` 61
6.9 *Rebuffering ratio and average bitrate for three sources of differing visual complexity under three network profiles* 65
6.10 *A 25-minute video on the chunked path: time to READY and compute by chunk configuration and capacity* 69
6.11 *A four-hour video: time from the end of the upload to READY, by step, at 16 and at 8 vCPU* 70
6.12 *Four 25-minute videos submitted together: when each waited, encoded and finalised, at 8 and at 16 vCPU* 71

List of Tables

2.1 Comparison of transcoding approaches across platforms 7

3.1 Functional requirements of the system 14
3.2 Functional requirements of moderation and operation 15
3.3 Non-functional requirements of the system 16
3.4 Use case specification: upload a video 19

4.1 Jobs of the chunked path 25
4.2 Collections that support moderation and operation 27
4.3 Actions assigned to Rekognition labels by the screening policy 31
4.4 Outcome class of a human decision, given the machine's conclusion . . 33
4.5 Permissions of the four roles 34

5.1 Terraform modules and their responsibilities 44
5.2 Automated backend tests by area, 492 tests in 35 suites 52
5.3 Automated transcoder tests by area, 361 tests in 17 suites 53

6.1 Playback quality by emulated network profile 63
6.2 Declared against actual bitrate for each rendition 63
6.3 Playback quality before and after the manifest correction 64
6.4 Unthrottled playback at device pixel ratio one against two 66
6.5 Transcoding pipeline latency by input size 67
6.6 Single-container path against chunked path for videos of five to fifteen minutes 68
6.7 Simultaneous submissions at 16 vCPU 71
6.8 Cross-validation of the load test across two independent tools 73
6.9 Backlog drain under synthetic and real payloads 74
6.10 Monthly operating cost comparison for the part of the bill that the architecture controls (ap-southeast-1) 75
6.11 Non-functional requirements against the evidence 76

7.1 Objectives of the project against their results 78

A.1 Provenance of each measurement table 89

B.1 Time-to-First-Frame measurement results 91
B.2 Time-to-First-Frame through CloudFront after the price class correction 92
B.3 Time-to-First-Frame by client location, before and after `PriceClass_All` 92
B.4 Playback quality across three sources of differing visual complexity . . 93
B.5 A 25-minute video on the chunked path, by chunk configuration and capacity 94
B.6 A four-hour video at two capacities 94
B.7 Four 25-minute videos submitted together, by capacity 95

Chapter 1

Introduction

1.1 Background and Rationale

Video has become the dominant form of content on the Internet, and platforms such as YouTube, Vimeo, and TikTok have made uploading and sharing video an everyday activity. Behind that apparent simplicity lies a demanding engineering problem. When a user uploads a source file, that file may use any container format, any codec, and any resolution, yet the platform must deliver a stream that plays smoothly on every device and adapts to whatever bandwidth the viewer happens to have at that moment.

The industry solution to this problem is HTTP Live Streaming (HLS), a protocol that splits a video into short segments, typically between two and ten seconds long, and produces several quality renditions of the same content. A playlist file written in the `.m3u8` format enumerates the available renditions, and the player selects among them in real time according to measured network throughput. This mechanism, known as Adaptive Bitrate Streaming (ABR), is what allows playback to degrade gracefully rather than stall when a connection weakens.

Producing those renditions is computationally expensive, and this is where the architectural difficulty arises. Transcoding is a bursty workload: it consumes a large amount of CPU while a video is being processed and nothing at all when no one is uploading. A platform that keeps a dedicated server cluster or a Kubernetes system running continuously simply to wait for work therefore pays for a substantial amount of idle capacity. The alternative failure mode is equally undesirable: a system provisioned only for the average case cannot absorb a sudden burst when many users upload at the same time, and uploads either queue for a long time or are dropped altogether.

This tension between idle cost and burst capacity motivates the present project. Rather than maintaining permanently allocated compute, the system proposed here starts

a container only when there is a video to process and destroys it immediately afterwards, so that cost accrues only for the seconds during which transcoding actually runs.

1.2 Project Objectives

The project aims to build a complete video sharing platform for end users, together with the automated transcoding subsystem that supports it. Specifically, the objectives are:

1. To build a web platform that allows users to register an account, upload videos, watch them at multiple quality levels, and share them through public, unlisted or private links.

2. To automate the entire processing pipeline, from the moment a source file lands in Amazon S3 through transcoding, storage, and delivery via a content delivery network, until the video becomes ready for playback in a browser.

3. To perform transcoding with FFmpeg inside a Docker container running on AWS Batch and AWS Fargate, such that containers are created on demand and terminate automatically, thereby eliminating the cost of idle resources.

4. To automate software deployment and infrastructure management through a CI/CD pipeline and Infrastructure as Code.

5. To evaluate the resulting system empirically in terms of quality of experience, transcoding performance, scalability under load, and operating cost.

The first two objectives define what the platform does for its users, and the third defines how the processing is carried out and why, since the choice of on-demand containers is the central hypothesis of the project. The fourth makes the system reproducible and safe to change. The fifth turns the claims of the other four into measurements, which is why every performance figure in Chapter 6 comes from a script that is committed with the source code.

1.3 Scope and Deliverables

The scope of this project covers the full vertical slice of a video sharing service: the browser-based user interface, the backend API, the transcoding engine, the cloud infrastructure that hosts them, and the automation that deploys them.

Several areas are deliberately placed outside the scope. The project does not implement digital rights management systems such as Widevine or FairPlay, which would be required to prevent an authorised viewer from redistributing content they can legitimately access. It does not implement live streaming, since the architecture is designed around video on demand. It also does not attempt per-title bitrate ladder optimisation, using instead a fixed ladder of six rungs from 144p to 1080p, of which each video receives those that its source resolution can fill; the rationale and the trade-off involved are discussed in Chapter 7.

1.4 Research Methodology

The work combines a review of existing practice with an iterative build-and-measure cycle.

The theoretical component established how the problem is solved at industrial scale before any code was written. Published engineering accounts from Netflix describe how a large platform decomposes video processing into microservices and how a bitrate ladder is optimised per title, and the AWS Video on Demand reference architecture describes the managed-service equivalent. The protocol itself was taken from its specification, RFC 8216, and the encoding parameters from the FFmpeg HLS muxer documentation, rather than from secondary sources. Peer-reviewed studies of serverless video processing supplied both the motivation for the architecture and the measurement vocabulary used later in the evaluation, and the multi-threaded H.264 literature supplied the expected bound against which the vCPU scaling results of Chapter 6 are interpreted.

The practical component followed an incremental development model. Each subsystem, comprising the backend API, the transcoding container, the frontend, and the cloud infrastructure, was built and integrated in turn, with the whole environment described as Terraform code so that every change to it is reviewable and reproducible rather than applied by hand. Correctness was checked by an automated test suite executed on every push through a continuous integration pipeline, which also runs dependency, container and infrastructure security scans.

Evaluation was empirical rather than analytical. Every performance figure reported in Chapter 6 was produced by a measurement script committed alongside the source code, executed against the deployed system rather than a local approximation of it, with the raw output stored as JSON under `docs/results/` so that any table in this report can be traced back to the run that produced it; Table A.1 records that mapping. Where a result could plausibly be an artefact of the measurement tool, it was reproduced with a

second, independently implemented tool before being reported.

1.5 Practical Significance

The immediate beneficiaries of this design are small teams and individual creators who need a video platform but cannot justify a permanently running transcoding cluster. The cost comparison in Section 6.7 estimates that processing three hundred videos a month costs roughly $3.40 on the architecture built here against $133.35 on an always-on instance sized for the same work, because compute exists only while a job runs. That difference is what makes video hosting feasible for a service whose upload volume is intermittent, which describes most platforms before they become large.

The design also addresses a requirement that is easy to state and easy to implement incorrectly. Marking a video private is meaningless if the underlying storage remains publicly addressable, and Chapter 4 shows that restricting the API alone does not achieve it. The two-layer scheme used here, keeping the bucket private and enforcing authorisation at the content delivery network, is directly reusable by any application that serves user-owned media from object storage.

Beyond the product itself, the project serves as a worked reference for an event-driven serverless pipeline on AWS, covering the parts that are usually omitted from tutorials: how duplicate message delivery is made harmless, how the infrastructure is expressed as reviewable code across two environments, how security scanning is placed in the path of a merge rather than beside it, and how the resulting system is measured once it is running. Each of these is documented here in enough detail to be repeated.

1.6 Main Contributions

The principal contribution of this work is an end-to-end demonstration that a serverless container architecture can serve as the transcoding backbone of a video platform, together with an empirical assessment of what that choice costs and what it saves.

More concretely, the work contributes an event-driven pipeline in which an upload event propagates through Amazon S3, Amazon SQS, and AWS Lambda to AWS Batch without any polling service in between; a containerised FFmpeg worker whose conditional database writes make a duplicate delivery harmless, so that exactly-once *effects* are obtained without relying on exactly-once delivery; a two-layer content protection scheme combining Origin Access Control with CloudFront Signed Cookies so that private videos remain inaccessible even to a viewer holding the exact manifest URL; a chunk-parallel

transcoding path that splits a video of five minutes or more across concurrent jobs and brings a four-hour source to playback in 135 minutes at the production quota; a content moderation pipeline and a role-based console whose recorded decisions make the false positive and false negative rates measurable; a complete Infrastructure as Code description of the AWS environment in thirteen Terraform modules; and a DevSecOps pipeline whose security gate genuinely blocks merges rather than merely reporting findings.

1.7 Structure of the Report

The remainder of this report is organised as follows. Chapter 2 reviews how comparable platforms solve the transcoding problem and justifies the technology choices made here. Chapter 3 analyses the functional and non-functional requirements of the system and presents the use case model. Chapter 4 describes the architecture, the data model, the content protection scheme, and the interface design. Chapter 5 details the implementation of each subsystem, the infrastructure automation and continuous integration, the difficulties encountered during development and how they were resolved, and the automated test suite. Chapter 6 presents the deployed system and its user interface, then reports the experimental results for delivery latency, transcoding performance, behaviour under load, and operating cost. Chapter 7 summarises what was achieved, states the strengths and limitations of the delivered system, and outlines directions for future work.

Chapter 2

Literature Review and Technologies

2.1 Related Platforms and Comparison

Understanding how established platforms solve the transcoding problem provides the necessary context for the design decisions taken in this project.

Netflix offers the most thoroughly documented case. Its original video processing system, internally known as Reloaded, was a monolithic pipeline that proved difficult to evolve as requirements changed. Netflix subsequently rebuilt it as a set of independent microservices running on a platform called Cosmos, decomposing the workload into a Video Inspection Service, a Video Encoding Service, and a Video Quality Service [1]. Cosmos itself is layered: an API tier receives requests, a workflow tier orchestrates the processing steps, and a serverless compute tier performs the actual media work, with the three tiers communicating asynchronously through a priority-based messaging system [2]. Two aspects of this design are directly relevant here. First, Netflix also relies on asynchronous messaging rather than synchronous calls to decouple job submission from job execution, which is precisely the role played by Amazon SQS in this project. Second, Netflix moved from linear encoding to distributed chunk-based encoding, allowing hundreds of machines to encode segments of the same title in parallel and reducing processing latency from days to hours. This project applies the same principle at a far smaller scale: for videos of five minutes or longer, a planner job divides the source into chunks that an array of AWS Batch jobs encodes concurrently and a finalizer job merges, with the degree of parallelism bounded by the Fargate vCPU quota rather than by a fleet of dedicated machines (Section 6.5).

Amazon Web Services publishes a reference architecture for video on demand that provides a useful point of comparison at the level of managed services [3]. That architecture routes an upload from Amazon S3 through an AWS Lambda job submitter into AWS Elemental MediaConvert, stores the output in a second S3 bucket, and delivers it through Amazon CloudFront, with AWS Step Functions orchestrating the workflow. The architecture presented in this report follows the same overall shape but substitutes a self-packaged FFmpeg container on AWS Batch for MediaConvert. This substitution is the central engineering trade-off of the project: MediaConvert is a fully managed service billed per minute of output and requires almost no operational effort, whereas a self-managed container is considerably cheaper per unit of work but transfers the responsibility for image maintenance, dependency updates, and failure handling onto the development team. AWS documentation also confirms that Origin Access Control is the currently recommended pattern for securing an S3 origin, superseding the legacy Origin Access Identity mechanism, which validates the approach adopted in Section 4.4.

Table 2.1 summarises the comparison across the dimensions most relevant to this work.

Table 2.1: Comparison of transcoding approaches across platforms

ASPECT	**NETFLIX (COSMOS)**	**AWS VOD REFERENCE**	**THIS PROJECT**
Compute model	Dedicated microservice platform with serverless compute tier	Managed service (MediaConvert)	Serverless container on AWS Batch / Fargate
Parallelism	Chunk-based, distributed across many nodes	Managed internally by the service	Chunk-based on AWS Batch array jobs from five minutes of video, one container below that
Bitrate ladder	Per-shot optimisation driven by VMAF	Configurable job template	Fixed ladder: six rungs from 144p to 1080p, limited by the source
Queueing	Priority-based messaging system	AWS Step Functions	Amazon SQS with dead letter queue
Content protection	DRM	Signed URLs or signed cookies	OAC plus CloudFront Signed Cookies
Operational effort	Very high	Very low	Moderate

2.2 Core Models and Principles

2.2.1 Serverless Containers

The term serverless container describes a model in which a container image is executed without the developer provisioning or managing the underlying virtual machines, and in which billing is based on execution time rather than allocated capacity. AWS Fargate is the implementation used in this project.

Figure 2.1 contrasts this model with the conventional alternative for a transcoding workload of the size assumed in Section 6.7. In the always-on arrangement on one side of the figure, a cluster is sized for the busiest moment the service expects and then runs continuously, so it is billed for all 730 hours in a month while performing roughly 105 vCPU-hours of actual encoding out of the 2,920 that a four-vCPU instance offers, a utilisation of under four percent. Its capacity is also a hard ceiling: an upload burst larger than the cluster was provisioned for must either wait or be rejected, and the only remedy is to provision for a still larger peak and waste correspondingly more. In the serverless arrangement on the other side, no compute exists between uploads. An upload event enters a queue, a container is started to serve it, and that container terminates when the job finishes, so charges accrue per vCPU-second of real work. The burst is absorbed by the queue rather than by spare capacity, and the degree of parallelism is a configured number rather than a fixed fleet size. The two figures shown, $124.10 against $0.05 of monthly compute, are the same workload priced under the two models.

The choice of Fargate over a function-as-a-service platform such as AWS Lambda deserves justification, because Lambda is the more common answer to serverless workloads. Video transcoding is unsuitable for Lambda in this context for two reasons. The first is duration: transcoding a large file routinely exceeds the maximum execution time of a Lambda invocation. The second concerns resource configuration. A measurement study of serverless video processing found that selecting a cost-efficient memory configuration for transcoding is far from trivial, because the optimal setting depends not only on the type of task but even on the content of the video itself, and the cost difference between the least and most generous memory configuration reached 498% for sequential workloads [4]. Fargate avoids this difficulty by allowing vCPU and memory to be specified explicitly and independently.

A known weakness of the container model is cold start latency. A systematic review of the subject reports that more than half of function invocations spend at least 80% of their startup time pulling the container image [5]. This finding directly motivates the use of a multi-stage Docker build in this project, described in Section 5.2, whose purpose is

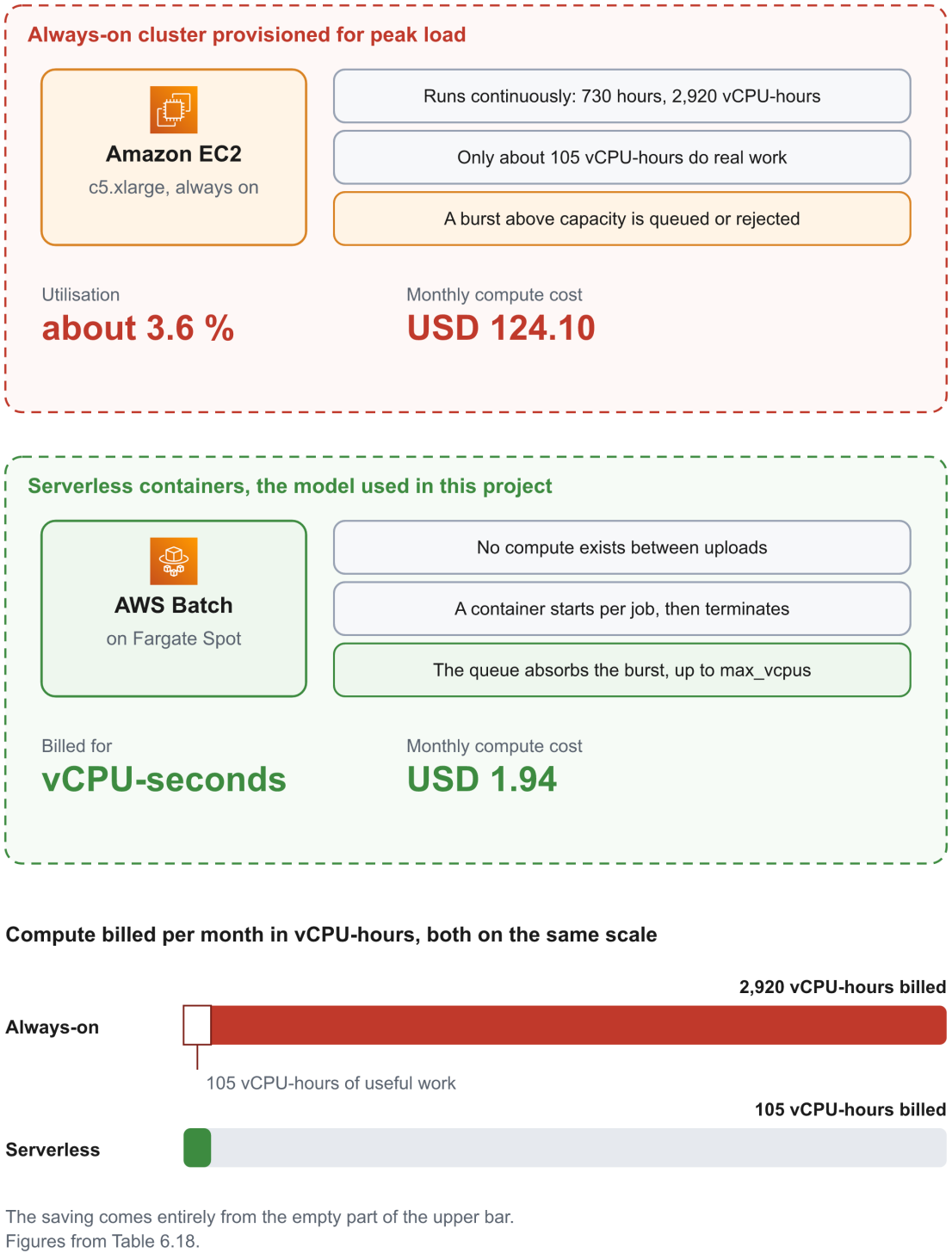


Figure 2.1: *Compute models for a bursty transcoding workload: an always-on cluster compared with on-demand serverless containers*

to minimise the size of the image that must be transferred before work can begin.

2.2.2 Event-Driven Architecture

An event-driven architecture propagates work through the emission and consumption of events rather than through direct synchronous invocation. In this project, the completion of an upload to Amazon S3 emits an event notification that is placed on an Amazon SQS queue, from which an AWS Lambda function submits a job to AWS Batch.

The queue is not merely a routing mechanism; it is the component that makes burst absorption possible. If a hundred users upload simultaneously, the queue retains all hundred messages while AWS Batch works through them at whatever rate the compute environment allows. Without this buffer, the system would have to either reject work or provision for peak load permanently. The queue is paired with a dead letter queue so that messages which repeatedly fail processing are isolated for inspection rather than being retried indefinitely.

2.2.3 Adaptive Bitrate Streaming and the Bitrate Ladder

HLS delivers a video as a master playlist referencing several rendition playlists, each of which lists the media segments for one quality level. The player measures throughput and switches between renditions at segment boundaries.

Figure 2.2 shows the artefacts this produces and the order in which a player retrieves them. A single source file of arbitrary container, codec and resolution is encoded once per rung of the ladder, and each rung yields both a rendition playlist and the sequence of transport stream segments it indexes. The master playlist sits above them and carries, for each rung, the bandwidth the client should expect to need in order to play it. A player therefore begins by fetching only the master playlist, which is a few hundred bytes, chooses a rung from the declared bandwidths and its own throughput estimate, and only then requests that rung's playlist and its segments. Because every rung is segmented on the same boundaries, the player may change its choice at any segment boundary without interrupting playback, which is the mechanism that lets quality fall rather than the video stall when a connection weakens. The consequence for the transcoding subsystem is that the cost of preparing a video scales with the number of rungs, since each rung is encoded separately even though the source itself is decoded only once. The implementation exploits exactly that distinction, as Section 5.2 describes.

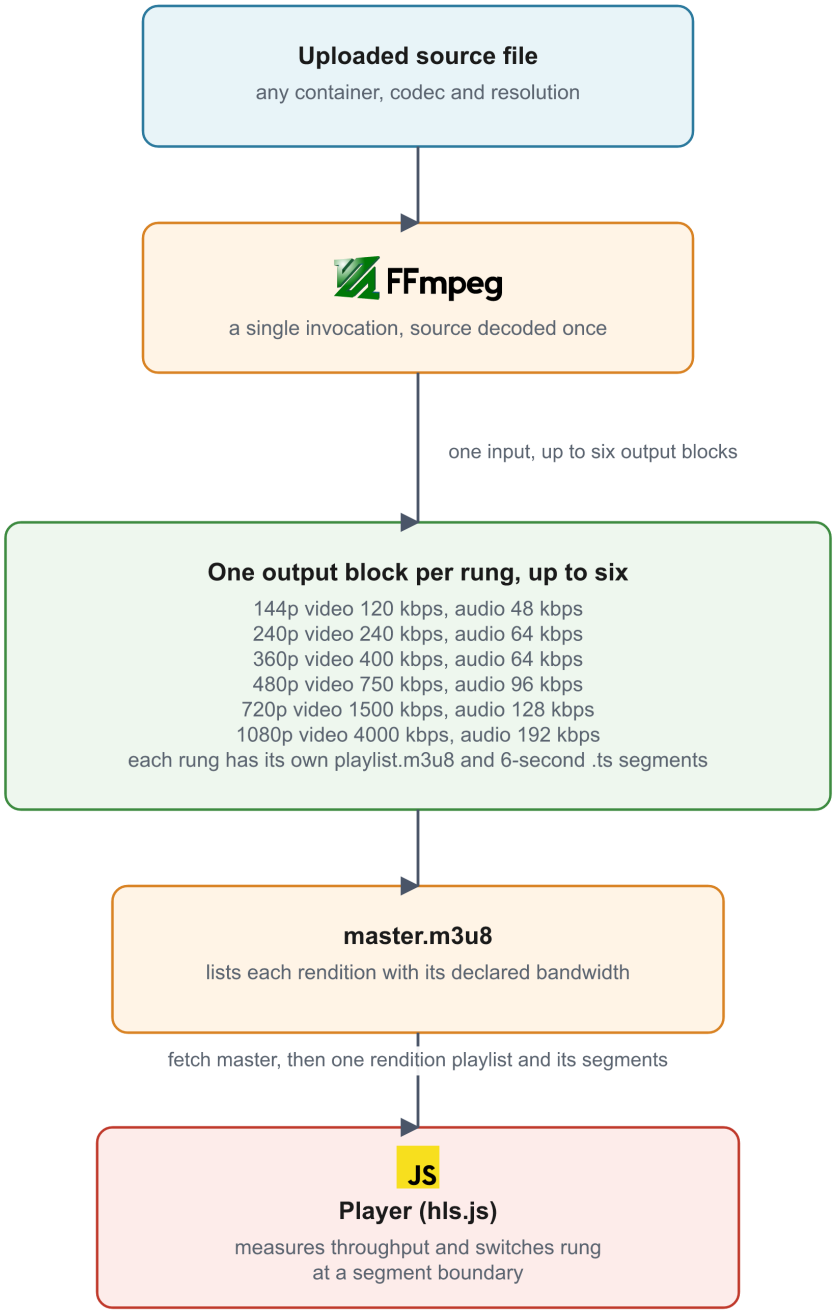


Figure 2.2: *HLS artefacts produced from one source file and the order in which a player retrieves them*

The set of resolution and bitrate pairs offered is called the bitrate ladder, and its construction is an active research area. Netflix demonstrated that the optimal ladder is not fixed but content-dependent, and that the ideal operating points lie on the convex hull of the quality-versus-bitrate curve [6]. By encoding each shot separately under the control of the VMAF perceptual quality metric, Netflix reported bitrate reductions of

between 28% and 38% at equivalent visual quality [7]. The present project uses a fixed ladder of six rungs, which is simpler to implement and reason about; the implications of this simplification are revisited in Section 7.4.

2.3 Technology Stack Justification

The frontend is implemented as a React single page application built with Vite, using HLS.js for playback. HLS.js is necessary because most desktop browsers do not support HLS natively; it parses the manifest in JavaScript and feeds segments to the browser through the Media Source Extensions API. Safari, which does support HLS natively, is handled through a fallback path.

The backend is built on Node.js with Express. The decisive factor is that the backend's workload is almost entirely input/output bound rather than CPU bound: it issues pre-signed URLs, queries MongoDB, and returns JSON, while the heavy computation is deliberately delegated to the transcoder containers. Node.js's event loop is well suited to this profile.

MongoDB Atlas provides the persistence layer. Video metadata is naturally document-shaped and its schema evolves as features are added, which suits a document store better than a rigid relational schema. Using the managed Atlas service also removes the need to operate a database cluster inside the project's own infrastructure.

Terraform was selected for Infrastructure as Code over alternatives such as AWS CloudFormation because its module system allows the infrastructure to be decomposed into reusable units with explicit inputs and outputs, and because its plan step makes the consequences of a change visible before it is applied. GitHub Actions was selected for CI/CD because the source code is already hosted on GitHub, which removes the need for a separate automation server.

Chapter 3

Requirements Analysis

3.1 Stakeholders and User Requirements

Four groups interact with the system, and their differing needs shape the requirements.

The *anonymous visitor* arrives without an account. This group needs to browse public videos, search and filter by category, and watch content immediately, because requiring registration before any content can be seen would deter adoption. The *registered user* additionally needs to upload videos, control whether each video is public, unlisted or private, manage the videos on their own channel, and interact through likes and comments. The *system administrator*, in the context of this project the development team itself, needs the platform to be observable and deployable without manual intervention. Finally, the *operations staff* decide whether uploaded content may be shown: they need a queue of the videos that the automatic screening could not settle, a way to hear an owner's appeal, and figures that tell them how often the screening is wrong.

3.2 Functional Requirements

Table 3.1 enumerates the functional requirements derived from the stakeholder analysis. Each requirement is given an identifier so that it can be traced through the design and implementation chapters.

Table 3.1: Functional requirements of the system

ID	REQUIREMENT	DESCRIPTION
FR-01	Registration and login	Users register with a username, email address, and password. Authentication uses JSON Web Tokens carried in the Authorization header. A new account must prove that it can receive mail at its address before it may upload videos; signing in with Google counts as that proof. A new password, at registration, reset or change, is refused when it is a commonly used one, contains the user's own details or appears in a known breach.
FR-02	Password recovery	Users request a reset link by email; the token is single-use and expires after fifteen minutes. A completed reset also confirms the address.
FR-03	Direct upload to S3	The client requests a pre-signed URL from the backend and transfers the file directly to Amazon S3 without routing the payload through the application server. A file above 512 MiB is transferred in parts of 32 MiB, so that a part that fails is retried without repeating the whole file.
FR-04	Automatic transcoding	Once the upload completes, the system transcodes the source into HLS renditions without any manual trigger.
FR-05	Adaptive playback	The player switches automatically among the renditions produced for the video, chosen from six rungs between 144p and 1080p according to the source resolution, as the available bandwidth changes.
FR-06	Visibility control	Each video is public, unlisted, or private. A public video is listed in the catalogue; an unlisted one is reachable only by its direct link and appears in no listing; a private one is visible to its owner alone. The owner can change the setting after upload.
FR-07	Channel management	Users view their own uploads, manage visibility, update video details, and delete a video together with all of its stored artefacts.
FR-08	Search and filtering	Users search by title, description, or tags, and filter by category.
FR-09	Engagement	Users express likes and dislikes and post comments; a comment may be removed by its author or by the video owner.
FR-10	View counting	The system records a view only after playback has genuinely started, and does not count repeated views from the same viewer within a short window.
FR-11	Related videos	The watch page suggests further videos drawn from the same category and the same uploader. A suggestion is subject to the same visibility rule as the video itself, so the list never reveals one the viewer could not open directly.

Table 3.2 continues the list with the requirements of moderation and operation, which Section 4.5 designs.

Table 3.2: Functional requirements of moderation and operation

ID	REQUIREMENT	DESCRIPTION
FR-12	Automatic screening	Before a video is published, the system samples frames from it and classifies them; a video with clearly violating content is removed automatically, and one with doubtful content is held back for review. Screening that cannot be completed holds the video back and does not publish it.
FR-13	Viewer reports	A signed-in viewer can report a video for a reason chosen from a short list, and has at most one open report per video.
FR-14	Staff review	Operators approve or remove the held-back and reported videos, and every decision carries a reason from a closed list.
FR-15	Appeals	The owner of a removed video can appeal once per decision, an operator resolves the appeal with a note, and a person other than the owner cannot learn that the video exists.
FR-16	Accountability	Every decision is recorded together with what the automatic screening had concluded, so that the rates of false positives and false negatives can be estimated, the latter from a random audit sample, and an auditor can read them without being able to change a decision.
FR-17	Role management	An administrator assigns the roles operator, auditor and administrator, cannot change their own role and cannot demote the last administrator; every change is recorded.

Requirement FR-03 deserves particular comment because it shapes the architecture more than any other functional requirement. Routing a multi-gigabyte upload through the backend would consume server memory and bandwidth proportional to the number of concurrent uploads and would make the API server the bottleneck of the entire platform. Issuing a time-limited pre-signed URL and letting the browser transfer the file directly to Amazon S3 removes that bottleneck entirely: the backend handles only a small JSON request regardless of file size.

3.3 Non-Functional Requirements

The non-functional requirements are stated in measurable terms wherever possible, so that Chapter 6 can report whether they were met; they are listed in Table 3.3.

Table 3.3: Non-functional requirements of the system

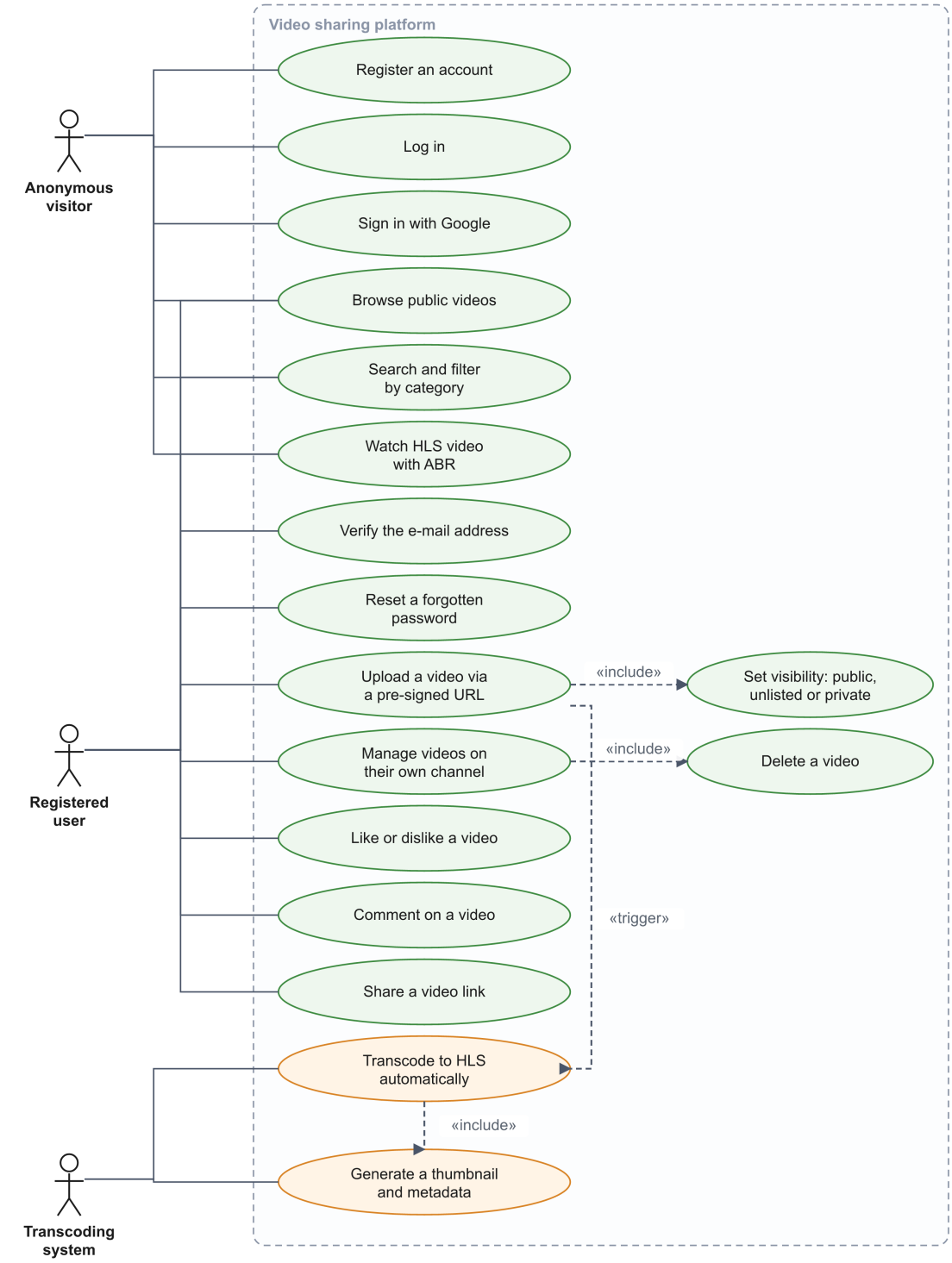
ID	ATTRIBUTE	REQUIREMENT
NFR-01	Latency	Time-to-First-Frame should remain low enough that playback feels immediate; the measurement methodology is defined in Section 6.3.
NFR-02	Scalability	The pipeline must absorb a burst of concurrent uploads without losing work, processing them as capacity permits rather than rejecting them.
NFR-03	Reliability	A failed transcoding job must not silently disappear: it must be retried where retrying could succeed, and must raise an alert carrying the reason when it cannot. The retry and alerting mechanisms that satisfy this requirement on the deployed path are described in Section 5.5.
NFR-04	Cost efficiency	Compute cost must accrue only during actual transcoding, with no charge for idle capacity.
NFR-05	Security of content	Private videos must be inaccessible to unauthorised viewers even when the exact manifest URL is known.
NFR-06	Security of the API	Authentication endpoints must resist brute-force attempts, and the origin of cross-origin requests must be restricted to an explicit allow-list.
NFR-07	Reproducibility	The entire infrastructure must be reproducible from source control through a single command.
NFR-08	Maintainability	Changes must pass automated tests and security scanning before reaching the main branch.

Requirement NFR-05 is worth distinguishing from ordinary access control. Restricting an API response is straightforward, but a video is not delivered by the API: it is delivered by a content delivery network as hundreds of individual segment files. An access control scheme that protects only the API therefore leaves the actual content unprotected, a gap that Section 4.4 addresses explicitly.

3.4 Use Case Model

Figure 3.1 presents the use cases of the people who use the platform. Three actors appear: the anonymous visitor, the authenticated user, and the transcoding system itself, which is modelled as an actor because it initiates processing autonomously in response to events rather than in response to a direct user action. Verifying the e-mail address is the use case that lets a new account upload (requirement FR-01).

The diagram makes visible an important property of the design: the transcoding use cases are not invoked by the user at all. The user's upload action triggers them indirectly through the event chain, which is why they are attached to the system actor rather than

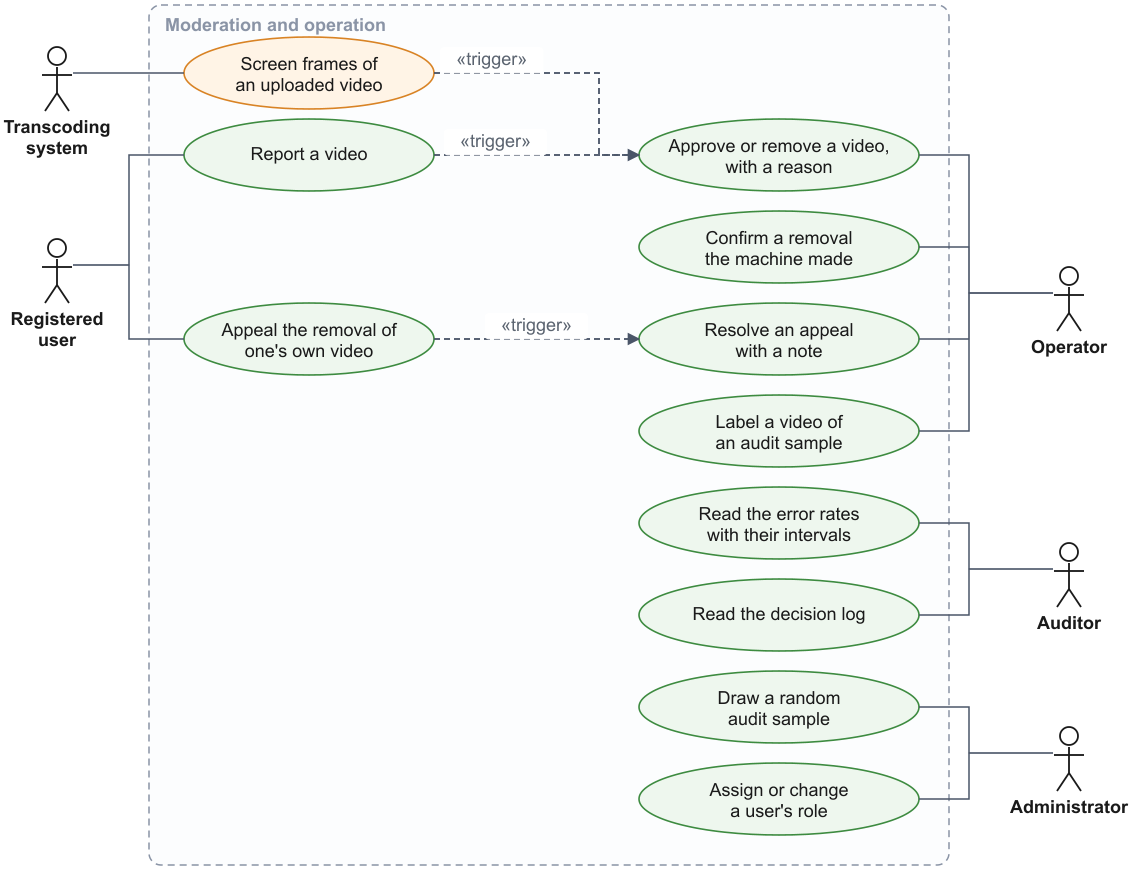


Association lines carry no arrowhead, as UML specifies. Only «include» and «trigger» relationships are directed. The transcoding use cases attach to the system actor because events start them, not a human action.

Figure 3.1: *Use case diagram of the video sharing platform*

to the human actors. This separation is what allows the browser to return control to the user immediately after the upload completes, without waiting for processing to finish.

Figure 3.2 adds the use cases of moderation and operation, which are requirements FR-12 to FR-17 (Table 3.2), designed in Section 4.5. They are drawn apart because adding them to Figure 3.1 would make that figure taller than a page. Three more actors appear, the roles operator, auditor and administrator. The use cases of each role stand together, and an administrator can also do everything an operator or an auditor can, which is why each association is drawn once. The transcoding system keeps its place as an actor because the screening runs inside the transcoder. Three things lead to a decision that only staff can take: what the screening holds back, a report from a viewer, and an appeal from the owner of a removed video.



Reporting and appealing are open to every signed-in user. An administrator can also do everything an operator or an auditor can, so each association is drawn once, at the role that owns the use case. Screening acts on its own; what it holds back, and what viewers report, reaches an operator for a decision.

Figure 3.2: *Use case diagram of moderation and operation*

Table 3.4 specifies the most architecturally significant use case in detail.

Table 3.4: Use case specification: upload a video

FIELD	**CONTENT**
Identifier	UC-07
Actor	Authenticated user
Precondition	The user holds a valid JSON Web Token and a verified e-mail address.
Trigger	The user selects a video file and submits the upload form.
Main flow	1. The client validates the file type and size locally. 2. The client calls the initiate-upload endpoint with the file metadata. 3. The server validates the metadata, creates a database record with status `UPLOADING`, and returns a pre-signed URL valid for fifteen minutes. 4. The browser transfers the file directly to the raw S3 bucket. 5. The client confirms completion and the record moves to `PROCESSING`. 6. The transcoding pipeline runs asynchronously and eventually sets the record to `READY`.
Alternative flow	If the file type is not an accepted video format or the size exceeds the limit, the server rejects the request before issuing a URL, and no orphan record is created. For a file larger than 512 MiB, step 3 returns a URL for each part of 32 MiB instead, and the server checks the stored parts before the object is assembled. If the e-mail address is not verified, the server refuses the request with the code `EMAIL_NOT_VERIFIED` and the interface offers to send the link again.
Postcondition	The video appears on the user's channel, initially as processing and subsequently as ready to play.

Chapter 4

System Design

4.1 High-Level Architecture

The system is organised into four tiers, shown in Figure 4.1: a client tier running in the browser, a delivery tier built on Amazon CloudFront, an application tier comprising the backend API and the database, and a processing tier that performs transcoding.

The separation between the application tier and the processing tier is the defining characteristic of this design. The backend API never touches video content: it issues pre-signed URLs, records metadata, and answers queries, while every byte of video flows directly between the browser, Amazon S3, and CloudFront. This separation has three consequences that together justify the architecture. The API server's resource consumption becomes independent of video size and upload concurrency. The processing tier can scale according to transcoding demand without affecting the responsiveness of the interface. And a failure in the processing tier degrades the system gracefully, leaving already-published videos playable while newly uploaded ones remain in a pending state.

Two later additions sit inside this structure and do not change it. Content moderation runs in the processing tier, as the last step before a video is published, so that the check neither delays the upload nor adds a component to operate (Section 4.5). And the console that human reviewers use is a role-restricted part of the same single page application and the same backend API, not a separate system.

4.2 Transcoding Pipeline Design

Figure 4.2 traces the sequence of interactions from the moment the user initiates an upload until the video becomes available.

Two design decisions embedded in this sequence merit explanation.

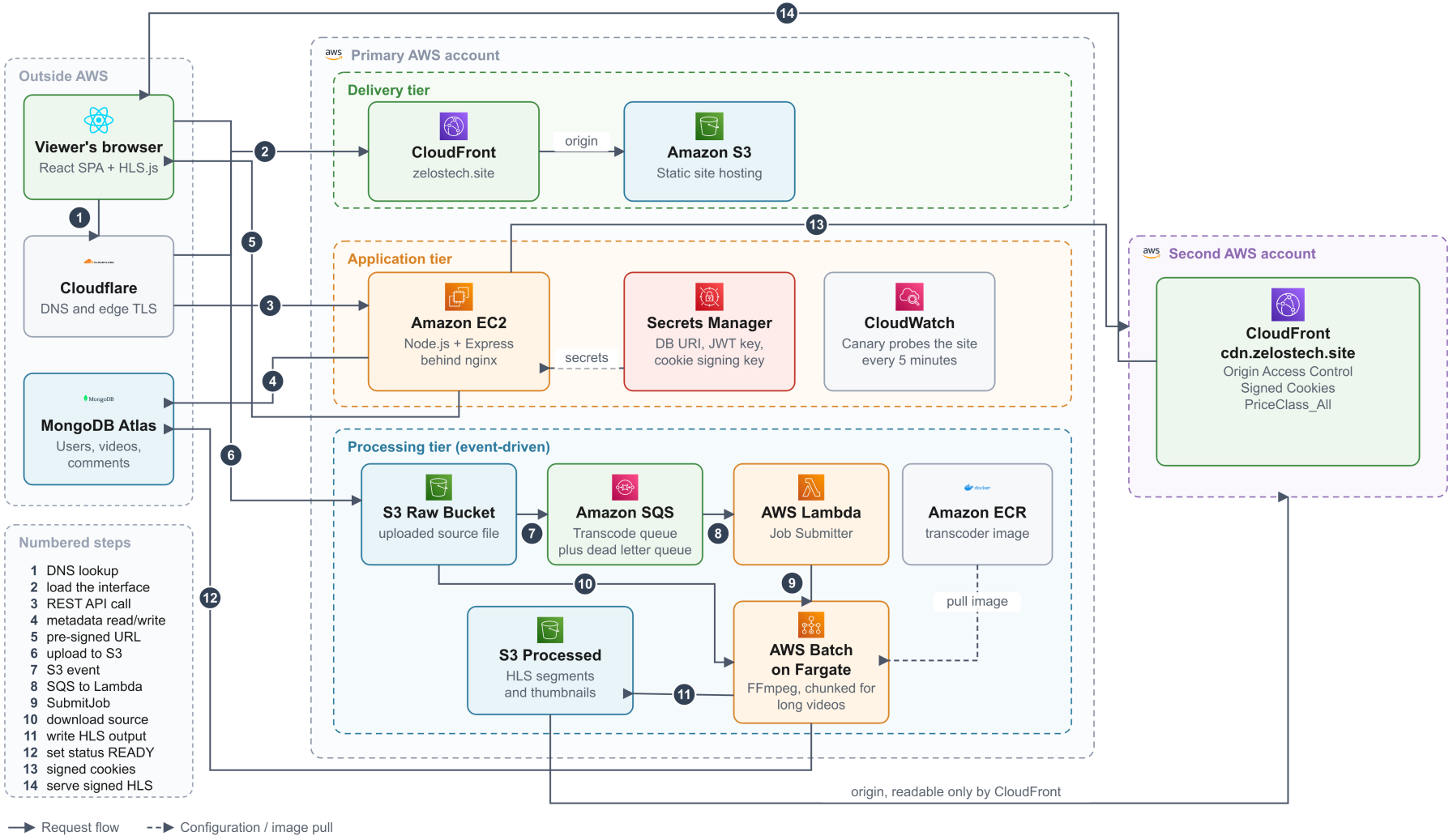


Figure 4.1: *Overall system architecture, showing the deployment boundaries and the numbered data flow from an anonymous visit through upload, transcoding and signed delivery; the key at the lower left names each step*

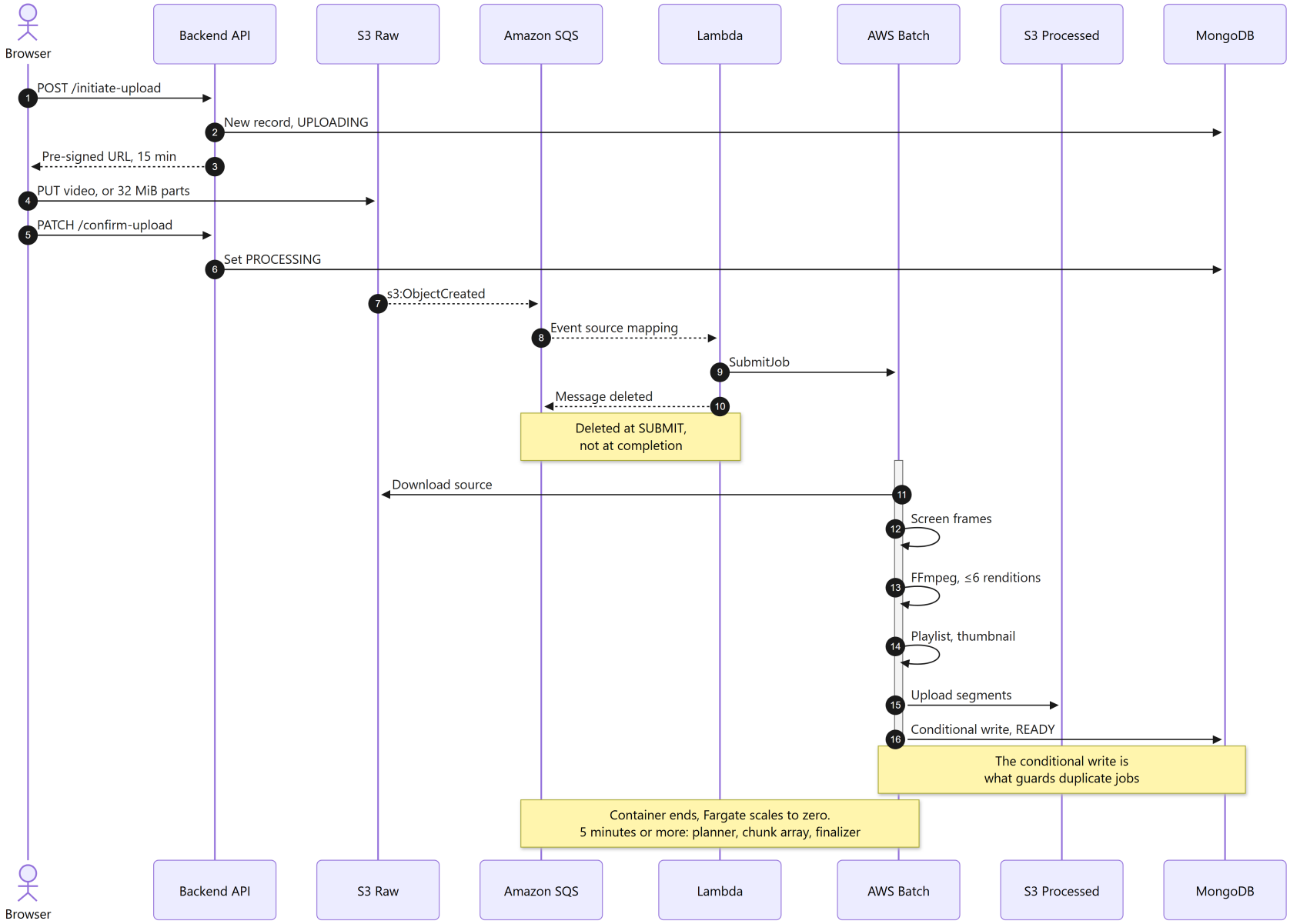


Figure 4.2: *Sequence diagram of the event-driven transcoding pipeline for one container; a video of five minutes or more takes the chunked path of Table 4.1*

The first concerns the ordering of database record creation and URL issuance. The backend creates the database record *before* generating the pre-signed URL, and derives the S3 object key from the identifier of that record. The alternative, generating a key first and creating the record afterwards, introduces a race condition: the S3 event notification can arrive and trigger transcoding before the record exists, at which point the transcoder has no row to update and the job fails. Creating the record first eliminates this class of failure entirely.

The second concerns the visibility timeout of the SQS message. Amazon SQS hides a message from other consumers for a configured period after it is received, and redelivers it if the consumer has not deleted it by then. Transcoding a large file can easily exceed any reasonable fixed timeout, which would cause the same video to be processed twice concurrently. The transcoder therefore implements the heartbeat pattern: while FFmpeg runs, a timer periodically extends the visibility timeout of the in-flight message. The message is deleted only after the job completes successfully. If the container crashes, the heartbeat stops, the timeout lapses, and the message returns to the queue for another attempt.

That description is accurate about the transcoder's `worker` mode, in which the container consumes from the queue itself, but it is important to be precise that this is not the mode the deployed system runs. The container image and the AWS Batch job definition both invoke `batch` mode, in which the video identifier and object key arrive as environment variables and the container never contacts SQS at all. The queue is consumed instead by the Lambda submitter, which reads the message, submits a Batch job, and returns immediately. Because the Lambda's work is finished at that point, the message is deleted when the job is *submitted*, not when transcoding completes.

The consequence is that on the deployed path there is no heartbeat, and the message is long gone before FFmpeg starts. Two properties the pattern was chosen to provide therefore have to be obtained elsewhere. Protection against duplicate processing rests entirely on the conditional database writes described in Section 5.2, which is where it belongs in any case, since Burrows' analysis of leases (Section 5.6) shows that a renewal timer cannot by itself guarantee mutual exclusion. Retrying a failed job, which the heartbeat design delegated to SQS redelivery, has no equivalent unless AWS Batch is configured to provide it; that it originally was not is discussed in Section 5.5.

Retaining the worker-mode implementation is a deliberate choice rather than an oversight: it allows the transcoder to be run against the queue directly during development without provisioning Batch. Recording which of the two modes the evaluated system actually executes matters, though, because a mechanism present in the source but absent

from the deployed path cannot be credited with the guarantees it would provide if it ran. Section 5.5 draws the same distinction about content protection.

4.2.1 Upload of Large Files

The sequence of Figure 4.2 uses one pre-signed request, which suits files up to a few hundred megabytes. For larger files the design switches to a multipart upload, because a single transfer that fails near its end must be repeated in full and Amazon S3 accepts at most 5 GiB in one request. The backend chooses the path from the declared size: a file of up to 512 MiB is sent with one pre-signed POST, and a larger file is divided into parts of 32 MiB.

Both paths keep the property that motivated direct upload, that the payload never passes through the application server, and both keep the limits enforced by the storage service and not by the client. For the single request, the signed policy fixes the object key, the content type and the size range. For a multipart upload the backend signs one URL per part for exactly that part's length, so S3 itself refuses a part of any other size, and when the client reports the upload complete the backend lists the parts that S3 holds and checks them before it asks S3 to assemble the object. The ceiling on the size of a file is a configuration value, with a hard limit of 20 GiB in the source, which is 640 parts of 32 MiB, and 20 GiB in the deployed system. An upload that is abandoned is cleaned up in two ways: a lifecycle rule aborts incomplete multipart uploads after two days, and a periodic reconciler in the backend removes a draft that has stayed in the `UPLOADING` state for more than a day.

4.2.2 Chunk-Parallel Transcoding of Long Videos

The pipeline of Figure 4.2 gives each video to one container. That is adequate for short videos and fails for long ones: on a single vCPU, FFmpeg ran roughly four to five times slower than real time in the measurements of Sections 6.4 and 6.5, so a long source would exceed the two-hour limit placed on a job however many vCPUs the account holds. The design therefore divides a video of five minutes or more into chunks that are encoded concurrently, and keeps the single-container path for shorter videos, where splitting costs more than it saves.

The job that the Lambda submitter starts becomes a planner. It reads the structure of the source over HTTP without downloading it and divides the video at boundaries that are whole multiples of a six-second group of pictures, counted in frames so that a rate such as 29.97 frames per second does not accumulate a fractional error. It then submits

the jobs of Table 4.1 to the same AWS Batch service.

Table 4.1: Jobs of the chunked path

JOB	COUNT	TASK SIZE	RESPONSIBILITY
Planner	1	Default	Probes the source, computes the chunk boundaries, and submits the other jobs
Audio	2	Default	Encodes the audio once for the whole video, one job per bitrate (64 and 128 kbit/s)
Chunk	One per chunk, as an array job	4 vCPU, 8 GiB	Reads its own range of the source over HTTP and encodes every rendition of the ladder for that range
Finalizer	1	Default	Runs after every chunk has ended; joins the playlists, measures the bandwidth of each rendition from the segments, extracts the thumbnail, screens the content, and sets `READY`

Three properties of this design are deliberate. The chunk array goes to a second, lower-priority job queue that shares the Fargate Spot environment of the main queue, so that a short video submitted while a long one is encoding is scheduled ahead of its remaining chunks and does not wait for all of them. The number of chunks follows the number of tasks that can run at once, about three chunks per slot and none shorter than about a minute, so a larger quota produces smaller chunks. And the planner declines to split a source that cannot be split safely, which means a variable frame rate, an unknown duration, or audio and video that differ by more than 0.25 seconds; such a video falls back to the single-container path and is not failed.

While a chunked video is encoding, its child jobs periodically touch the `updatedAt` field of the video. The reconciler that returns a video stuck in `PROCESSING` for more than six hours to a consistent state would otherwise mistake a long and healthy encode for a stuck one. How this path behaves, including where adding vCPUs stops helping, is measured in Section 6.5.

4.3 Database Design

The persistence layer uses MongoDB Atlas with eight collections. Three of them, `User`, `Video` and `Comment`, model the platform itself and are shown in Figure 4.3. The other five exist to moderate and operate it, and are described after the state machine of the video.

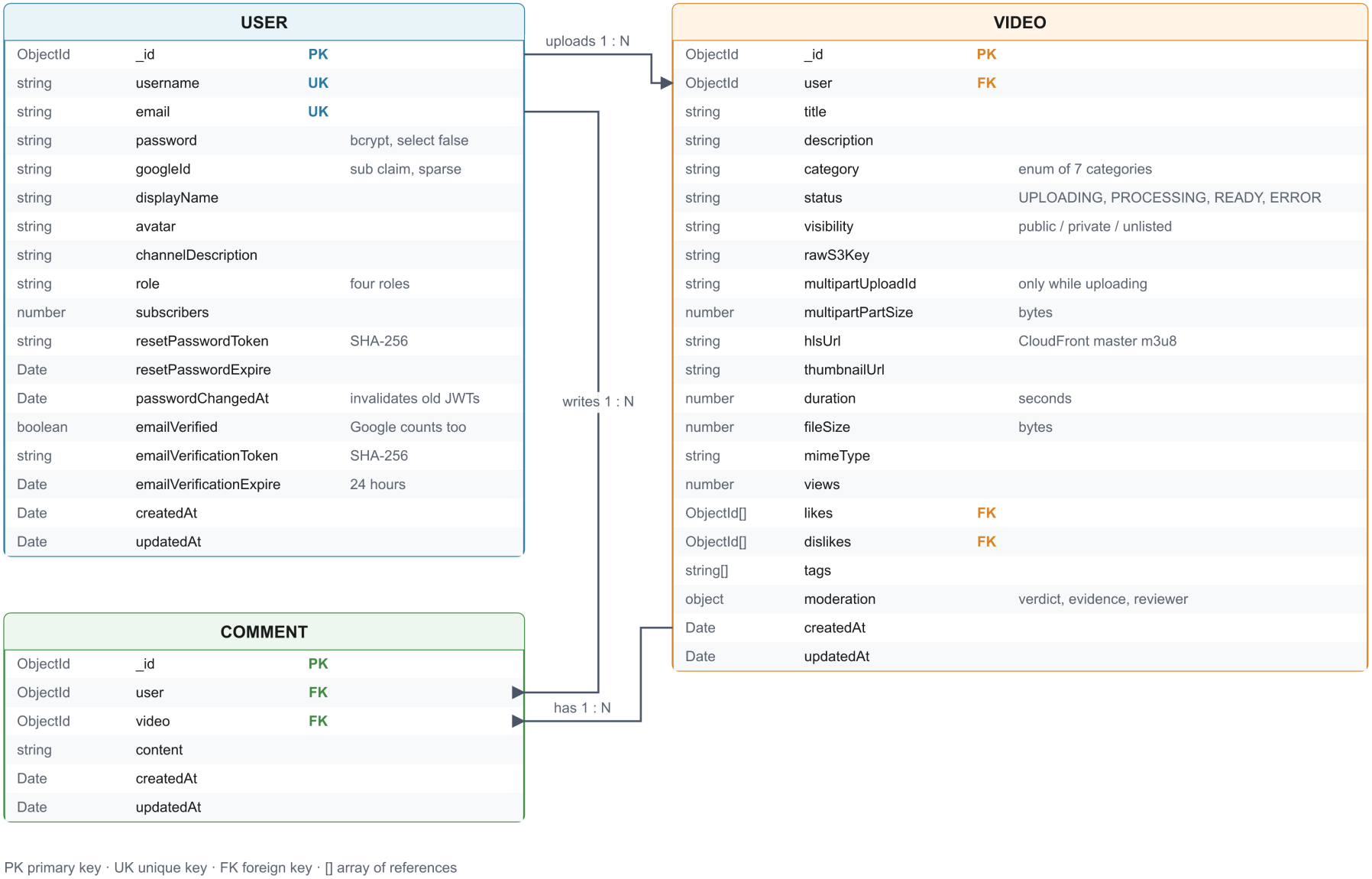


Figure 4.3: *Entity relationship diagram of the core MongoDB collections*

The Video document carries a status field that acts as a state machine with four values. A record begins as UPLOADING when the pre-signed URL is issued, moves to PROCESSING once the upload is confirmed and the job is submitted, and finally reaches either READY or ERROR. Modelling the lifecycle explicitly rather than inferring it from the presence of other fields makes the interface straightforward: the client simply renders a different state for each value, and a video that failed to transcode is distinguishable from one that is merely slow.

A second state, independent of the first, lives in the moderation sub-document of Video: the verdict (approved, flagged or blocked), whether it came from the machine or from a person, the labels that were its evidence, and, written exactly once and before any person first changes the verdict, autoStatus, the machine's own conclusion (Section 4.5). The five collections that support moderation are summarised in Table 4.2 and shown in Figure 4.4.

Table 4.2: Collections that support moderation and operation

COLLECTION	ONE DOCUMENT IS	CONSTRAINT
Report	A viewer's report of a video	At most one open report per viewer and video, by a partial unique index
ModerationEvent	One decision by a person	Append only: no route edits or deletes it
Appeal	An owner's appeal against a removal	One per video and decision time, by a unique index
AuditSample	A video drawn into a random audit	A video enters at most once, by a unique index on the video
RoleChange	A change of one user's role	Append only: who changed whom, from which role to which

Likes and dislikes are stored as arrays of user identifiers embedded within the video document rather than as a separate collection. The denormalisation is justified at this scale because the counts are always read together with the video, and because a single user can appear at most once in each array, so the arrays cannot grow through repeated interaction by the same person.

It should not be justified more strongly than that. Uniqueness bounds each array by the number of distinct users, which is a bound only in the formal sense; on a widely viewed video it is exactly the growth that a document store is poorly suited to hold, against MongoDB's document size limit. The cost is also paid on reads that do not want it, since a listing of twelve videos retrieves every identifier in all twenty-four arrays merely to display counts. The conventional remedy is a separate collection with a compound unique index on video and user, plus a denormalised counter on the video. That is the

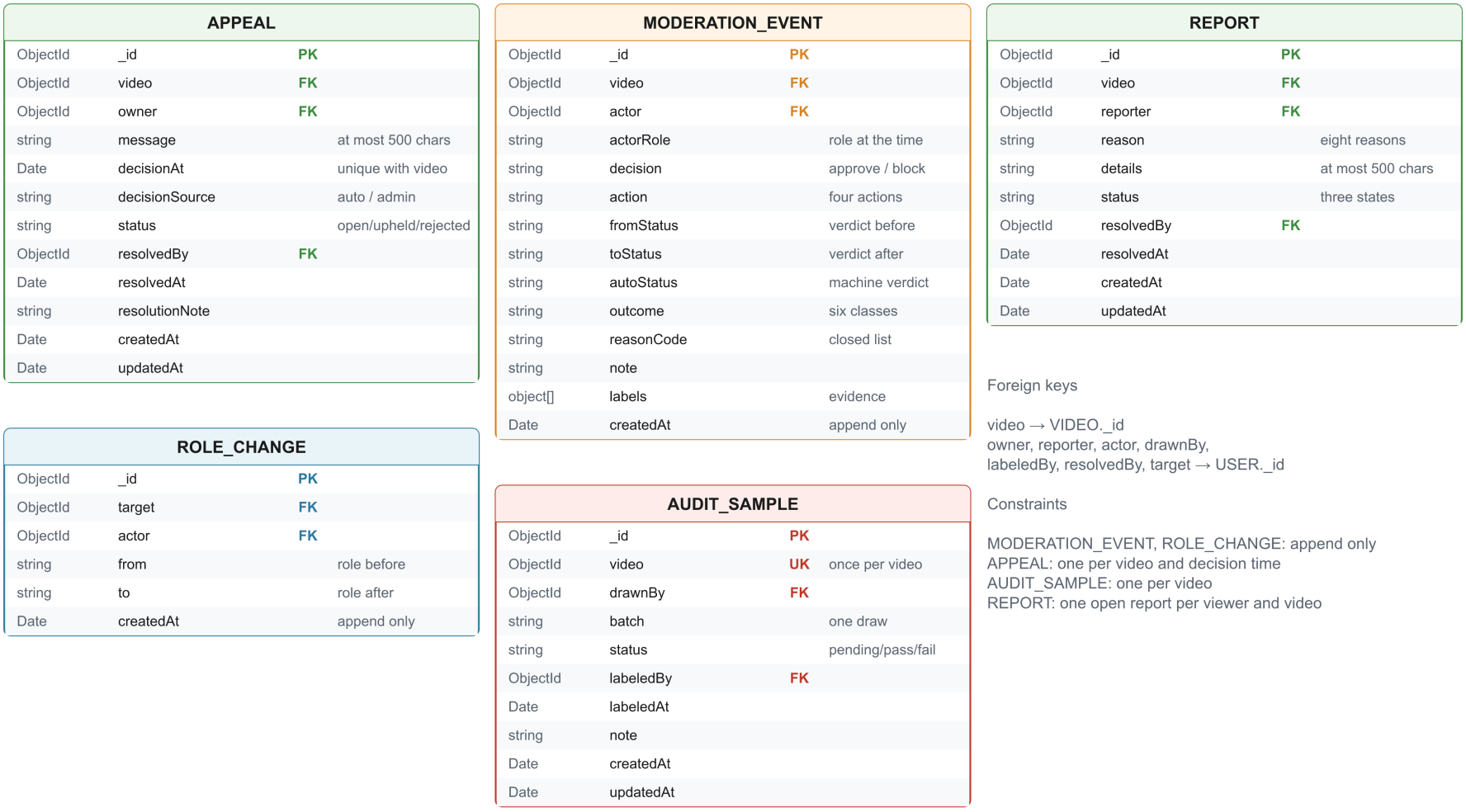


Figure 4.4: *Entity relationship diagram of the collections that support moderation and operation; every video field refers to* `Video` *and every user field to* `User`

right structure for a system expecting popular content, and the reason it was not adopted here is scale rather than principle. That is a different claim from the design being correct in general, and it is recorded as such.

Indexes are defined to match the actual query patterns. A compound index on visibility, status, and creation time supports the home page listing, which always filters on the first two fields and sorts on the third.

Search is the exception. The collection carried a text index on title and description, described as supporting the search feature, but no query ever used it: search is implemented as a case-insensitive regular expression matched against title, description and tags, and MongoDB cannot serve an unanchored regular expression from an index. Every search was a full collection scan, and the index cost write throughput and storage while serving nothing. The index has been removed rather than the query rewritten, because switching to a text search would change the feature's behaviour and not merely its cost: the regular expression matches substrings inside words and covers the tags field, neither of which a text index does. At the scale of this project a full scan is acceptable.

4.4 Content Protection Design

Protecting private content requires two complementary mechanisms, because each addresses a different threat.

Origin Access Control makes the processed S3 bucket entirely private and grants read access exclusively to the CloudFront distribution through a bucket policy. This prevents a viewer from bypassing the content delivery network and fetching objects directly from S3, which would both circumvent any access control implemented at the edge and incur higher data transfer charges.

Origin Access Control alone, however, does not determine *who* may retrieve content through CloudFront. Without a second mechanism, any person holding a manifest URL can download the video regardless of its visibility setting, which would make the private mode ineffective in practice. The system therefore adds CloudFront Signed Cookies backed by a trusted key group [8]. The backend verifies the requester's permission using exactly the same service function that governs the API response, signs a policy scoped to the directory of that single video, and sets the resulting cookies on the response. CloudFront rejects any request lacking a valid signature with HTTP 403 at the edge.

Signed cookies were chosen over signed URLs because of the structure of HLS. A playback session retrieves a manifest and then hundreds of individual segment files, each as a separate request. Signing each URL individually would require rewriting every

entry inside the manifest, typically using an edge function, which adds considerable complexity and cost. A single set of cookies covers the entire directory and is attached automatically by the browser to every subsequent request.

The policy is signed as a custom policy rather than a canned policy. This distinction is not cosmetic: a canned policy does not permit wildcards in the resource path, whereas the required scope is an entire directory. The custom policy expresses the resource as a pattern covering the manifest and all segments of one specific video, which grants exactly the access needed and no more.

One consequence of this design deserves stating precisely, because it is easy to describe the scheme as protecting private videos and thereby understate it. The edge does not distinguish one visibility level from another: *every* manifest and segment requires a valid signature, and a request without one is refused whatever the video's visibility. What visibility governs is the authorisation decision made earlier, when the backend decides whether to issue cookies at all. All three levels are thus protected identically at the edge and differ only in who can obtain the credential.

That uniformity has a limit that the design has to accommodate. A cookie authorises exactly one video's directory, whereas a catalogue page renders thumbnails for many videos at once, and no single cookie can cover them. Rather than weaken the policy scope to span the whole bucket, the distribution carves out a separate cache behaviour for the thumbnail path that requires no signature. The justification is that a thumbnail is not the asset the mechanism exists to protect: it is a single low-resolution still, already shown to anyone permitted to see the video in a listing, and treating it as public costs nothing that the scheme was designed to prevent. The narrower point is that the protected unit is the stream, not every byte associated with a video.

4.5 Content Moderation and Operations Design

A platform that publishes what its users upload has to decide, for each upload, whether it may be shown. No single check can be trusted with that decision, because an automatic classifier errs in both directions: it removes harmless videos, which are false positives, and it lets harmful ones through, which are false negatives. The design therefore stacks four layers, each of which can correct the one before it: automatic screening, reports from viewers, review by staff, and a random audit of what was approved. It also records every human decision together with what the machine had concluded, so that the error rates of the earlier layers can be estimated and not assumed.

4.5.1 Automatic Screening

Screening runs inside the job that finishes the video and writes its verdict in the same conditional write that marks the video `READY`, so a video is never published without a verdict unless screening was switched off explicitly. On the single-container path it runs before transcoding, and on the chunked path in the finalizer. A video that is blocked is still transcoded, so that staff can watch it when its owner appeals.

The job samples one frame every five seconds, at most 120 frames, scaled to at most 720 pixels wide, and sends each to the image moderation operation of Amazon Rekognition [9]. The image operation was preferred to the video operation for a concrete reason: the video operation accepts stored videos only if they are encoded with H.264 in an MPEG-4 or MOV container and are at most 10 GB [10], whereas the platform accepts MKV, WebM and AVI uploads of up to 20 GiB, and FFmpeg in the container can decode all of them. The image operation also runs synchronously inside a job that is already running, so it needs no notification topic and no polling, and its cost has a ceiling: at the Singapore list price of 0.00125 USD per image, for the first million images of the month, a video costs at most 0.15 USD. The price of this choice is that sparse sampling can miss content that appears for less than the interval between two samples, which is why screening is the first layer and not the only one.

Each label returned for a frame is looked up in a policy table that maps it to one of three actions (Table 4.3). The table sets policy at the coarse levels of the label taxonomy and uses the finest level only to exclude a specific concept, so that a bare male chest, for example, does not send a video to review merely because its parent label is non-explicit nudity.

Table 4.3: Actions assigned to Rekognition labels by the screening policy

ACTION	LABELS	EFFECT
block	Explicit nudity and sexual activity, sex toys, graphic violence, death and emaciation, hate symbols and extremism	The video is removed automatically when a label reaches 90% confidence
review	Non-explicit nudity, obstructed intimate parts, violence, explosions, visually disturbing content, crashes	The video is held back for review when a label reaches 60% confidence
ignore	Weapons, bare back, exposed male nipple, and any label that is not in the table	No effect

Two refinements keep the policy from over-reaching. A frame that Rekognition classifies as animated or illustrated with at least 80% confidence lowers graphic violence and death from *block* to *review*, since violence in a game or a cartoon is not the violence

the rule is about; explicit sexual content and hate symbols are not lowered, because a drawing violates the rule as much as a photograph does. And screening fails closed: a video for which no frame could be analysed, or fewer than 80% of the planned frames, is held back for review and is not published, so that an unavailable service produces a delay and not a publication.

4.5.2 Moderation States and Visibility

The verdict is stored as `approved`, `flagged` or `blocked`, independently of the transcoding status of Section 4.3. Whether a video has finished encoding says nothing about whether it may be shown, and a video can be `READY` and still hidden. A flagged video appears in no listing and is visible only to its owner and to staff. A blocked video can be played by staff alone, and its owner can appeal the removal. The filter that implements this is one function used by every public query path, for the reason given for authorisation in Section 5.1: copies of a rule drift apart. A video uploaded before moderation existed carries no verdict and is treated as approved.

4.5.3 Review, Appeals and Audit

Amazon's own guidance expects people to review a small share of the content, typically one to five per cent of the total volume, already flagged by the model [9]. Staff clear the review queue by approving or blocking each video, and every decision must carry a reason code from a short, closed list: for an approval, for example, a model error, fiction or a game, or an acceptable context; for a removal, for example content missed by the model, an upheld report or a failed audit; and, for a video that the machine has already removed, a confirmation that it was right. The list is closed because free text cannot be counted, whereas a count of how many removals were reversed because the content was a game points to the threshold that needs adjusting.

Each decision is appended to an event log that records who decided and in which role, the verdict before and after, the conclusion the machine had reached, the reason code, and the outcome class of Table 4.4. The machine's conclusion is copied onto the video once, immediately before a person first changes the verdict. Without that snapshot the first human decision would overwrite the very fact that is needed to say whether the machine was right. No route edits or deletes an event, because the auditor's work depends on the log being intact.

Table 4.4: Outcome class of a human decision, given the machine's conclusion

OUTCOME	MACHINE	PERSON	MEANING
False positive	blocked	approves	The machine removed a video that was acceptable
Flag noise	flagged	approves	The flag did not need a person; it is not counted as a wrongful removal
False negative	approved	blocks	A violating video was published
True positive	blocked or flagged	blocks	The machine was right
True negative	approved	approves	The machine was right, for example a report found unfounded or an audit sample that passed
Unclassified	not known	either	A video from before the snapshot existed; it enters no rate

The owner of a removed video can appeal, once per decision, with a message of at most 500 characters; a new decision, such as the video being restored and later removed again, allows a new appeal. A person who does not own the video receives a not-found response, for the same reason as for visibility, and an operator resolves the appeal as upheld or rejected with a note. This follows the Santa Clara Principles, which ask a platform to notify the person affected and to offer an appeal that is reviewed by a person who took no part in the first decision [11]. Appeals are also the most reliable source of false positives, since only the owner has a reason to say that the machine was wrong.

False negatives are harder to find, because a missed video is one among thousands that were approved and viewer reports see only the most conspicuous of them. The design therefore draws a random sample of approved videos, which staff label *pass* or *fail*; the proportion of failures in the sample estimates the rate at which violating videos get through. A video can enter a sample only once, so its label is never counted twice.

4.5.4 Measuring the Screening

The console reports four figures, each with its denominator and a 95% Wilson confidence interval [12]: the share of automatic removals that staff reversed, which is the false positive rate; the share of flags that staff confirmed, which is the precision of the flag; the share of each group that has been reviewed at all, which is coverage; and the false negative rate. The Wilson interval is used because the counts are small, and unlike the normal approximation it stays within zero and one and remains usable when the observed rate is near zero. The false negative rate comes only from the random audit and never from viewer reports, which would bias it low. A video that no one has reviewed is

reported as unverified and enters no rate. Confirming a removal that the machine made is a separate action in the console, because such a video goes straight to the Removed list, where the only other action is to restore it. Without the confirmation the removals that staff had reviewed would be those they reversed, and the false positive rate would be computed over reversals alone; with it, a confirmed removal is a true positive and stays in the denominator.

4.5.5 Roles and Permissions

Four roles divide the work (Table 4.5). Operators decide, auditors read the error rates and the log without being able to change a decision, and administrators assign roles and draw audit samples. The separation of operator and auditor is deliberate: the person who reads the error rates is not the person whose decisions produce them, so the monitoring does not depend on the monitored.

Table 4.5: Permissions of the four roles

PERMISSION	ADMIN	OPERATOR	AUDITOR	USER
Approve or block a video, with a reason	yes	yes	–	–
Resolve an appeal	yes	yes	–	–
Label a video of an audit sample	yes	yes	–	–
Play a removed or held-back video	yes	yes	–	–
Draw a new audit sample	yes	–	–	–
Read the rates and the event log	yes	–	yes	–
See the owner's e-mail address in the queue	yes	–	–	–
Change a user's role	yes	–	–	–
Appeal the removal of one's own video	yes	yes	yes	yes

Authorisation is expressed as permissions and not as role names, so that adding a role means editing one table. A role change is itself restricted: only an administrator may make it, nobody may change their own role, the last administrator cannot be demoted, and every change is appended to a log that the auditor can read.

4.5.6 Verifying the Owner of an Account

Registration checks that an e-mail address is well formed and nothing more, so anyone can register with an address that belongs to someone else. Two harms follow. The registrant can upload, the most expensive operation of the system, under another person's address, which makes uploads anonymous in practice. And when the real owner of the address later signs in with Google, the account is already taken by someone who knows its

password. The design therefore asks each new account to prove that it can receive mail at its address, and lets only verified accounts upload.

After registration the backend generates a random token of 32 bytes, stores only its SHA-256 hash with an expiry of 24 hours, and e-mails a link that carries the token itself, as is done for the tokens of a password reset. The link opens a page of the single page application, and that page sends the token in a POST request, so that a mail client that previews links by fetching them cannot verify the account by itself. The server consumes the token in one conditional update, so two clicks on the same link cannot both succeed. An account that has not verified its address can still browse, like and comment, but the upload endpoint refuses it with a distinct error code, which the interface turns into a reminder with a button that sends the link again, limited to five mails per hour for each account and, whoever asks, to five per hour for each recipient address.

Three events count as proof of ownership: following the link; signing in with Google, whose identity token carries an address that Google has verified; and completing a password reset, since the reset link also goes only to the mailbox. A Google sign-in whose address matches an unverified local account takes that account over. The password is removed, every token issued earlier is invalidated through the password-change timestamp, and any pending verification or reset token is deleted, so that whoever registered first cannot keep access. An account that is already verified is never merged automatically: its owner must sign in with the password and link Google from the settings page.

4.6 Interface Design

Figure 4.5 sets out the paths a user can take through the application, and it is worth reading before the component-level discussion that follows, because two of its properties constrain the design.

The first is that the catalogue and the watch page are reachable without an account. Browsing, searching, filtering by category and playing a public video all work for an anonymous visitor, and authentication is demanded only at the point where an action requires an identity, which in practice means uploading, commenting, reacting, or managing one's own channel. A visitor who reaches the upload page without a session is therefore diverted through sign-in or registration and returned to it, rather than being asked to sign in on arrival.

The second is that the upload path does not end where the user's involvement ends. Once the browser has transferred the file directly to Amazon S3, the interface returns

control immediately and the video enters the `PROCESSING` state, in which it is visible only to its owner on the channel management view. Only when the pipeline marks it `READY` does it join the public catalogue. The interface consequently has to represent a video that exists but cannot yet be played, which is why the state machine of Section 4.3 is modelled explicitly rather than inferred. The same holds after the pipeline has finished: a video that screening held back is visible to its owner and not in the catalogue until a person approves it.

The frontend is a single page application composed of functional React components with hooks, using React Context for authentication and theme state. Redux was considered unnecessary at this scale, since the shared state consists only of the current user and the selected theme.

The interface is available in Vietnamese and English, selected by a control in the header and remembered in the browser. One consequence of adding a second language is worth recording, because it constrains the data model rather than only the presentation layer. Video categories are stored in the database as Vietnamese strings, and the schema validates each new video against exactly those strings, so translating a category would have made both the catalogue filter and the upload form reject valid input. The display label and the stored value are therefore kept apart: the client renders a translated label while always sending the original value to the API. The same separation applies to messages the API returns, which are selected from the request's `Accept-Language` header rather than being fixed at the point where the error is raised.

The video player is implemented as a custom component rather than relying on the browser's native controls. The reason is that native controls cannot be extended: the browser renders its own separate menu for playback speed and picture-in-picture, which cannot be merged with the quality selector that HLS.js exposes. Presenting the user with two unrelated menus would be confusing, so the component reimplements play and pause, seeking, volume, fullscreen, playback speed, picture-in-picture, and rendition selection within a single settings menu.

The player also determines when a view is counted. Rather than incrementing the counter when the page loads, which would allow the count to be inflated by repeated refreshes, the component notifies the page only after playback has passed a threshold of several seconds, and the server additionally ignores repeated reports from the same viewer within a thirty-minute window and never counts the owner's own views.

The operations console is part of the same application, at its own address and reachable only by staff. Its tabs are the review queue, the removed videos, appeals, audit samples, the rates, the event log and the users, and a person sees only the tabs that the

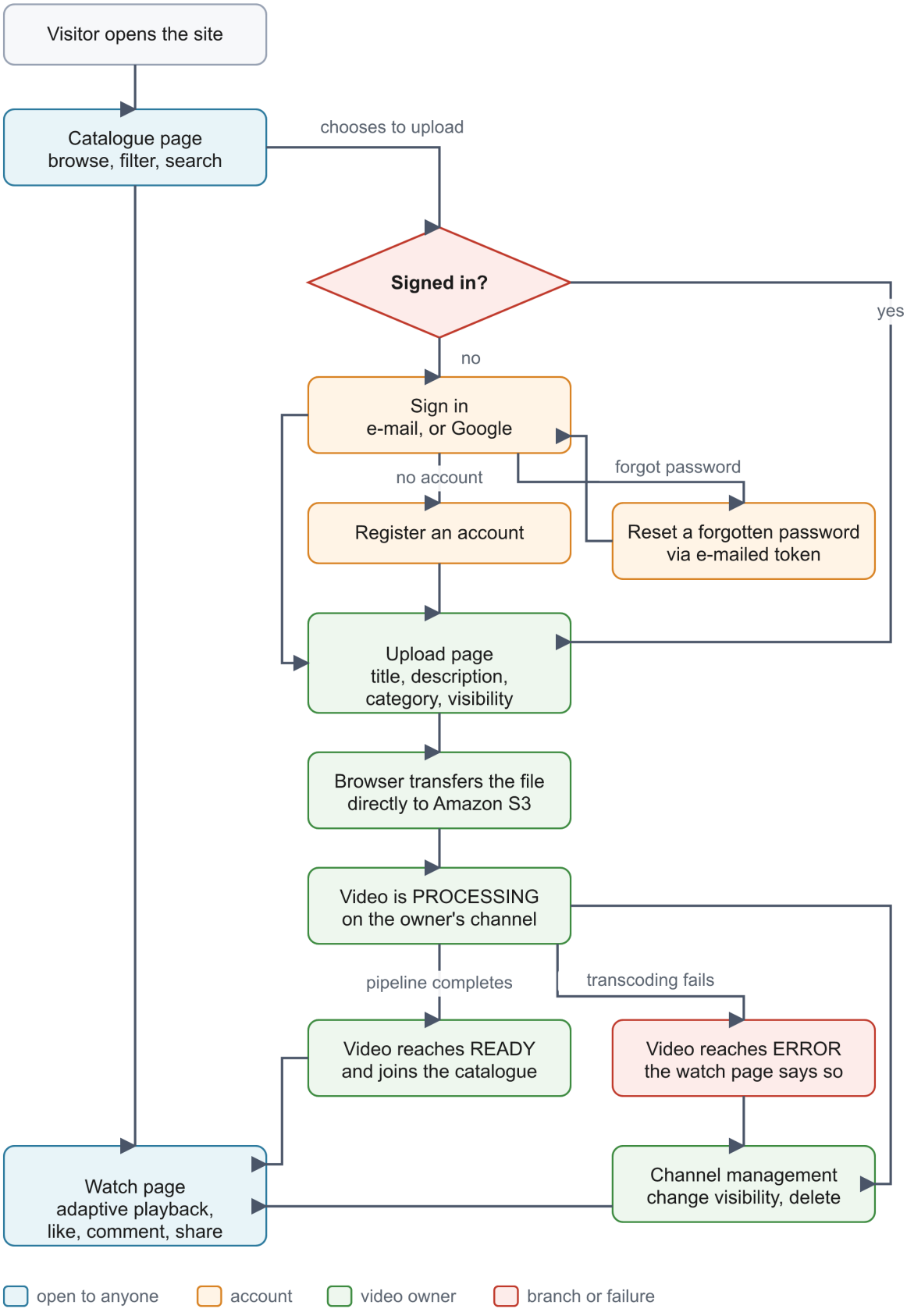


Figure 4.5: *User flow through the application, from an anonymous visit to a published video*

permissions of their role allow, so an operator never sees the rates and an auditor never sees a decision button. Thumbnails of videos under review are shown blurred until a reviewer chooses to reveal them, so that reading the queue does not by itself expose staff to the content.

Chapter 5

Development and Testing

5.1 Backend Implementation

The backend follows a layered structure in which routes delegate to controllers, controllers delegate to services, and only services interact with the data models. Business logic is kept out of route handlers so that it can be unit tested without an HTTP server.

Authorisation for video access is concentrated in a single service function. Every path that exposes a video, whether the detail endpoint, the like and comment endpoints, or the playback authorisation endpoint, calls that same function. This is a deliberate choice: duplicating the permission check across endpoints is the most common way for an access control gap to appear, because a later change updates one copy and not the others.

Operational authority follows the same principle. The four roles of the platform, user, operator, auditor and administrator, are mapped to permissions in a single table, and each route asks whether the requester holds a named permission, such as `moderation:review` or `metrics:read`, instead of comparing role names. Adding a role therefore means editing one table and not searching the code for comparisons. The requester's record, and with it the role, is read from the database on every request and is not copied into the token, so demoting a person takes effect at once and not when a token expires.

A decision is applied by one service function. It validates the reason code against the closed list for that decision, refuses to let an operator change a decision that an administrator made, copies the machine's verdict into `autoStatus` only if no earlier decision has done so, resolves the open reports on the video, dismissed on an approval and actioned on a removal, and then appends the event. The event is written after the decision has been applied. If writing it fails the decision stands and the failure is logged,

because withdrawing a removal on account of a faulty log would be the worse outcome; the false positive rate, the precision of the flag and the coverage are computed from the video documents themselves, so a missing event does not skew them.

The upload flow validates metadata on the server before issuing a pre-signed URL. Client-side validation alone is insufficient, since a user can call the API directly and obtain a URL for an arbitrary file, effectively turning the project's storage bucket into free hosting for unrelated content. The server therefore checks the declared MIME type against an allow-list of video formats and rejects files above the configured ceiling, which has a hard limit of 20 GiB in the source and is set to 20 GiB in the deployed system. Files up to 512 MiB go through a single pre-signed POST whose policy fixes the object key, the content type and the size range. Larger files use an S3 multipart upload in parts of 32 MiB: the backend signs one URL per part for exactly that part's length, so the storage service itself refuses a part of any other size, and on completion it lists the parts that S3 holds before asking for the object to be assembled.

Verification is a property of the account on which three places must agree: the guard of the upload route, the profile that the interface reads, and the takeover of an account by a Google sign-in. All three read it through one method of the user model, which treats an account with a Google identity as verified, and not through the stored field. The guard stands on the creation of the upload record, ahead of the upload rate limiter, so that a refused request does not use up the hourly quota; the later steps of a multipart upload need no guard of their own, because they act only on a record that the guarded step created.

Several protective measures were added at the HTTP layer. Security headers are applied through Helmet. Cross-origin requests are restricted to an explicit allow-list rather than being accepted from any origin, which is also a prerequisite for signed cookies to work, because the CORS specification forbids credentials in combination with a wildcard origin. Authentication endpoints are rate limited to ten attempts per fifteen minutes per address, which makes password guessing impractical without affecting legitimate users. Finally, the process validates its required environment variables at startup and refuses to start if the token signing secret is missing, converting a dangerous runtime failure into an immediate and obvious one.

A new password is checked in two stages, at registration, reset and change alike. The first runs locally: a short list of the passwords tried first, 131 of them and including common Vietnamese ones; obvious patterns, such as one repeated character, runs like `12345678` or `abcdefgh` and a short block repeated; a common base word followed by a few digits or symbols, or with letters swapped for digits; and the user's own name, the

start of their e-mail address and the name of the service. The second asks the Pwned Passwords service whether the password appears in a known breach, by the k-anonymity protocol [13]: only the first five hexadecimal characters of its SHA-1 hash leave the server, and the reply lists the suffixes that share them. A reply that takes more than two seconds, or fails, counts as no answer and lets the password through, so that registration does not depend on an outside service. Both stages apply only from eight characters, the length the user model already requires, and a refusal is a 400 response with the code `PASSWORD_TOO_COMMON`, `PASSWORD_PERSONAL` or `PASSWORD_BREACHED`, which the interface shows as a message. This follows the guidance that a verifier compare a new secret with commonly used, expected or compromised values and not impose composition rules [14].

The per-address limit on authentication does not stop one mailbox from being flooded, because an attacker who changes address is counted afresh while the mail still comes from the project's own account. A second limit therefore counts the mails sent to one recipient, under a hash of the normalised address, with one count shared by verification and reset mail: five per hour. Beyond it, registration still creates the account and skips the mail, the interface offering to send it again; a request to send the link again is refused with a 429 response before a new token is issued; and a reset request returns the same generic reply as every other outcome, so it still does not reveal which addresses have an account, and issues no new token.

5.2 Transcoder Implementation

The transcoder is a Node.js worker packaged with FFmpeg in a Docker image built through a multi-stage build. The base image went through two upgrades over the course of the project. Node 18 was used initially but had to be abandoned because Mongoose 9 depends on the Web Crypto API, which is not fully available in that version, causing the container to fail when connecting to MongoDB Atlas. Node 20 was then adopted, and later replaced with `node:24-slim` once Node 20 reached end-of-life on 30 April 2026 and stopped receiving security patches; Node 24 was chosen over the newer Node 26 because it is the current Active LTS release, supported until April 2028, whereas Node 26 had not yet entered its LTS phase at the time of the upgrade. The Alpine variant was avoided throughout because FFmpeg requires glibc.

The heartbeat mechanism described in Section 4.2 is, in effect, a lease in the sense introduced by Burrows for the Chubby lock service [15]: a holder retains a resource by periodically renewing it before a timeout expires, and Burrows already identified

the classical failure mode of that pattern: a lease can expire while its holder is merely stalled rather than dead, so a second holder may be granted the same resource while the first is still alive and will eventually act on it. A defect matching exactly this failure mode surfaced during a literature-driven review of the pipeline's reliability guarantees, discussed further in Section 5.6: neither `updateVideoReady` nor `updateVideoError` checked the video's current status before writing, so a duplicate job triggered by an SQS redelivery could silently overwrite a successful result, and in the worst case could revert an already-`READY` video back to `ERROR`. Both functions were changed to a conditional `findOneAndUpdate` that only writes when the current status is not already `READY`, and `processVideo` now checks the video's status before downloading anything, so a duplicate delivery that arrives after the first job has already finished is rejected immediately instead of re-downloading and re-transcoding the source.

A single FFmpeg invocation produces every rendition of the ladder that the source can fill, which is more efficient than separate invocations because the source is decoded only once. The ladder has six rungs, from 144p at 120 kbit/s to 1080p at 4 Mbit/s, and each rung fits the source inside its own frame without enlarging it: a 480p source yields a 360p rung and one at 854×480, not a 1080p rendition enlarged 2.25 times. Only the single-container path for a video longer than twenty minutes keeps the earlier three-rung ladder of 360p, 720p and 1080p, to stay within the time limit of a job. The command emits segments of six seconds, generates the per-rendition playlists, and the worker then writes the master playlist that lists the available bandwidths, codecs and resolutions. Those bandwidths and codec strings are derived from the encoded output rather than from the encoder settings: the bandwidth is the peak segment bit rate measured from the segments themselves, which Section 6.3.3 examines in detail, and the codec string is read out of each stream's sequence parameter set, since the constraint-set byte that RFC 6381 requires appears in neither the profile name nor the level number reported by a probe. A thumbnail is extracted in a separate pass.

The chunked path of Section 4.2.2 runs the same container image under four commands. The planner computes the chunk boundaries with functions that perform no input or output, so the arithmetic can be tested directly: a boundary is a whole number of groups of pictures counted in frames, and every chunk, the first included, writes its timestamps from the same base offset, so that the playlists of independently encoded chunks join without a gap or a repeated segment. A chunk job reads only its own range of the source over HTTP and encodes every rendition of the ladder for it. The finalizer joins the playlists and, as the single-container path does, measures the bandwidth of each rendition from the segments and not from the encoder settings. Chunk jobs are submitted

with a task size of their own, 4 vCPU and 8 GiB, because a four-vCPU task measured 14% faster and used 7% fewer vCPU-seconds than a one-vCPU task; the size is checked against the table of valid Fargate combinations when the configuration is loaded, since Batch would otherwise reject the submission only after the video had been marked as processing.

Content screening is one function that the single-container job and the finalizer both call, with the same policy and thresholds. It extracts the sampled frames with FFmpeg, sends them to Rekognition three at a time with retries and client-side rate limiting when the service throttles, applies the label policy of Section 4.5.1, and returns the verdict together with its evidence labels and the share of planned frames that could be analysed. A failure inside the step is converted into a flagged verdict that carries the error, so that the video waits for a person and is neither lost nor published. Screening is on unless `MODERATION_ENABLED` is explicitly set to false, so that a missing setting cannot silently publish every video.

Failure handling is deliberate. When FFmpeg exits with a non-zero status, the error propagates to the top-level handler, which records the `ERROR` status in the database and, in `worker` mode, does *not* delete the SQS message: leaving it in flight allows the visibility timeout to lapse so that the job is retried, and after the configured number of failed attempts the message is moved to the dead letter queue where it can be inspected rather than lost.

As Section 4.2 establishes, however, the deployed system runs `batch` mode, where the container holds no message to retain. The retry and dead-letter behaviour just described therefore governs only the Lambda that submits jobs, not the transcoding itself, and the recording of `ERROR` in the database is the sole part of this paragraph that applies to a deployed transcoding failure. The gap this left, and its consequences for how such failures were detected, is examined in Section 5.5.

5.3 Infrastructure as Code

The AWS environment is described by thirteen Terraform modules, summarised in Table 5.1, and is described by three environment definitions: `dev`, which hosts the deployed system; `staging`, a reduced copy used to test a branch before it is merged; and `prod`, a variant that uses On-Demand Fargate capacity, keeps logs for thirty days and does not force-destroy buckets. Only the first two have been applied: the state of `prod` does not exist, and its definition was checked with `terraform validate` and never deployed. The `dev` environment, which serves real users, carries the same protection

against a mistaken `terraform destroy`: its buckets and its registry cannot be force-deleted while they hold objects or images, and a deleted secret stays recoverable for seven days.

Table 5.1: Terraform modules and their responsibilities

MODULE	RESPONSIBILITY
`s3`	Raw and processed buckets, CORS configuration, Glacier lifecycle transition, event notification
`sqs`	Transcode queue, dead letter queue with a redrive policy, queue access policy
`ecr`	Container registry with image scanning on push and a lifecycle policy retaining recent images
`iam`	Execution roles for AWS Batch, the ECS task, the transcoder task, and the Lambda submitter
`vpc`	Virtual network, public subnets, internet gateway, security groups, S3 gateway endpoint
`secrets`	AWS Secrets Manager entries for the database connection string and the token signing secret
`sns`	Notification topics for transcode completion and dead letter queue alerts
`monitoring`	CloudWatch log groups, alarms and dashboard, a scheduled health check of the public API, and a rule that alerts on failed Batch jobs
`batch`	Fargate Spot compute environment and an optional On-Demand one for overflow, the main and the lower-priority job queue, and the job definition
`lambda`	Job submitter function and its SQS event source mapping
`cloudfront`	Distribution, Origin Access Control, cache policy, and the trusted key group used for signed cookies
`backend-userdata`	Builds the start-up script of the backend instance from a single template, so that every environment runs the same script; creates no resources
`cloudflare-ips`	The IPv4 ranges from which Cloudflare reaches the origin, used to restrict the backend's security group; creates no resources

Two configuration details are worth highlighting. The S3 lifecycle policy transitions raw source files to Glacier after thirty days, since they are never read again once transcoding has succeeded but must be retained in case a re-encode becomes necessary. The cache policy assigns a long time-to-live to segments, which are immutable once produced, and a short one to manifests, which may change if the set of renditions changes.

5.4 CI/CD Pipeline and DevSecOps

Seven GitHub Actions workflows automate verification and deployment, as shown in Figure 5.1.

The security gate is structured in two tiers, and the reason is practical. Scanning at the combined critical and high severity levels and failing the build on any finding sounds rigorous, but in practice most high-severity findings originate in transitive dependencies for which no patch yet exists. A gate configured that way fails continuously for reasons the team cannot act upon, and the predictable result is that developers begin ignoring it. The pipeline therefore reports critical and high findings for visibility without blocking, and blocks only on critical findings for which a fix is available. This keeps the gate meaningful, because a failure now always corresponds to an action the team can actually take.

Static application security testing is performed with `eslint-plugin-security` rather than SonarQube. The rule set detects patterns such as command execution built from user input, regular expressions vulnerable to catastrophic backtracking, and cryptographically unsafe random number generation. The choice avoids operating a separate analysis server while still executing genuinely within the pipeline. Where a finding is a false positive, it is suppressed with an inline annotation that documents why, which forces each suppression to be justified rather than silently ignored.

Infrastructure code is validated in its own workflow, which checks formatting, initialises without a remote backend so that no credentials are required, validates both environments, and scans the configuration for insecure settings.

A later audit of the pipeline found a weakness in its own supply chain. All five workflows referenced the vulnerability scanner as `aquasecurity/trivy-action@master`. A branch reference resolves to whatever that repository's default branch contains at the moment a job runs, so a change made there, including one made by someone who had compromised the action, would execute inside this project's continuous integration, which holds AWS credentials. Every other action in the pipeline was already pinned at least to a major version tag. The references are now pinned to a release commit hash, with the version tag retained in a comment so that upgrades remain deliberate and readable.

The same audit narrowed one permission. The Lambda that submits transcoding jobs held `batch:SubmitJob` on `Resource: "*"`, which allowed it to submit to any job queue in the account rather than the project's own. It is now scoped to the specific queue and job definition. The scoping is expressed through the naming convention rather than by referencing the Batch module's outputs, because the Batch module already depends

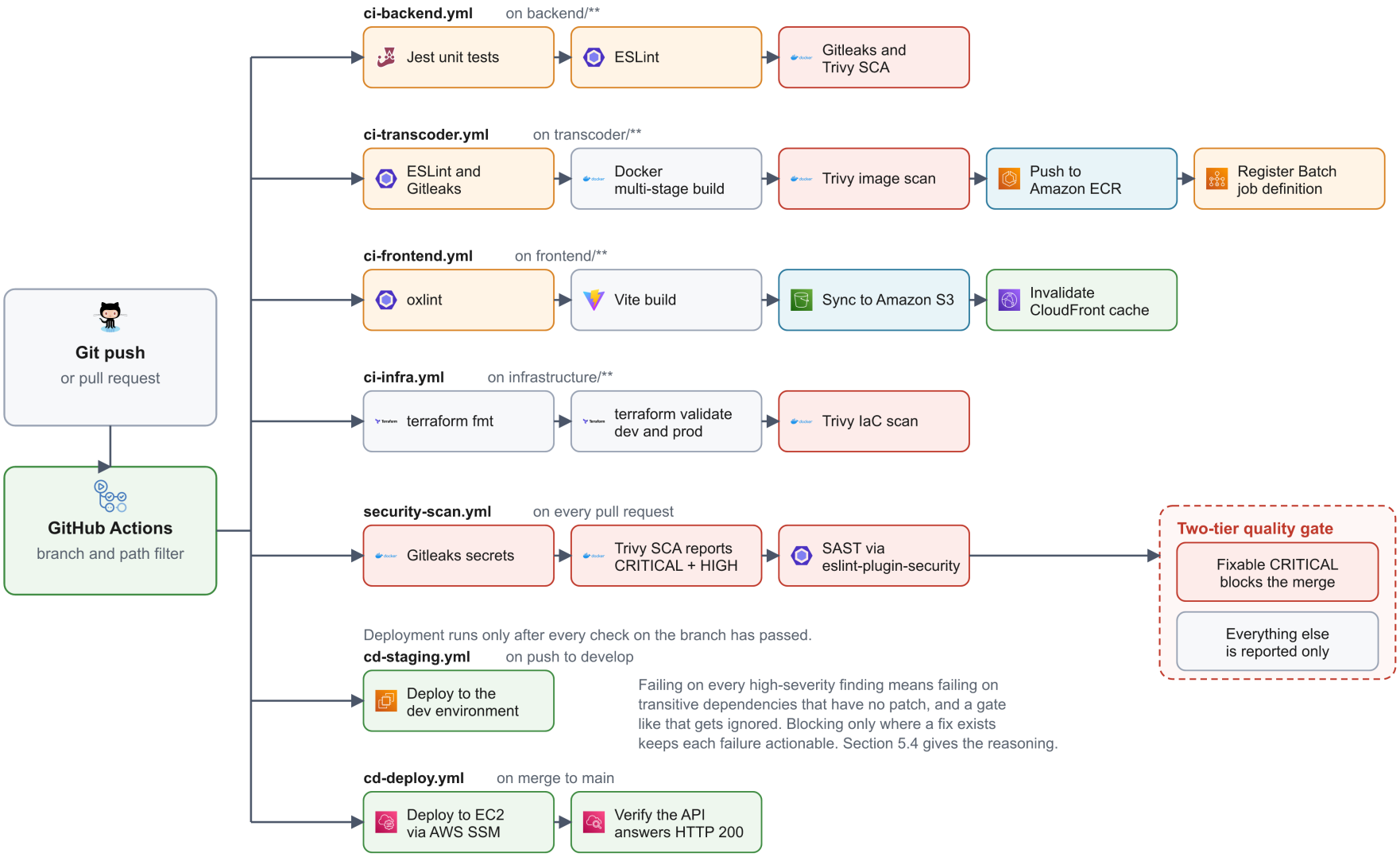


Figure 5.1: *The seven GitHub Actions workflows, the file paths that trigger each of them, and the two-tier DevSecOps quality gate*

on the IAM module for its roles and referring back would introduce a dependency cycle; both modules carry a note recording the coupling that this creates. The companion action `batch:DescribeJobs` keeps the wildcard, because it accepts arbitrary job identifiers and AWS provides no resource-level control for it, and the module records that reason in a comment so the wildcard is not mistaken for an oversight.

5.5 Challenges During Development

Several difficulties arose between a design that was correct on paper and a system that behaved correctly once deployed. Each is recorded here with the resolution adopted, because those resolutions shaped parts of the final architecture.

5.5.1 Provisioning Blocked by Account Policy

The content protection scheme of Section 4.4 was implemented and covered by unit tests well before it could be provisioned. Applying the Terraform configuration failed with an access-denied error, because AWS requires manual account verification before a new account may create CloudFront distributions, and the request filed to lift that restriction remained pending with no bound on how long it might take. Until it was resolved the processed-content bucket was reachable directly over the public internet, so the private visibility setting had no effect at the storage layer.

The restriction was worked around by provisioning the distribution in a second AWS account not subject to it. Terraform supports this without duplicating any module: the distribution, its Origin Access Control, its cache and response-header policies, and its Signed Cookie trusted key group were given an explicit `provider` argument pointing at the second account, while the buckets, Batch, Lambda and the database connection remained in the original one. Only the bucket policy could not move, since a bucket policy may be attached only by the account that owns the bucket. This required no special handling, because an Origin Access Control policy is expressed as a service principal together with an `AWS:SourceArn` condition naming the distribution, a form that is indifferent to which account owns the ARN. With the distribution in place, the temporary public bucket policy was removed and the bucket returned to being reachable only through CloudFront.

5.5.2 A Signed Cookie Policy That Matched Nothing

Exercising Signed Cookies against the new distribution revealed that every request returned `403 Forbidden`, whatever the video's visibility. The function building the resource pattern that a cookie authorises, `buildResourcePattern`, signed `videos/*/{videoId}/*`, with a wildcard segment between the prefix and the identifier, whereas the transcoder writes every rendition and the master playlist directly under `videos/{videoId}/`. Each cookie was therefore well formed and correctly signed, and authorised a resource that no viewer could request.

The existing unit test did not detect this because it compared the function's output against a literal string copied from the function itself rather than against the transcoder's key layout, so it could confirm only that the function had not changed. The pattern was corrected to `videos/{videoId}/*` and the test rewritten to assert against the layout the transcoder actually produces. Signed Cookies were then verified end to end against the deployed system: a manifest request without a valid cookie is rejected at the edge, and the identical request succeeds once the playback-authorisation endpoint has issued cookies for that video.

5.5.3 Two Consequences of Gating Every Object

Gating the whole bucket behind Signed Cookies also gated video thumbnails, which a catalogue listing must render for many videos at once even though a cookie authorises exactly one. A separate unsigned cache behaviour was carved out for the thumbnail path, since a thumbnail is not the content the mechanism exists to protect.

Separately, the frontend's own CloudFront distribution, added because an Amazon S3 website endpoint cannot return `200` for a single-page application's client-side routes on direct navigation, had no cache-invalidation step in its deployment workflow, so a fix already synced to the origin bucket continued to be served from the edge for hours afterwards. The workflow now invalidates that distribution immediately after every deployment.

5.5.4 Playback Across the Cookie Expiry Boundary

Signed Cookies are valid for two hours, and the watch page requested them once when it mounted. Crossing that boundary produced a `403` on the next segment, and the player's error handler responded to any fatal network error by calling `startLoad()` with no attempt counter and no delay, so an error that does not resolve on its own became an

unbounded retry loop behind a "reconnecting" message that never changed.

Two properties of hls.js governed the fix. The library marks the first `403` as non-fatal while it retries internally, so the handler must inspect the status code before filtering on the fatal flag, or it discards the very error it exists to report. And `startLoad()` resumes segment loading, whereas an expired cookie fails at the manifest, so recovery requires re-invoking `loadSource()`. The handler now refreshes the cookies on a `403`, reloads the source, and gives up after a bounded number of attempts with a message the viewer can act on.

5.5.5 An Empty Secret That Stopped the Pipeline

Every transcoding task began failing at `ResourceInitializationError` before its entrypoint ran. The Terraform module for Secrets Manager created the entry holding the notification e-mail password unconditionally but populated its value only when a password had been configured, so leaving e-mail unconfigured produced a secret with no version carrying the `AWSCURRENT` label. The Batch module decided whether to declare that secret in the job definition by testing whether its ARN was non-empty, and the ARN of an empty secret is non-empty. The fix made the secret itself conditional, so that the unconfigured state is internally consistent: no secret, an empty ARN, and no declaration in the job definition.

The defect persisted longer than it should have because the only jobs submitted while it was live belonged to the concurrency stress test, whose synthetic payloads were expected to fail. Checking the cause recorded in `statusReason` rather than the terminal state alone would have identified it immediately, and the alerting described next was changed to carry that field.

5.5.6 Retry and Alerting on the Deployed Path

Section 4.2 describes retry as delegated to SQS redelivery, which holds in `worker` mode. On the deployed path the Lambda submitter deletes the message once the job is submitted, so SQS no longer holds anything to redeliver, and the Batch job definition declared no `retryStrategy`. Nothing retried a failed job at any layer, and a transient fault such as a network error while downloading the source lost the video with no record beyond an `ERROR` row.

Detection had the same shape. The alarm whose description read "video transcoding failures detected" watched the depth of the dead letter queue, which receives a message only when the Lambda fails to submit, so a job that fails after submission is invisible to it.

The job definition now declares three attempts, retrying infrastructure faults and exiting immediately when the exit status indicates that FFmpeg rejected the content, together with a two-hour timeout that bounds a process stalled on malformed input. Because AWS Batch publishes no CloudWatch metric for failed jobs, an EventBridge rule matches job state transitions to `FAILED` and publishes to the alert topic, with the notification carrying `statusReason`. The alarm description was corrected to state what it genuinely covers.

5.5.7 Monitoring the Service Rather Than the Server

Both of the alarms defined at that point watched the transcoding pipeline, the dead letter queue and the queue depth, and neither watched the backend API, which runs as a single process on a single instance. The gap surfaced when assigning the backend a static address moved its public IP while DNS still pointed at the released one, and the API returned `522` for several minutes. Every infrastructure indicator stayed green, because nothing at the infrastructure level was wrong.

A scheduled function now calls the public domain every five minutes along the same path a viewer takes and publishes a metric that an alarm watches, with an instance status-check alarm alongside it. A self-hosted check was chosen over a managed one on cost, since a managed health check on an endpoint outside the provider costs roughly 1.75 USD per month against zero within the free tier, and the trade-off it accepts, that checks originate from a single location, is recorded with the implementation. The alarm's evaluation period is three times the check interval, so that each window holds several datapoints and one late execution cannot open a gap that the missing-data policy would read as a breach.

5.5.8 Infrastructure Drift in the Instance Definition

The AMI for the backend instance is resolved through a data source with `most_recent = true`, and the `ami` attribute forces replacement when it changes, so every Ubuntu security release armed the next `terraform apply` to destroy and rebuild the running server. What would have been lost is the state Terraform does not manage: the environment file holding the database URI, signing key and mail credentials, the nginx configuration, the TLS certificates, the process manager's state, and the public address that DNS points at. Changes to the attribute are now ignored, which makes an image upgrade deliberate, and an Elastic IP keeps the address stable across any rebuild that does occur.

5.6 Testing

The automated test suite comprises 853 tests across 52 suites, 492 in 35 suites for the backend and 361 in 17 for the transcoder, and Tables 5.2 and 5.3 group them by area.

Table 5.2: Automated backend tests by area, 492 tests in 35 suites

AREA	TESTS	FOCUS
Authentication and account verification	102	Registration conflicts, login failures and token claims; the issue, expiry and single use of the verification link, which is stored only as a hash; the Google sign-in that takes over an unverified account; missing, malformed, forged and expired tokens, and tokens issued before the account's last password change; the password policy, which refuses common, personal and breached passwords and lets a password through when the breach lookup fails; and a reset response that is identical whether or not the address has an account, including when e-mail delivery fails
Visibility and authorisation	34	Privacy enforcement across every query path; that the recommendation endpoint reuses the rule of the detail endpoint and never surfaces a private or unlisted video; and that a video exposes how many likes and dislikes it has but not who gave them
Upload	98	Metadata and ownership checks, the refusal of an unverified account, the deletion cascade, pre-signed URL creation, file type and size validation, the ceiling and its hard limit, the choice between a single and a multipart upload, part URLs signed for exact sizes, completion by listing the stored parts, and the clean-up of abandoned uploads
Content delivery	30	Cookie signing, policy scope and cookie attributes, and that revocation repeats the attributes used when issuing
Listing, reactions and views	31	Pagination bounds, a stable sort order for channel pages, likes and dislikes as conditional updates, view deduplication, and the order of routes
Operation and reconciliation	50	Reconciliation of videos stuck in processing or abandoned in uploading, the client address behind Cloudflare and nginx, rate-limit keys, the cap on mail to one recipient with its sliding window and bounded memory, e-mail configuration, and the health endpoint
Moderation and operations console	147	Classification of each decision as a false positive, a false negative or neither; the snapshot of the machine's conclusion written once; the confirmation of a removal the machine made, logged as its own action and counted as a true positive; the permission table for the four roles; one appeal per decision; audit sampling and labelling; the Wilson interval and the rates; and that the last administrator cannot be demoted

Table 5.3: Automated transcoder tests by area, 361 tests in 17 suites

AREA	TESTS	FOCUS
Chunked pipeline	184	Chunk boundaries as multiples of the group of pictures counted in frames, the conditions that fall back to one container, command construction, playlist joining, submission to the lower-priority queue, task-size validation, and the planner, audio, chunk and finalizer jobs end to end
Ladder, frame rate and encode guard	78	Rung selection from the source resolution and the reduced ladder for long videos; that the group of pictures follows the real frame rate, that a variable-frame-rate file cannot poison it, and that every rendition cuts at the same instant; giving up early when a video cannot be encoded within the budget of a job; the thumbnail
Playlist metadata	51	That the declared bandwidth of each variant is the peak measured from the segments rather than a configured target, rounded upward as RFC 8216 requires, and that the codec string is read from the encoded stream and omitted rather than guessed
Persistence and delivery	30	Conditional writes that prevent a duplicate job from overwriting a result, object storage access, and the content of the READY and ERROR notifications, which never expose raw failure detail to the viewer
Content screening	18	The label policy including parent labels and the downgrade of animated content, the frame sampling plan, and holding back a video that could not be analysed fully

The visibility suite is the most important from a security standpoint, since it asserts that a private video is absent from the public listing, returns a not-found response rather than a forbidden one to non-owners, and remains accessible to its owner. Returning not-found is deliberate: a forbidden response would confirm that a video with that identifier exists, which leaks information.

The value of testing was demonstrated concretely during development of the signing service. An initial implementation produced a canned policy, which the test suite immediately flagged because the resulting cookie set contained an expiry field instead of a policy field. Since canned policies do not support wildcard resources, that implementation would have failed for every video once deployed. The defect was caught before any infrastructure was provisioned.

Testing caught a second defect in a similar way, this time surfaced not by writing a test first but by a literature review of the reliability guarantees the pipeline was implicitly assuming (Section 5.2). Once the conditional-write fix was implemented, six tests were

written to pin down the exact contract: a normal write succeeds when the video is not yet `READY`; a write that loses the race to an earlier successful write is rejected and the function returns the winning record rather than throwing; and a late failure from a duplicate job can never revert an already-`READY` video to `ERROR`. As a falsifiability check, the same six tests were run against the original, unconditional implementation: four of the six failed, either because it never calls `findOneAndUpdate` at all or because it crashes outright when handed a plain record with no `save` method, which is exactly the shape a conditional read would have to return. The two that still passed were the not-found cases, which neither version distinguishes from a duplicate write. The result confirms both that the new tests are actually exercising the fixed code path and that the original implementation could not have satisfied the contract by construction, not merely by accident of the test data chosen.

Chapter 6

Deployment and Evaluation

This chapter describes the deployed system and its interface, then evaluates it: playback quality as a viewer experiences it, transcoding latency, the chunked path for long videos, behaviour under a burst of uploads, and the monthly operating cost. It ends by setting the non-functional requirements of Section 3.3 against this evidence. Every figure was measured on the live deployment or on a staging environment with the same configuration.

6.1 The Deployed Environment

The frontend is a single-page application served from an Amazon S3 bucket through its own CloudFront distribution. The backend API runs as a Node.js process on an Amazon EC2 instance, behind nginx for TLS termination and a process manager that restarts it on failure. Media is served from a third hostname, the video distribution, whose origin is the processed-content bucket and which admits a request only when it carries a valid Signed Cookie. Cloudflare provides DNS and edge TLS for the two application hostnames, MongoDB Atlas holds the database, and the transcoding tier of Chapter 5, made of the SQS queue, the Lambda submitter and AWS Batch, occupies no capacity between jobs.

A scheduled health check calls the public API every five minutes and publishes a metric that an alarm watches, so the service is monitored rather than only the instance hosting it. The instance is the only component that costs money continuously, and it can be stopped and restarted without losing its environment file, its TLS certificates or its public address.

6.2 User Interface Showcase

This section shows the main screens of the deployed system and what a user does on each.

Figure 6.1 shows the catalogue, the first page a visitor reaches. The header holds the search field, a theme toggle and the sign-in links, and seven category tabs below it filter the listing without a page reload. Each card shows the thumbnail made during transcoding, the duration, the title, the channel, the view count and the age of the upload. Public videos can be read without an account, so a visitor sees the same catalogue as a registered user.

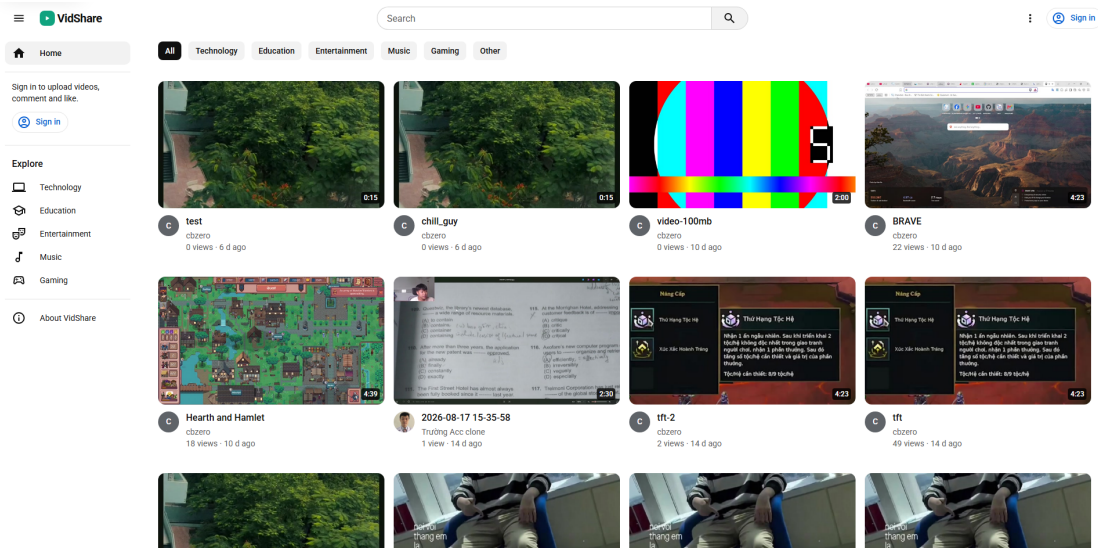


Figure 6.1: *Catalogue page, the entry point for a visitor*

Selecting a card opens the watch page in Figure 6.2. The player has its own control bar, because the quality selector of HLS.js cannot be merged into the browser's native controls (Section 4.6). It offers seeking with a time tooltip, volume, fullscreen, playback speed, picture-in-picture and rendition selection. Before the stream loads, the page asks the playback-authorisation endpoint for the Signed Cookies that the CDN requires. Below the player are the title, the like, dislike and share controls, the channel and the comments, and a sidebar lists related videos by category and author.

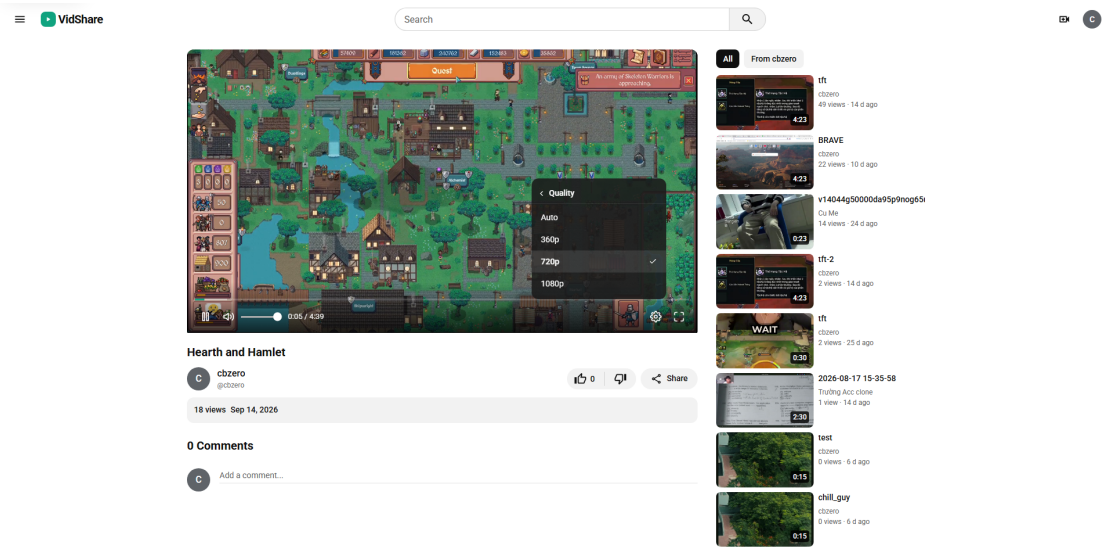


Figure 6.2: *Watch page with custom player, rendition selector and related videos sidebar*

Figure 6.3 shows the upload page. The form collects the title, description, category and visibility; the browser then receives a pre-signed URL and sends the file directly to Amazon S3, so the progress bar measures a transfer that never passes through the backend. The page shows the bytes sent, the speed and the time remaining, and its cancel button aborts the transfer and discards the draft record. When the transfer ends the page returns at once, and the video waits on the channel in a processing state until the pipeline marks it ready.

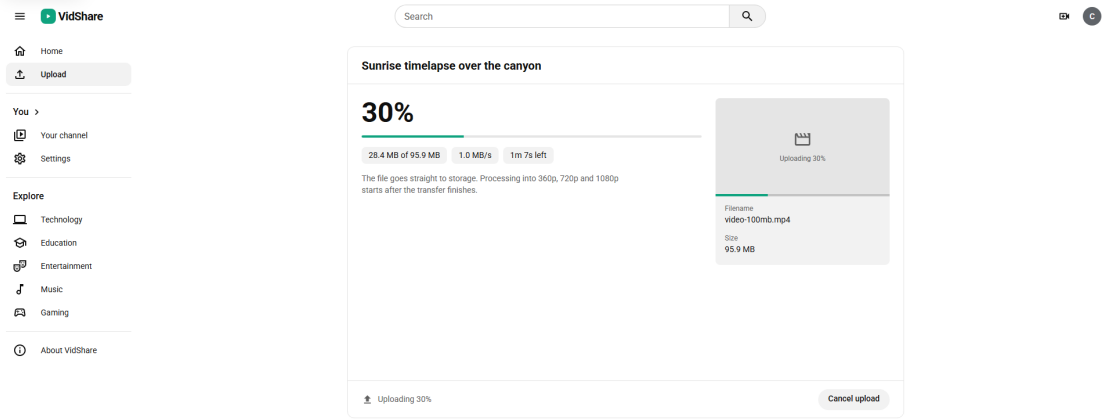


Figure 6.3: *Upload page, showing direct-to-storage transfer with live telemetry*

Figure 6.4 shows the channel management view. It lists all of the user's own videos with their state, and it is the only place where a video in the `PROCESSING` or `ERROR` state is visible, since the catalogue lists only `READY` videos. Here the user edits a video's title,

description, category and visibility, or deletes it.

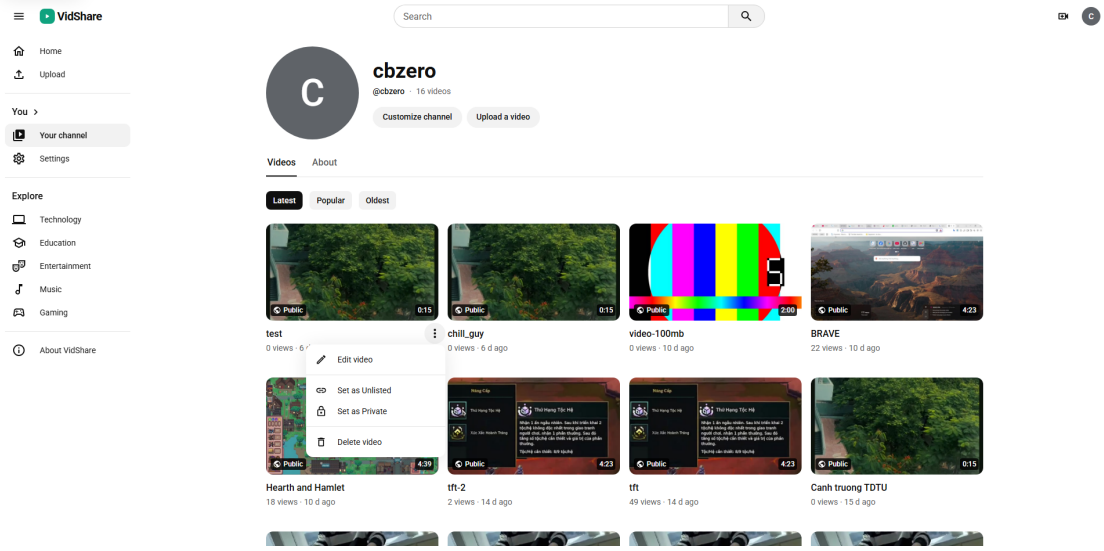


Figure 6.4: *Channel management view, including videos still being processed*

Staff reach the operations console of Figure 6.5 from the Moderation entry of the sidebar, which only operators, auditors and administrators see. Three counters show the videos waiting for review, the open viewer reports and the removed videos. The *Removed* tab lists each removed video with a blurred poster, the labels the machine detected with their confidence and their time in the video, and who removed it, with actions to watch it, read its reports, restore it, or confirm a removal the machine made. Of the two test uploads in the figure, one was removed by an operator after a viewer report; the other, a deliberately violent clip, was removed by the machine with 98.9 per cent confidence for both Blood and Gore and Graphic Violence, and an administrator then kept it removed.

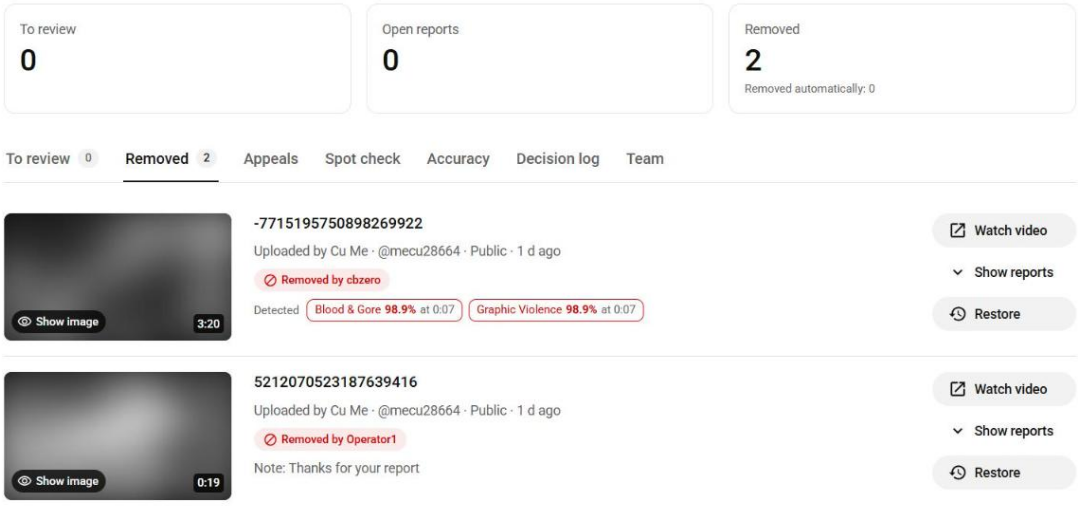


Figure 6.5: *Operations console, Removed tab, with the labels the machine detected and who removed each video*

Figure 6.6 shows the Accuracy tab, which administrators and auditors see. It gives four figures, each with its 95 per cent interval and its sample size: the share of automatic removals made wrongly, the share of flags that were right, the missed share estimated from the random sample, and the number of known misses. With one removal reviewed, the interval for wrong removals runs from 0.0 to 79.3 per cent. The screen thus shows that so small a sample says little, which is why this report makes no claim about the error rates of the screening on real traffic.

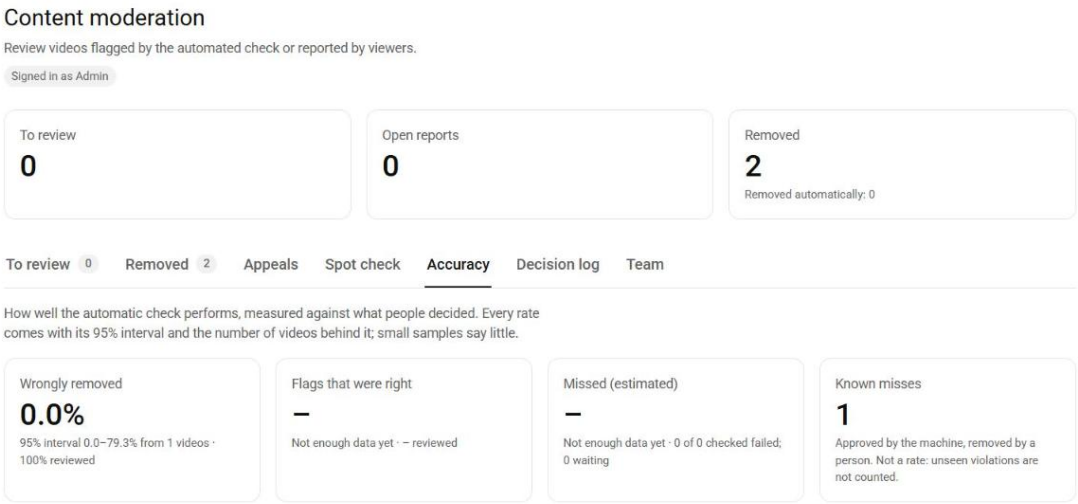


Figure 6.6: *Operations console, Accuracy tab, each rate shown with its interval and sample size*

6.3 Quality of Experience Evaluation

The playback measurements were taken in September 2026, when the bitrate ladder had three rungs, 360p, 720p and 1080p; it was extended to six rungs on 7 October 2026 (Section 5.2). The findings below do not depend on the number of rungs.

Time to first frame (TTFF) is measured by a script that fetches the master playlist, then a rendition playlist, then the first media segment, and records the time of each stage. Each measurement is repeated twenty times, because one sample means little when network latency varies and the first request is usually a cache miss. Measured directly against the S3 bucket, before the CDN was deployed, the median TTFF was 251.52 ms. This origin latency is the baseline for the CDN measurements; its statistics are in Table B.1 (Appendix B).

6.3.1 Time to First Frame Through the CDN

Through the CDN every request must carry a Signed Cookie, so the script first requests cookies from the playback-authorisation endpoint and attaches them to all three stages, as a browser does. Figure 6.7 compares the three delivery configurations.

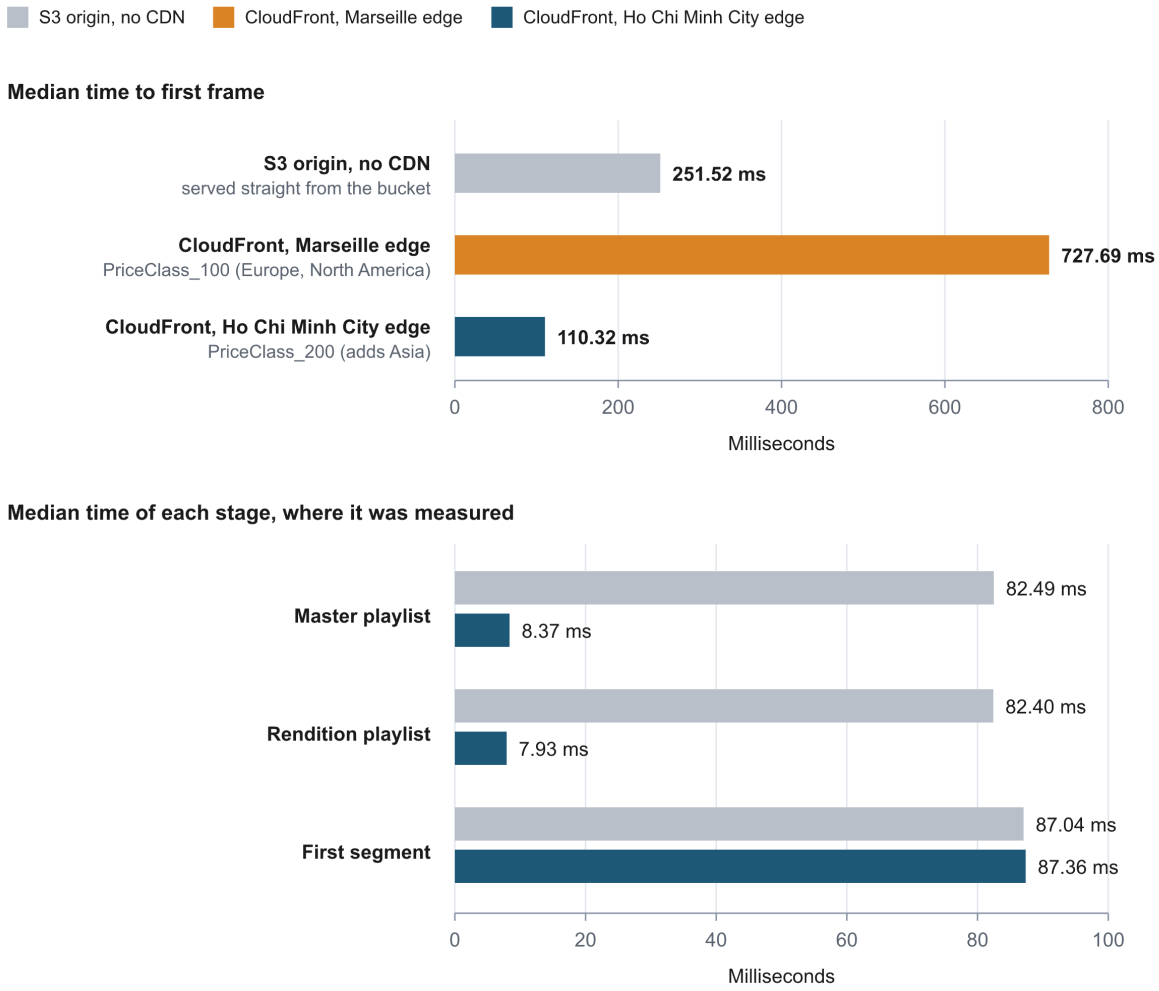


Figure 6.7: *Median time to first frame in three delivery configurations, and the median of each stage where it was measured*

Note: $n = 20$ *per configuration. Through the Ho Chi Minh City edge, the eighteen samples that hit the cache at all three stages have a median of 107.57 ms. The file of the 727.69 ms run was overwritten by the later run, so its stages are not shown. Full statistics are in Table B.2 (Appendix B).*

The first run through the CDN gave a median of 727.69 ms, 2.9 times slower than the origin it was meant to accelerate. Caching was not the cause: nineteen of twenty samples were edge hits, and the single miss took 4655 ms. Every response carried `X-Amz-Cf-Pop: MRS53`, an edge in Marseille, France, because both distributions

used `PriceClass_100`, the cheapest class, which serves only from North America and Europe. Every request from Vietnam, where the users are, therefore crossed to Europe and back.

Changing both distributions to `PriceClass_200`, which includes Asia, moved delivery to `SGN50` in Ho Chi Minh City. The median fell to 110.32 ms, 6.6 times faster than through Marseille and 2.3 times faster than the S3 origin. The two playlists, small cached files, now take single-digit milliseconds, and the segment, the only stage with a real payload, takes most of the time. The fault was invisible to every check except a measurement taken from where the users are: the configuration was valid, the cache hit rate was high and every request succeeded.

6.3.2 Delivery Latency by Client Location

Because one vantage point had hidden that fault, the probe was deployed as an AWS Lambda function in five regions, run twenty times in each, and deleted afterwards. Figure 6.8 shows the result.

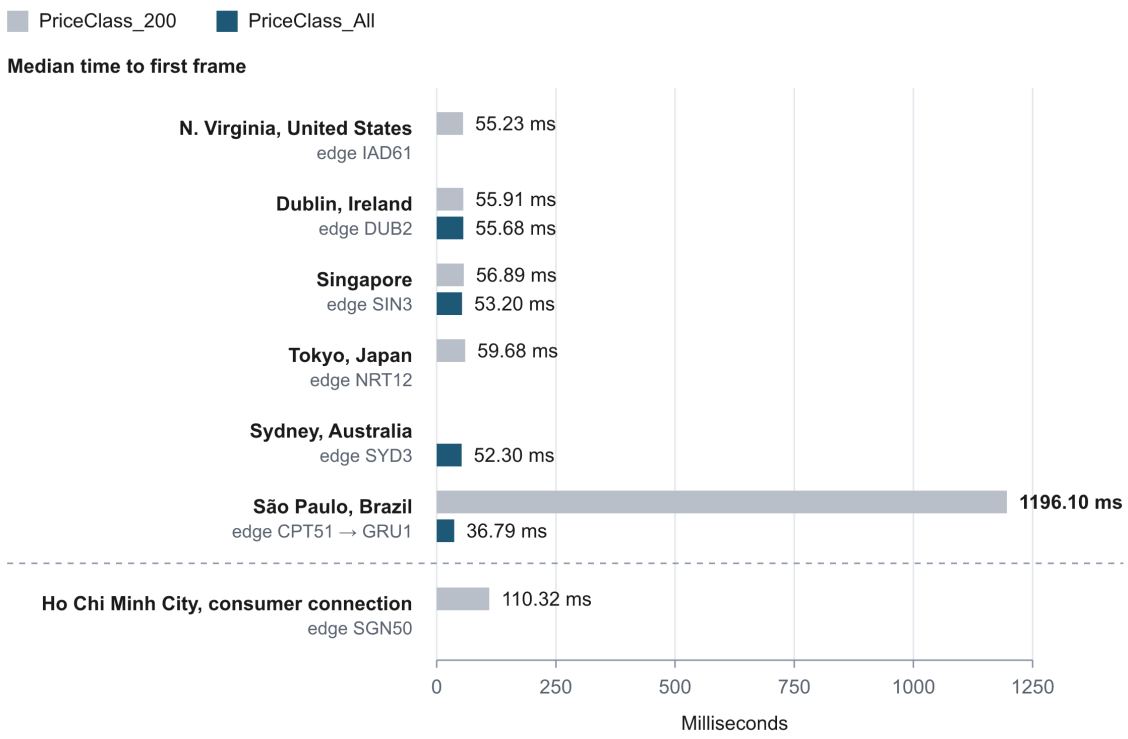


Figure 6.8: *Median time to first frame by client location, before and after* `PriceClass_All`

Note: milliseconds, $n = 20$ per location and configuration. A location with one bar was measured in one configuration only. The Ho Chi Minh City bar is the consumer-connection measurement of Figure 6.7, shown for comparison. Medians over cache hits

are in Table B.3 (Appendix B).

Under `PriceClass_200`, four locations clustered between 55 and 60 ms, each served by an edge in its own region. São Paulo was the exception: it was served by `CPT51` in Cape Town at a median of 1196 ms, because this price class excludes South America and Oceania. Both distributions were therefore moved to `PriceClass_All`. Brazil is now served by `GRU1` in São Paulo at 36.79 ms, a thirty-two-fold improvement, and Sydney by `SYD3` at 52.30 ms, while the two control locations moved by less than 4 ms.

The system has no users in South America or Oceania, so this change met no existing need. It was adopted because it costs nothing in practice: the CloudFront free tier covers one terabyte of transfer and ten million requests a month, far above this deployment's traffic. If traffic ever exceeds that allowance, this setting should be revisited first, since those regions have the highest per-gigabyte rates.

These probes ran inside AWS regions over well-provisioned paths, which is why they report about 55 ms while the consumer connection in Ho Chi Minh City reports 110 ms to an edge in the same city. The figures therefore measure edge proximity under favourable conditions, not what a home viewer sees; on this evidence the last mile costs roughly a factor of two.

6.3.3 Playback Under Constrained Bandwidth

Rebuffering, bitrate switches and the bitrate a viewer receives exist only while a player is playing, so HTTP requests alone cannot measure them. The harness written for this drives the deployed watch page in headless Chromium with Playwright, reads the `waiting`, `playing`, `ended` and `resize` events of the <video> element, and caps bandwidth through the Chrome DevTools Protocol. Startup buffering is excluded from the rebuffering ratio, the first rendition choice is not counted as a switch, and the average bitrate is computed from the bytes received, weighted by the time each rendition was on screen. Each profile discards two warm-up runs before recording five, because with one warm-up the first slow 3G run took 26.04 s to start against about 1.90 s for the others.

Table 6.1: Playback quality by emulated network profile

METRIC	UNTHROTTLED	FAST 3G	SLOW 3G
Startup delay, median (s)	2.576	2.076	3.803
Startup delay, max (s)	5.221	2.389	3.920
Rebuffering ratio, median	0.00%	0.00%	5.72%
Rebuffering ratio, mean	0.00%	1.20%	6.49%
Rebuffering ratio, max	0.00%	5.99%	14.83%
Runs exhibiting a stall	0/5	1/5	3/5
Bitrate switches, median	1	4	5
Average bitrate (kbit/s)	1833	1551	1200

Note: five recorded runs per profile, 120 s of playback each, one 4:23 source. Emulated ceilings are 1678 kbit/s for fast 3G and 512 kbit/s for slow 3G. Rebuffering is given as median, mean and maximum because stalls are rare but severe: the fast 3G median of 0.00% hides one run at 5.99%.

Computing the received bitrate exposed a defect in the master playlist. The `BANDWIDTH` attribute of each variant was written as the sum of two configured target rates, without looking at the encoded output. RFC 8216 (Section 4.3.4.2) defines it as the peak segment bit rate of the variant and requires it to be derived from the segments when they exist, as they do here. Table 6.2 compares the declared value with the actual peak and mean, computed from the size and the `#EXTINF` duration of every segment.

Table 6.2: Declared against actual bitrate for each rendition

RENDITION	DECLARED	PEAK	MEAN	SHORTFALL
360p	464	637	514	37%
720p	1628	2148	1714	32%
1080p	4192	5390	4338	29%

Note: kbit/s; $n = 44$ segments per rendition. The shortfall is how far the declared value falls below the actual peak.

A declared value below the real peak leads hls.js to choose a rendition the connection cannot sustain. Fast 3G sits on that boundary: its 1678 kbit/s ceiling admits the declared 1628 kbit/s of the 720p rung, but not its actual mean of 1714 or peak of 2148. The correction computes `BANDWIDTH` from the segments, taking the peak and rounding up. Existing videos were repaired by rewriting only the master playlist, so the segments stayed the same and the two rounds of Table 6.3 differ in nothing else.

Table 6.3: Playback quality before and after the manifest correction

PROFILE	REBUFFERING	STALLED	STARTUP MAX	BITRATE
Unthrottled	0.00% → 0.00%	0/5 → 0/5	3.29 → 5.22	1833 → 1833
Fast 3G	2.48% → 1.20%	2/5 → 1/5	16.81 → 2.39	1823 → 1551
Slow 3G	6.49% → 6.49%	3/5 → 3/5	3.83 → 3.92	1236 → 1200

Note: rebuffering is the mean of five runs, startup maximum in seconds and average bitrate in kbit/s. Only `BANDWIDTH` *changed between the two rounds.*

The correction helps only on fast 3G. There the worst startup delay falls from 16.81 s to 2.39 s, mean rebuffering halves and the stalled runs drop from two to one. The price is a 15% lower average bitrate and a rise in the median number of switches from one to four, as the player now adapts instead of holding a level it cannot carry. The unthrottled profile is a control and does not change: its startup maximum moved from 3.29 to 5.22 s through one slow run, with rebuffering at zero in both rounds. Slow 3G does not change either, to within 0.02 percentage points, because even the lowest rung, with an actual peak of 637 kbit/s, exceeds its 512 kbit/s ceiling. A manifest cannot help a player that has no lower rung to choose, so the remedy for this case would be a lower rung.

Three limits apply to these results. The peak that the harness observes is a lower bound, because it sees only the segments the player fetched. The profiles ran in sequence, so the edge cache state is confounded with profile order for startup delay, which likely explains why fast 3G started faster than the unthrottled profile. And the harness exports its event log in the input format of the ITU-T P.1203 model but computes no MOS score, so none is claimed.

6.3.4 Effect of Content Complexity

The results above come from one source video, so they could describe that file rather than the system. The attribute that plausibly governs playback is visual complexity, measured here as the bitrate of a sixty-second excerpt encoded at constant quality (CRF 23): at constant quality the encoder lets the bitrate follow the content. Three sources measured 151, 405 and 2298 kbit/s, a spread of 15.2 times, and are called low, medium and high below. Each source ran under all three profiles before the next began, so the effect of profile order cancels when sources are compared.

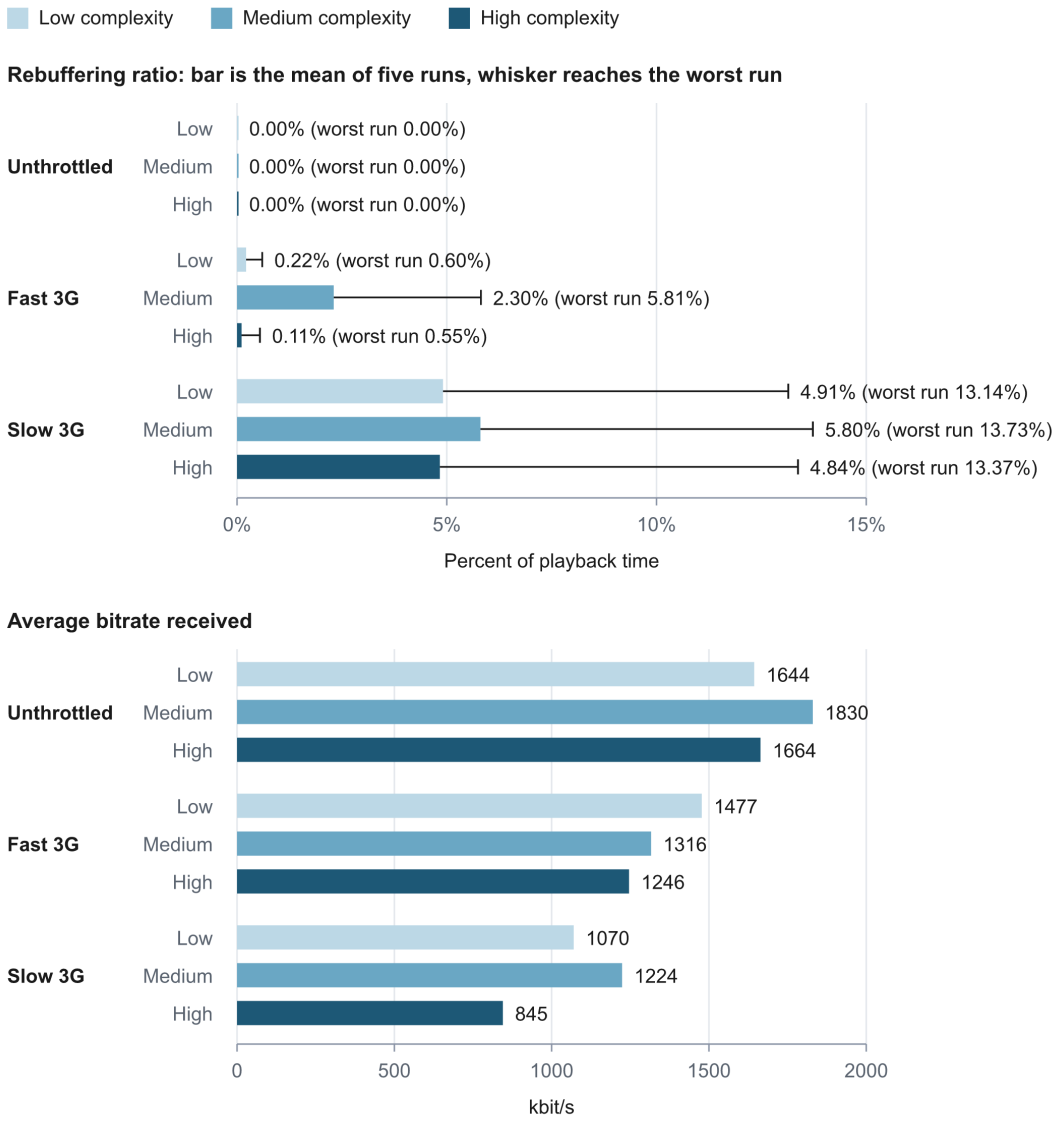


Figure 6.9: *Rebuffering ratio and average bitrate for three sources of differing visual complexity under three network profiles*

Note: five recorded runs per combination after two warm-up runs, 120 s of playback each, at a device pixel ratio of one. Source durations are 4:23, 4:23 and 4:40. Stalls and median startup delays are in Table B.4 (Appendix B).

Complexity does not order the result (Figure 6.9). Under fast 3G the mean rebuffering ratio is 0.22%, 2.30% and 0.11% for low, medium and high; under slow 3G the three lie within one percentage point, and unthrottled they are all zero. The reason is the encoder configuration. The ladder targets fixed bitrates in average-bitrate mode, so the encoder spends the same budget whatever the content, and the 720p rung averages 1650, 1694 and 1652 kbit/s for the three sources. Content is flattened when it is encoded, and little remains for it to influence at playback.

A second finding concerns the unthrottled rows. The player enables the hls.js option `capLevelToPlayerSize`, which limits the renditions to those needed to fill the video element, scaled by the device pixel ratio. The player column is about 880 px wide on any display, so at a ratio of one the player needs at most 1280×720 and the 1080p rung is unreachable: it received one segment in all nine measurements. Laptops commonly report a ratio between 1.25 and 2, so the unthrottled profile was measured again at a ratio of two (Table 6.4).

Table 6.4: Unthrottled playback at device pixel ratio one against two

SOURCE	DPR 1	DPR 2	RUNGS, DPR 1	RUNGS, DPR 2
Low	1644	4318	360p, 720p	360p, 1080p
Medium	1830	4312	360p, 720p	360p, 1080p
High	1664	4206	360p, 720p	360p, 720p, 1080p

Note: average bitrate in kbit/s over five recorded runs per cell. Rebuffering was 0.00% in every run.

At a ratio of two the average bitrate rises 2.5 to 2.6 times with no stall, and the median startup delay moves by at most 0.34 s, so the earlier unthrottled figures were limited by the resolution cap, not by the network. The middle rung is seldom used on a fast connection, since the low and medium sources jump from 360p straight to 1080p. With 1080p now fetched in quantity, the observed peak matched the declared `BANDWIDTH` exactly for all three sources, at 4 601 399, 5 389 584 and 5 495 113 bit/s, so the corrected value is reached by a real segment rather than overstated. Two limits remain: the 120 s window covers only about the first twenty of the forty-four segments, and under fast 3G two sources showed a bimodal startup, near 1.2 s and near 7.5 s, whose cause was not established.

6.4 Transcoding Pipeline Performance

Pipeline latency runs from the end of the upload to the moment the record reaches `READY`, which is the interval a user actually waits. It includes the S3 event delay, queueing in SQS, container start-up on Fargate, FFmpeg and the upload of the results. These measurements predate the six-rung ladder and chunked transcoding: one container per video produced 360p, 720p and 1080p, the configuration that Section 6.5 later compares against. The job definition revision that each job ran was checked afterwards, so the allocation of every row is confirmed rather than taken from the label the script recorded.

Table 6.5: Transcoding pipeline latency by input size

SIZE	DURATION	UPLOAD (S)	PIPELINE (S)	REALTIME FACTOR
100 MB (n=3)	120 s	3.13	475.80	3.97
500 MB (n=1)	600 s	13.13	2281.25	3.80
1 GB (n=1)	1600 s	72.00	5917.03	3.70

Note: 1 vCPU and 2 GB per job; mean across runs. The realtime factor is pipeline time divided by video duration, so a value above 1.0 means transcoding took longer than the video.

The realtime factor stays between 3.70 and 3.97 across a thirteen-fold range of duration, so at fixed compute the pipeline time grows linearly with duration. The three 100 MB runs, 346.32, 539.80 and 541.28 s, show no cold-start pattern, since the first was the fastest; at this scale the variation of Fargate Spot capacity appears to dominate. The 500 MB and 1 GB sizes were measured once each, because each run occupies 30 to 100 minutes of AWS Batch time.

Raising the container from 1 vCPU and 2 GB to 4 vCPU and 8 GB reduced the mean pipeline time from 475.80 s to 115.56 s for 100 MB (three runs each), and from 5917.03 s (one run) to 1078.70 s for 1 GB (two runs, 1080.29 and 1077.10 s), speedups of 4.12 and 5.49 times. Both exceed the four times that the vCPU count alone could explain. Amdahl's Law bounds the speedup below the resource ratio, because parts of an encoder such as entropy coding and rate control do not parallelise, and multi-threaded H.264 studies report this sub-linear pattern: 1.97, 3.96 and 7.71 times at 2, 4 and 8 threads [16], and 4.7 to 8.6 times across six Cell SPEs [17].

Two confounds explain the excess. Memory rose from 2 to 8 GB at the same time as the vCPU count, and the 1 vCPU baseline is unstable: its three 100 MB runs spread by 56.3%, against 24.0% at 4 vCPU. The speedup is therefore 3.29 comparing the fastest runs, 4.12 the means, 4.15 the slowest runs and 4.86 the medians. The data support a reduction of roughly four times, but not a precise scaling law. Cost per video is unaffected either way: Fargate bills per vCPU-second, so four times the rate for a quarter of the time leaves the charge per video about the same, and the 4 vCPU configuration buys a much shorter wait at almost no extra cost.

6.5 Chunked Transcoding of Long Videos

A single container transcodes at a roughly constant fraction of the video's runtime, and one AWS Batch job is limited to two hours. A five-minute video took 26.7 minutes as

a single job, which extrapolates to about twenty-one hours for four hours of content, so the requirement to accept videos of four hours or more needs the chunked path of Section 4.2.2. The packets of the produced renditions were checked for gaps and overlaps at every chunk boundary, and the check passed in every run below.

The measurements ran in the staging environment, which has the same container image, job definitions and compute settings as production but its own database and buckets. Capacity is the Fargate vCPU quota: 8 vCPU of Fargate Spot, which is the production quota, or 16 vCPU when an On-Demand environment of 8 vCPU was attached behind it. Each chunk runs in a 4 vCPU, 8 GB task, the size chosen in Figure 6.10. Latency runs from the end of the upload to `READY`. The 25-minute source is a 1080p H.264 file of 2.75 GiB with AAC audio. The four-hour source repeats a 1,600-second 1080p test pattern nine times with a 128 kbit/s tone, and was uploaded with S3 multipart, as the browser does for files larger than 512 MiB.

Table 6.6: Single-container path against chunked path for videos of five to fifteen minutes

VIDEO LENGTH	SINGLE JOB (MIN)	CHUNKED (MIN)	SPEED-UP
5 min	26.7	11.9	2.2
10 min	≈ 53.0	27.2	≈ 1.9
15 min	≈ 80.0	17.4	≈ 4.6

Note: values marked ≈ for the single-container path are extrapolated at 5.3 times the video duration from the measured five-minute run. The chunked values come from three videos sharing one 8 vCPU quota, so they overstate what a lone submission would see.

Table 6.6 explains the five-minute threshold. A five-minute video finished in 11.9 minutes on the chunked path against 26.7 minutes as one job, 2.2 times faster despite about 4.4 minutes of fixed cost for planning, audio encoding and finalisation. For a shorter video the fixed cost would exceed half of the processing time, so shorter videos keep the single job.

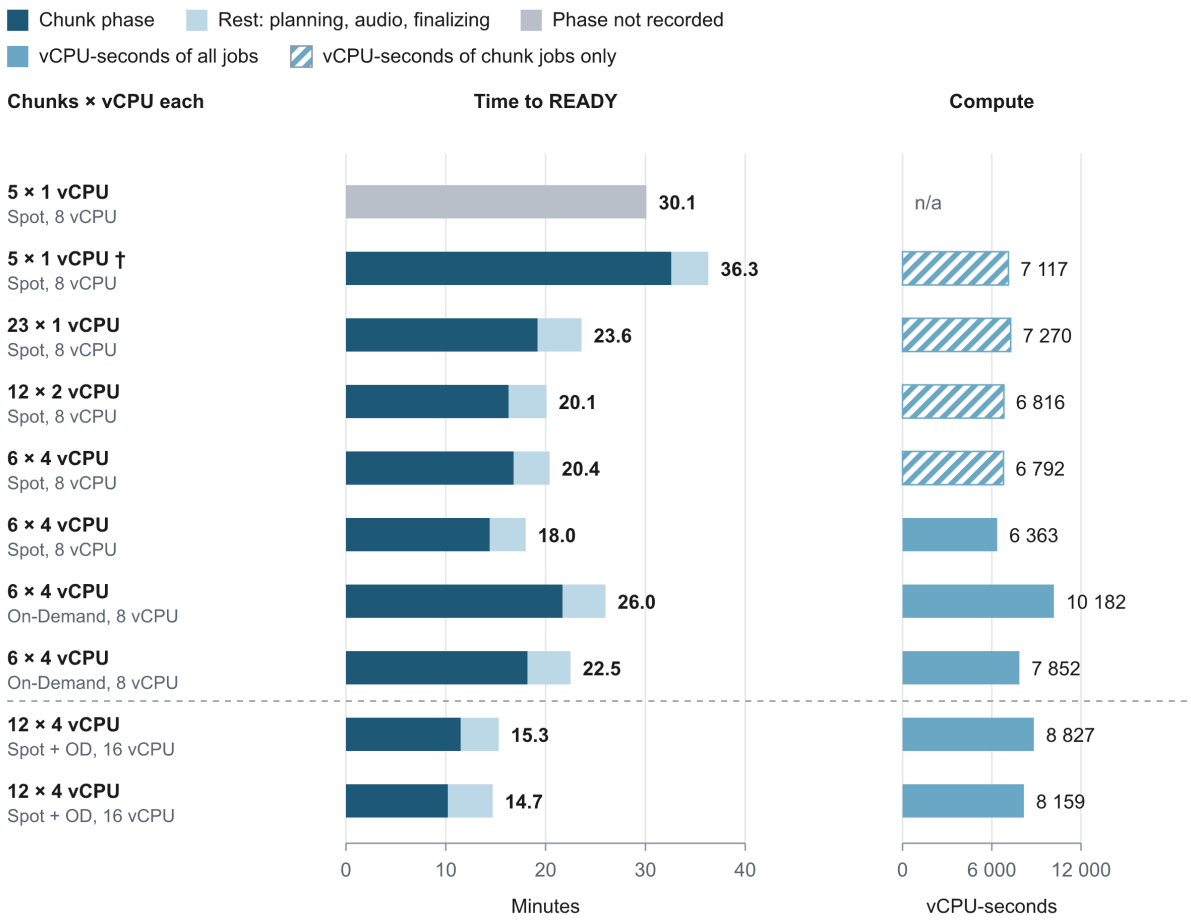


Figure 6.10: *A 25-minute video on the chunked path: time to READY and compute by chunk configuration and capacity*

Note: the first four bars were timed from the creation of the first job and the others from the end of the upload, about half a minute apart. † marks the run on production. The pale part of a time bar is planning, audio and finalisation. Hatched bars count chunk jobs only and plain bars every job, so the two kinds are not comparable. Phase lengths are in Table B.5 (Appendix B).

Figure 6.10 follows the 25-minute video through the changes made to the path. Five chunks of five minutes on 1 vCPU tasks took 36.3 minutes; they used only five of the eight vCPU, and chunks of equal size took between 18.2 and 32.6 minutes because Fargate Spot hosts differ. Choosing the chunk length from the capacity, which gave 23 chunks, cut the time to 23.6 minutes, and moving to 4 vCPU tasks cut it to 20.4 and, on a repeat, 18.0 minutes; a 4 vCPU task encoded a second of video 4.7 times faster than a 1 vCPU task. Twelve 2 vCPU tasks gave the same time and vCPU-seconds as six 4 vCPU tasks, so the larger size was kept. At 16 vCPU the video took 14.7 and 15.3 minutes, only 1.2 times faster for 1.3 to 1.4 times the vCPU-seconds, because about three minutes of fixed phases remain and each of the twelve smaller chunks pays a container start-up.

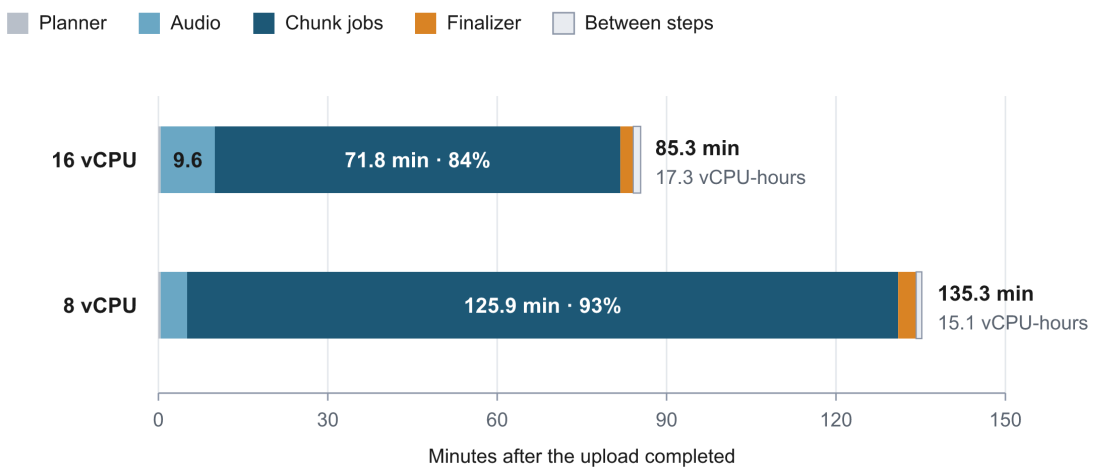


Figure 6.11: *A four-hour video: time from the end of the upload to READY, by step, at 16 and at 8 vCPU*

Note: the pale segment at the end of each bar is queueing and hand-over between jobs. Steps too short for a label are given in Table B.6 (Appendix B).

At the production capacity of 8 vCPU, a four-hour video reached `READY` 135.3 minutes after its upload finished, and at 16 vCPU 85.3 minutes (Figure 6.11). No chunk ran longer than seven minutes, far inside the two-hour limit of a job. The first measurement of the path, with 1 vCPU chunks at 8 vCPU, took 185.0 minutes, so the later changes saved 27% at production capacity. Capacity helped the long video more than the 25-minute one, 1.6 times against 1.2, because 48 chunks keep every slot busy through many rounds and the fixed phases are a small share of the total. Compute was 15.1 vCPU-hours at 8 vCPU and 17.3 at 16 vCPU, so doubling the capacity cost about 15% more compute for 37% less time.

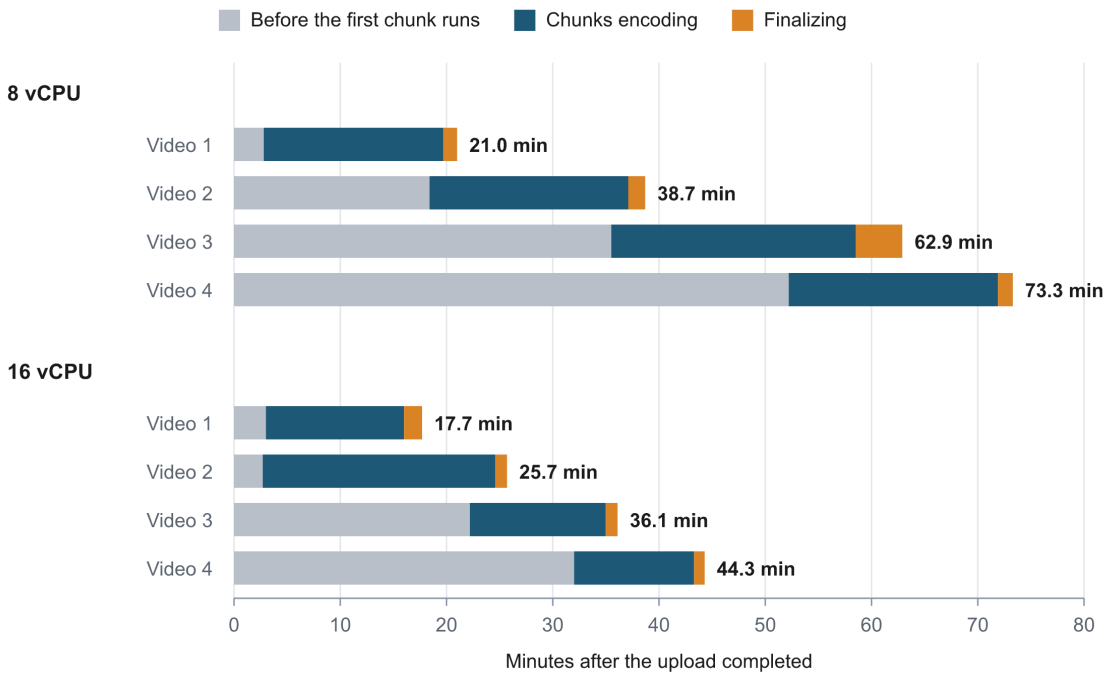


Figure 6.12: *Four 25-minute videos submitted together: when each waited, encoded and finalised, at 8 and at 16 vCPU*

Note: at 8 vCPU the four videos were uploaded together over one connection and timed from the first job creation; at 16 vCPU they were server-side copies of one file, so all four arrived within a second. Videos are numbered in order of completion. Chunk phases are in Table B.7 (Appendix B).

When four 25-minute videos arrived together (Figure 6.12), 8 vCPU finished them at 21.0, 38.7, 62.9 and 73.3 minutes, 1.4 times real time, and 16 vCPU at 17.7, 25.7, 36.1 and 44.3 minutes, 2.3 times real time. Doubling the capacity shortened the batch 1.65 times, closer to proportional than for one video, because the extra slots were always filled with chunks of the other videos. The queue is first-in first-out within a priority, so the videos finished in the order they arrived.

Table 6.7: Simultaneous submissions at 16 vCPU

BATCH	VCPU	READY	ALL (MIN)	P50 (MIN)	P95 (MIN)	PER MIN	VCPU-H
4 × 25min-1080p	16	4/4	44.3	36.1	44.3	0.09	9.2
100 × 60s-1080p	16	100/100	35.8	17.9	33.7	2.80	8.3

Note: ALL is the time of the whole batch, P50 and P95 the median and 95th percentile of a video's latency from its arrival, PER MIN the videos completed per minute, and VCPU-H the vCPU-hours used.

One hundred 60-second clips submitted at once at 16 vCPU all reached `READY` without a failure (Table 6.7). The batch took 35.8 minutes, the median clip waited 17.9 minutes and the 95th percentile 33.7, at 2.8 videos per minute for 8.3 vCPU-hours of compute. The fastest clip needed 3.7 minutes, the time of a single job running alone, so the rest of the latency is queueing: sixteen 1 vCPU jobs run at a time, and one hundred clips need about six rounds. The capacity quota, not the architecture, set the drain rate.

Fargate On-Demand was not faster. The 25-minute video on 8 vCPU of On-Demand took 26.0 and 22.5 minutes against 18.0 on Spot, with chunk medians of 414 and 313 seconds against 255 seconds. Two runs cannot show that On-Demand is slower, since Fargate does not guarantee the host hardware, but they give no reason to use it except as overflow when the Spot quota is exhausted. Spot was also reliable: in seven days of AWS Batch job history, 2 of 370 executions (0.5%) were interrupted, and AWS Batch retried both successfully.

Several limits apply to these figures. Most rows are single runs, and chunks of equal size varied by up to 1.8 times within one job, so differences of about ten percent between rows are not an effect. The four-hour source is a synthetic pattern at 5.35 Mbit/s that may encode faster than camera footage, although its chunks ran at 0.9 seconds per second of video against 1.0 for the 25-minute file. The 16 vCPU rows describe a quota that production does not have, and staging carried no other traffic during the runs.

6.6 Load Testing

The load test checks that a burst of uploads loses no work and that AWS Batch runs it at its configured concurrency. Fifty virtual users perform one hundred upload cycles, each of which initiates an upload, sends the file to S3 with the pre-signed URL and confirms it; the job queue is then polled every 25 seconds until it is empty. The compute environment allowed 8 vCPU and each job 1 vCPU, so at most eight jobs could run at once.

6.6.1 Queue and Admission Under Burst

The first runs uploaded a small synthetic byte string instead of a video, to exercise the upload API, the S3 event, SQS and job submission. Their jobs failed at container start-up because of the empty secret described in Section 5.5. This does not affect the queue

and admission behaviour measured here, but it means that the drain rate of these runs is not a transcoding throughput. The test was run with two independent load generators, a Node.js script and k6, so that a bottleneck in the generator could not pass for one in the system (Table 6.8).

Table 6.8: Cross-validation of the load test across two independent tools

METRIC	NODE.JS	K6
Upload cycles completed	100	100
Failure rate	0%	0%
Batch jobs queued (RUNNABLE)	92	92
Batch jobs admitted (STARTING)	8	8
SQS messages remaining	0	0
Queue drain, 76 jobs (s)	416	417
Consumption rate (jobs/min)	10.96	10.94

All 100 cycles succeeded with both tools. The Node.js script completed 15.52 cycles per second, with a mean of 2388.52 ms and a 95th percentile of 3317.79 ms per cycle. k6 times each request rather than each cycle: its mean of 670.03 ms over 300 requests is 2010.09 ms per three-request cycle, within 15.8% of the Node.js figure, and its steady-state rate of 17.24 cycles per second is close to 15.52.

The two tools agree on every observation of the queue. Immediately after each burst 92 jobs were waiting and 8 had been admitted, and SQS held no messages, so the Lambda submitter had moved every event into AWS Batch within seconds. The number of starting jobs stayed at seven or eight throughout, exactly `max_vcpus` of 8 divided by 1 vCPU per job. From the point where 76 jobs remained, the queue emptied in 416 s with Node.js and 417 s with k6, and no message was lost. SQS therefore kept every unit of work submitted faster than it could be processed, and AWS Batch admitted work at, and never above, its limit.

6.6.2 Throughput With Real Video

After the secret was fixed, the test was repeated with a real three-second H.264 clip in the same scenario. In the first such run all 300 HTTP requests and all 100 Batch jobs succeeded, at 23.47 requests per second with a 95th-percentile request time of 1827 ms. A second run, polled every 25 seconds, gives the drain curve: again 300 requests without failure, at 33.12 per second, all 100 jobs succeeded, and all 100 video records reached `READY` in the database. Table 6.9 compares it with the synthetic run.

Table 6.9: Backlog drain under synthetic and real payloads

	SYNTHETIC	REAL VIDEO
Jobs submitted	100	100
Jobs succeeded	0	100
Drain duration (s)	456	826
Drain rate (jobs/min)	13.16	7.26
Peak concurrent `RUNNING`	0	8
Implied wall-clock per job (s)	n/a	66.1

Note: both columns sampled every 25 s. The synthetic jobs failed at start-up, so none succeeded or reached `RUNNING`. Implied wall-clock per job is the drain duration times the observed concurrency divided by the number of jobs.

The real backlog drains at 7.26 jobs per minute against 13.16 for the synthetic one, so doing the work costs a factor of 1.8 in throughput compared with only admitting and rejecting it. `RUNNING` peaked at exactly eight, matching the configured limit, which the synthetic run could not show because its jobs never reached that state. At eight-way concurrency, 826 seconds for 100 jobs is about 66 seconds of wall-clock time per job for a three-second clip. Almost all of that is fixed cost, container provisioning, image pull and result upload, rather than encoding, so for short uploads it is this fixed cost of about a minute, not the speed of FFmpeg, that decides how long a user waits.

6.7 Cost Analysis

The comparison sets an always-on Amazon EC2 instance running FFmpeg against the serverless container architecture, for three hundred videos a month averaging five minutes each. The processing effort comes from the measurements of Section 6.5: a 25-minute video used 6,363 vCPU-seconds at the best configuration and a four-hour video 15.1 vCPU-hours, about 0.35 vCPU-hours per five minutes of video, so the month needs about 105 vCPU-hours. In other words, encoding one second of 1080p video takes about four vCPU-seconds.

Prices are the ap-southeast-1 list prices read from the AWS Price List API on 15 October 2026: 0.05056 USD per Fargate vCPU-hour and 0.00553 USD per GB-hour, which is 0.0616 USD per vCPU-hour with the 2 GB per vCPU of the chunk tasks. Spot capacity is taken at 30% of that, 0.0185 USD, since AWS publishes discounts of up to 70%; the real Spot price varies with demand.

Table 6.10: Monthly operating cost comparison for the part of the bill that the architecture controls (ap-southeast-1)

COMPONENT	EC2 (ALWAYS ON)	SERVERLESS	SAVING
Compute	`c5.xlarge` for 730 h: $124.10	Fargate Spot for 105 vCPU-h: $1.94	98.44%
Storage	EBS 100 GB plus S3 50 GB: $9.25	S3 with Glacier lifecycle: $1.45	84.32%
Queueing	Self-hosted broker	SQS and Lambda: $0.01	n/a
Total	**$133.35**	**$3.40**	**97.45%**

Two costs are left out of Table 6.10 because they do not depend on the architecture, and both are larger than the serverless column. Content delivery costs the same whichever compute model produced the renditions, about 12 USD a month after the free tier. Content screening is needed by both: a five-minute video sampled every five seconds gives 60 images, which at the Singapore price of 0.00125 USD per image is 0.075 USD per video and 22.50 USD for three hundred videos. With both costs added to both columns the totals are $167.85 against $37.90, a saving of about 77.4%, which is the figure to quote for total cost of ownership; the saving on the part that the architecture controls is 97.5%.

The saving comes entirely from removing idle capacity. The always-on instance offers 2,920 vCPU-hours a month, four vCPUs for 730 hours, while the work needs about 105 of them, a utilisation of roughly 3.6%. The advantage is therefore not universal: as sustained utilisation rises, per-second Fargate billing approaches and then exceeds the cost of a reserved instance. The architecture suits bursty workloads, which user-generated video uploads are.

6.8 Requirements Against the Evidence

Table 6.11 sets each non-functional requirement of Table 3.3 against the evidence in this report. NFR-01 states no numeric target, so its row reports the measurement rather than a pass or a fail.

Table 6.11: Non-functional requirements against the evidence

REQUIREMENT	EVIDENCE	SECTION
NFR-01 Latency	Median time to first frame of 110.32 ms from Ho Chi Minh City through the CDN, against 251.52 ms from the S3 origin.	6.3.1
NFR-02 Scalability	In a burst of 100 uploads, SQS kept every message and AWS Batch ran jobs at its configured limit; 100 clips submitted at once all reached `READY`.	6.5, 6.6
NFR-03 Reliability	AWS Batch retries a failed job and raises an alert with the reason when the attempts run out; the 2 Spot interruptions in 370 executions were retried successfully.	5.5, 6.5
NFR-04 Cost efficiency	Compute is billed only while a job runs: 3.40 USD a month against 133.35 USD for an always-on instance.	6.7
NFR-05 Security of content	A request without a valid Signed Cookie is refused at the edge, as verified on the deployed distribution; the visibility test suite covers the API side.	4.4, 5.5, 5.6
NFR-06 Security of the API	Authentication endpoints allow ten attempts per fifteen minutes per address, and cross-origin requests are limited to an allow-list.	5.1
NFR-07 Reproducibility	Thirteen Terraform modules describe the environment, which is created with `terraform apply` from one directory.	5.3, A.5
NFR-08 Maintainability	Seven workflows run tests and security scanning, and the security gate blocks a merge on fixable critical findings.	5.4

Chapter 7

Conclusion and Future Work

This chapter compares the results with the objectives of Section 1.2, then states the strengths and limitations of the delivered system and the work that would extend it.

7.1 Summary of Achievements

The project set out to build a video sharing platform whose transcoding runs on serverless containers driven by events, and to measure whether that choice delivers the cost and scalability it promises. Table 7.1 sets each objective against its result and the section that holds the evidence.

Table 7.1: Objectives of the project against their results

OBJECTIVE	RESULT	EVIDENCE
1. Web platform	Users register with a verified e-mail address, upload directly to Amazon S3, watch with adaptive bitrate over up to six renditions, set each video public, unlisted or private, manage their channel, search the catalogue, and like and comment. A private video is refused at the edge even when its manifest URL is known.	Sections 4.4, 4.6, 6.2
2. Automated pipeline	An upload event passes through Amazon SQS and AWS Lambda to AWS Batch, which produces the renditions, the master playlist and a thumbnail without manual action. Conditional writes stop a duplicate delivery from overwriting a finished job, and a failed job is retried and then raises an alert with its reason.	Sections 4.2, 5.5
3. On-demand containers	FFmpeg runs in a container on AWS Batch over Fargate that exists only while a job runs. Videos of five minutes or more are split into chunks encoded in parallel, so a four-hour video was ready 135 minutes after its upload at 8 vCPU and 85 minutes at 16 vCPU.	Sections 4.2.2, 6.5
4. CI/CD and IaC	Thirteen Terraform modules describe three environment definitions, and seven GitHub Actions workflows run linting, tests, security scanning, infrastructure validation and deployment. The suite holds 853 automated tests in 52 suites.	Sections 5.3, 5.4, 5.6
5. Empirical evaluation	Time to first frame, playback under constrained bandwidth, transcoding latency, burst load and monthly cost were measured on the deployed system.	Chapter 6

One result goes beyond the five objectives. The platform screens every upload with Amazon Rekognition and holds doubtful videos for staff, who work in a console with separate roles: operators decide, auditors read the error rates, and administrators assign roles. Each decision is stored beside the machine's own verdict, so the error rates can be estimated with confidence intervals, and the owner of a removed video can appeal (Section 4.5).

7.2 Strengths of the System

The application tier never handles video bytes. The backend issues pre-signed URLs, records metadata and answers queries, while video flows directly between the browser, Amazon S3 and CloudFront. The load on the API therefore does not depend on video size or upload concurrency, and a failure in the processing tier leaves published videos playable.

The processing tier costs nothing while idle, because a transcoding container exists only while a job runs. For the part of the bill that the architecture controls, Section 6.7 estimates $3.40 a month against $133.35 for an always-on instance; with screening and delivery added to both sides, the saving is about 77%.

Additional capacity turns directly into lower latency. Chunk jobs are ordinary AWS Batch jobs on the same compute environment, so a larger vCPU quota encodes more chunks at once, and no chunk came near the two-hour limit of a job (Section 6.5).

The performance claims are measured rather than assumed. Measuring time to first frame on the deployed CDN showed that its price class served Vietnam from Europe, and measuring from other regions showed the same gap for Brazil; the two corrections reduced the median latency by a factor of 6.6 from Vietnam and 32 from Brazil (Section 6.3). The automated suite likewise caught a cookie signing defect before deployment (Section 5.6).

7.3 Limitations

Transcoding. The bitrate ladder has six fixed rungs whatever the content. Optimising the ladder per title or per shot can save bandwidth; Netflix reported reductions of 28% to 38% at equal perceptual quality [7]. Extra capacity also helps a short video little: a 25-minute video took 18.0 minutes at 8 vCPU and about 15 minutes at 16 vCPU, because planning, audio encoding and finalisation take about three minutes whatever the capacity. Production has a Fargate Spot quota of 8 vCPU, so four 25-minute videos submitted together finished after 73 minutes. Below five minutes a video keeps the single-container path, where fixed overhead dominates: about 66 seconds per job for a three-second clip (Section 6.6.2).

Moderation. Screening samples one frame every five seconds, so content shown for less than that can pass. Its thresholds of 60 and 90 per cent follow the provider's guidance and were not calibrated on this platform's content. The 147 backend tests of moderation cover the implementation, but the error rates have not been measured on real traffic, so this report makes no claim about the accuracy of the screening. Screening is also the largest line of the monthly bill, 22.50 USD under the stated workload.

Accounts and access. A new password is checked against common passwords, the user's own details and a breach service [14], but existing passwords were not rechecked and the breach lookup is skipped when the service does not answer within two seconds. Mail to one address is capped at five per hour, but the count lives in process memory and registration has no CAPTCHA. Signed Cookies stay valid for two hours, so making a video private does not revoke cookies already issued, and the system has no digital

rights management.

Operation. Rate limits and view deduplication are held in process memory, which is correct only while the backend is one process. That process runs on one instance in one region and is the only single point of failure of the system. The CloudFront distributions live in a second AWS account, because the primary account was not permitted to create them, which splits the infrastructure across two sets of credentials.

Interface. Tags can be set only at upload, and a thumbnail cannot be replaced. The account stores no language preference, so the e-mails sent by a transcoding job, which has no request to read a language from, are written in both languages.

The limits of each measurement, such as single runs, a synthetic four-hour source and probes inside AWS regions, are stated beside the result in Chapter 6 and are not repeated here.

7.4 Future Work

The first extension is a per-title bitrate ladder chosen with a perceptual quality metric such as VMAF. Encoding a short probe of the source at several operating points and keeping those on the convex hull of quality against bitrate would lower delivery bandwidth, and with it the delivery cost, at the same perceived quality.

The second is less fixed overhead on the chunked path. Each chunk pays a container start-up of tens of seconds. Encoding several chunks in one longer-lived worker, or raising the minimum chunk length, trades parallelism for less overhead; the measurements of Section 6.5 suggest that the best choice depends on the video length and the quota. A faster x264 preset, checked with VMAF on difficult clips, could shorten every chunk.

The third is removing the single point of failure. A second backend instance behind a health-checked address needs a load balancer and a shared store, such as Redis, for the rate limits, view deduplication and mail count; the persistent state is already in MongoDB Atlas and Amazon S3. A CAPTCHA on registration and password reset, and distributed tracing across the S3, SQS, Lambda and Batch boundaries, belong to the same step.

The fourth is tuning the screening from evidence. The console already records the reason codes of reversed removals and an unbiased audit sample; after a few hundred decisions, the thresholds and the label policy can be adjusted and the effect measured again.

Two measurements should also be repeated: the 1 vCPU baseline, with more runs and with memory held constant (Section 6.4), and time to first frame from residential connections rather than from AWS regions (Section 6.3.2). Further ahead, GPU-enabled

serverless encoding [18] and a more efficient codec such as HEVC or AV1 next to H.264 would raise throughput and reduce bandwidth.

Bibliography

[1] Netflix Technology Blog, "Rebuilding netflix video processing pipeline with microservices." https://netflixtechblog.com/rebuilding-netflix-video-processing-pipeline-with-microservices-4e5e6310e359, 2024. Accessed: 2026-08-10.

[2] Netflix Technology Blog, "The making of VES: the cosmos microservice for netflix video encoding." https://netflixtechblog.com/the-making-of-ves-the-cosmos-microservice-for-netflix-video-encoding-946b9b3cd300, 2024. Accessed: 2026-08-10.

[3] Amazon Web Services, "Video on demand on AWS: Implementation guide." https://docs.aws.amazon.com/solutions/latest/video-on-demand-on-aws/, 2025. Accessed: 2026-08-10.

[4] M. Zhang, Y. Zhu, C. Zhang, and J. Liu, "Video processing with serverless computing: A measurement study," in *Proceedings of the 29th ACM Workshop on Network and Operating Systems Support for Digital Audio and Video (NOSSDAV)*, 2019.

[5] M. Golec, G. K. Walia, M. Kumar, F. Cuadrado, S. S. Gill, and S. Uhlig, "Cold start latency in serverless computing: A systematic review, taxonomy, and future directions," *ACM Computing Surveys*, vol. 57, no. 3, 2025. Published version of arXiv:2310.08437.

[6] Netflix Technology Blog, "Per-title encode optimization." https://netflixtechblog.com/per-title-encode-optimization-7e99442b62a2, 2015. Accessed: 2026-08-10.

[7] Netflix Technology Blog, "Dynamic optimizer: A perceptual video encoding optimization framework." https://netflixtechblog.com/dynamic-optimizer-a-perceptual-video-encoding-optimization-framework-e19f1e3a277f, 2018. Accessed: 2026-08-10.

[8] Amazon Web Services, "Serve private content with signed URLs and signed cookies: Amazon cloudfront developer guide." https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/PrivateContent.html, 2025. Accessed: 2026-08-10.

[9] Amazon Web Services, "Moderating content, Amazon Rekognition developer guide." https://docs.aws.amazon.com/rekognition/latest/dg/moderation.html. Accessed: 2026-10-15.

[10] Amazon Web Services, "Guidelines and quotas in Amazon Rekognition." https://docs.aws.amazon.com/rekognition/latest/dg/limits.html. Accessed: 2026-10-15.

[11] "The Santa Clara principles on transparency and accountability in content moderation." https://santaclaraprinciples.org/, 2021. Version 2.0. Accessed: 2026-10-15.

[12] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[13] J. Ali, "Validating leaked passwords with k-anonymity." Cloudflare Blog, https://blog.cloudflare.com/validating-leaked-passwords-with-k-anonymity/, 2018. Published 2018-02-21. Accessed: 2026-10-16.

[14] P. A. Grassi *et al.*, "Digital identity guidelines: Authentication and lifecycle management," NIST Special Publication 800-63B, National Institute of Standards and Technology, 2017. Includes updates as of 2020-03-02. https://doi.org/10.6028/NIST.SP.800-63b. Accessed: 2026-10-16.

[15] M. Burrows, "The chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006.

[16] S. Sankaraiah, L. H. Shuan, C. Eswaran, and J. Abdullah, "Scalable video encoding with macroblock-level parallelism," *EURASIP Journal on Advances in Signal Processing*, vol. 2014, p. 145, 2014.

[17] C.-Y. Chen, S.-C. Cheng, *et al.*, "Task-based parallel H.264 video encoding for explicit communication architectures," in *Proceedings of the International Confer-*

ence on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS), 2011.

[18] A. Salcedo-Navarro, R. Peña-Ortiz, J. M. Claver, M. Garcia-Pineda, and J. Gutierrez-Aguado, "Towards GPU-enabled serverless cloud edge platforms for accelerating HEVC video coding," *Cluster Computing*, vol. 28, p. 68, 2025.

Appendix A

Supplementary Material

A.1 Repository Structure

The source code is organised as a monorepo with the following top-level layout.

```
DACNTT_VideoStreaming/
  frontend/            React single page application (Vite, HLS.js)
  backend/             Node.js Express API and test suite
  transcoder/          FFmpeg worker and Docker image
  infrastructure/
    modules/           Thirteen reusable Terraform modules
    environments/      dev, staging and prod definitions
  .github/workflows/   Seven CI/CD workflows
  scripts/             Benchmarking and load testing scripts
  docs/                Architecture diagrams, cost analysis,
   measured results
  report/              LaTeX sources of this report
```

A.2 FFmpeg Transcoding Command

The following invocation is the one the worker builds for a 1080p source at 24 fps, which fills all six rungs of the ladder, and it produces every rendition from one read of the source. A single input is followed by one output block per rendition, each carrying its own scaling filter and encoder settings, so FFmpeg decodes the source once and feeds all six filter chains from it. The blocks after the first are abbreviated here, and the 360p, 480p and 720p blocks are left out of the listing; in the source every block repeats the first in full. The master playlist is not written by FFmpeg: the worker generates it afterwards,

which is why no `-master_pl_name` argument appears. It does so from the encoded output rather than from the settings above, measuring each rendition's peak segment bit rate from the segment sizes and their `#EXTINF` durations, and reading the codec string for each variant out of the sequence parameter set of its first segment.

The keyframe interval is derived from the source rather than assumed. The worker probes the first video stream and reads `avg_frame_rate`, preferring it over `r_frame_rate` because the latter is a base rate FFmpeg infers from the timing grid and need not describe the frames that are actually present.

A screen recording taken from the project's own test material shows how far apart the two can be. The file reports `r_frame_rate` as 60000/1001, that is 59.94 fps, while `avg_frame_rate` reports 28.725 fps; counting frames confirms the second, at 287 frames in ten seconds. Every interval between consecutive frames turns out to be an integer multiple of 1/144 second — four, five or six ticks, giving instantaneous rates of 36, 28.8 and 24 fps — so the real timing grid is the 144 Hz display, and neither the 59.94 that `r_frame_rate` advertises nor the 30 the source game was capped at. The cap cannot be met exactly, because 144 is not divisible by 30 and a frame would have to occupy 4.8 ticks.

The consequence for the group of pictures is direct. Taking `avg_frame_rate` gives 172 frames, or 5.988 seconds, which fits inside the segment. Taking `r_frame_rate` would give 359 frames, or 12.5 seconds, more than twice the segment length. A fixed 30 fps assumption would give 180 frames, or 6.27 seconds, which also overruns. This is also why `-force_key_frames` is retained alongside `-g`: when the source rate varies between 24 and 36 fps, an interval counted in frames cannot correspond to a fixed interval in time, and only the time-based expression can.

Once the rate is known the group of pictures is `floor(fps × 6)` frames, floored rather than rounded so that it never exceeds the segment length. The same count of 180 frames illustrates why the rate has to be right before the arithmetic can be: it lasts 6.27 seconds at the 28.725 fps just measured, and 6.006 seconds at 29.97 fps, where it arises instead from rounding 179.82 upward. Both overrun a six-second segment, and in both cases the boundary drifts past the point where a keyframe exists. The value shown below, 144, is what a 24 fps source produces.

Two further arguments keep the boundaries aligned. Scene detection is disabled, because letting the encoder place keyframes at scene changes would allow the renditions to cut at different points, and RFC 8216 requires that matching content in variant streams carry matching timestamps. `-force_key_frames` then pins a keyframe to every sixth second of media time; unlike `-g`, which only bounds the interval from above, it is absolute

and independent of the detected frame rate, so a misdetected source still yields aligned segments.

The H.264 profile is pinned but the level is not. Each rendition carries a different frame size, and the lowest conforming level therefore differs between them, so the level is left for the encoder to derive from the resolution rather than fixed in the command. The pixel format is pinned to 8-bit 4:2:0 for the same reason as the profile: a 10-bit or 4:4:4 source would otherwise be encoded in a form that the main profile does not allow and most browsers do not decode.

```
ffmpeg -i input.mp4 -hide_banner -loglevel warning -stats -y \
  -vf "scale=256:144,setsar=1" \
  -c:v libx264 -b:v 120k -maxrate 150k -bufsize 240k \
  -preset fast -profile:v main -pix_fmt yuv420p \
  -g 144 -keyint_min 144 -sc_threshold 0 \
  -force_key_frames "expr:gte(t,n_forced*6)" \
  -c:a aac -b:a 48k -ar 44100 -ac 2 \
  -f hls -hls_time 6 -hls_list_size 0 \
  -hls_segment_filename 144p/segment_%03d.ts  144p/playlist.m3u8 \
  \
  -vf "scale=426:240,setsar=1" -c:v libx264 -b:v 240k -maxrate 300k
   -bufsize 480k \
  -c:a aac -b:a 64k -f hls -hls_time 6 \
  -hls_segment_filename 240p/segment_%03d.ts  240p/playlist.m3u8 \
  \
  ... 360p, 480p and 720p in the same form ... \
  \
  -vf "scale=1920:1080,setsar=1" -c:v libx264 -b:v 4000k -maxrate
   5000k -bufsize 8000k \
  -c:a aac -b:a 192k -f hls -hls_time 6 \
  -hls_segment_filename 1080p/segment_%03d.ts 1080p/playlist.m3u8
```

A.3 Signed Cookie Custom Policy

The policy signed by the backend before issuing playback cookies. The wildcard in the resource path covers the manifest and every segment of one specific video, and nothing beyond it.

```
{
  "Statement": [
```

```
    {
      "Resource": "https://cdn.zelostech.site/videos/<videoId>/*",
      "Condition": {
        "DateLessThan": { "AWS:EpochTime": 1786300347 }
      }
    }
  ]
}
```

A.4 Reproducing the Measurements

The experimental results reported in Chapter 6 are produced by the scripts listed below. Detailed instructions, including the environment variables required and the order in which the measurements must be taken, are given in `docs/results/README.md` within the repository.

Every table of measurements in Chapter 6 and in Appendix B is listed below against the script that produced it and the file its figures were read from, so that any individual number can be traced back to its source. Paths are relative to the repository root, and the result files are those under `docs/results/`.

Table A.1: Provenance of each measurement table

TABLE	SCRIPT	RESULT FILE
B.1	scripts/benchmark-qoe.js	qoe-ttff.json
B.2	scripts/benchmark-qoe.js	qoe-ttff-cloudfront.json
B.3	scripts/benchmark-qoe.js, deployed as a Lambda to each region	qoe-ttff-multiregion.json
6.1	scripts/qoe/collect.js	qoe-playback-<profile>.json
6.2	Segment sizes and #EXTINF durations read from the processed bucket	—
6.3	scripts/qoe/collect.js, run before and after the correction	qoe-playback-<profile>.json
B.4	scripts/qoe/compare.js, driven by docs/results/qoe-sources.json	qoe-playback-<source>-<profile>.json, qoe-comparison.json
6.4	scripts/qoe/collect.js with -dpr	qoe-playback-<source>-unthrottled.json, qoe-playback-<source>-unthrottled-dpr2.json
6.5	scripts/measure-transcode.js	transcode-timing.json
6.8	scripts/node-load-test.js and scripts/k6-load-test.js	node-load-test.json, stress-test-concurrency.json; the k6 summary of this run was later overwritten and remains in k6-summary.json at commit f955ca9
6.9	scripts/k6-load-test.js	drain-rate-real-payload.json, stress-test-concurrency.json; the first real-payload run is in k6-summary.json
6.10	Published AWS list prices, computed analytically	—

The charts of Chapter 6 are drawn by the scripts docs/diagrams/svg/11-chart-ttff-cdn.mjs to docs/diagrams/svg/16-chart-concurrent-four.mjs, and each reads the result files named above for the table it replaces. The three that chart the transcoding measurements read report/generated/chart-*.json, which scripts/results/build-report-tables.py writes from the same runs as the generated tables, so those charts and tables cannot differ.

A.5 Deploying the Infrastructure

The complete AWS environment is provisioned from a single directory.

```
cd infrastructure/environments/dev
terraform init
terraform apply
```

Enabling signed cookies additionally requires generating an RSA key pair, supplying the public key to Terraform, and configuring the backend with the corresponding private key and the key pair identifier reported by `terraform output`. The full procedure is documented in `docs/PRIVATE_VIDEO_SIGNED_COOKIES.md`.

Appendix B

Measurement Tables

Chapter 6 shows six sets of measurements as charts, because what each of them reports is a comparison or a trend that is easier to see than to read. This appendix gives the same measurements as tables, so that every value can be read exactly and so that the statistics a chart leaves out, such as percentiles and cache-hit medians, remain available. Each chart reads the same result files as its table, and a test in the repository checks that the numbers in the two agree; Section A.4 names the files.

B.1 Time-to-First-Frame

Table B.1 gives the distribution of each stage of the baseline measurement against the S3 bucket; Figure 6.7 shows its medians.

Table B.1: Time-to-First-Frame measurement results

STAGE	MEAN	MEDIAN	P95	MIN	MAX
Master playlist	95.55	82.49	355.02	76.61	355.02
Rendition playlist	88.62	82.40	207.32	77.62	207.32
First segment	108.76	87.04	322.20	78.41	322.20
Total TTFF	**292.93**	**251.52**	**884.54**	**242.23**	**884.54**

Note: values in milliseconds, n = 20 samples, measured against a video served directly from the S3 bucket, before the CloudFront distribution described in Section 5.5 was deployed. These figures therefore represent origin latency without any CDN, and they serve here as the baseline against which Section 6.3.1 compares the deployed system. The mean (292.93 ms) exceeds the median (251.52 ms) because one outlier sample (884.54 ms) inflates both the mean and the P95/max columns, which coincide with the

single-sample maximum at this sample size.

Table B.2 gives the same statistics for the measurement through CloudFront after the price class correction, which Figure 6.7 compares with the baseline.

Table B.2: Time-to-First-Frame through CloudFront after the price class correction

STAGE	**MEAN**	**MEDIAN**	**P95**	**MIN**	**MAX**
Master playlist	28.65	8.37	263.43	6.54	263.43
Rendition playlist	29.29	7.93	348.78	5.89	348.78
First segment	112.75	87.36	487.31	68.87	487.31
Total TTFF	**170.69**	**110.32**	**1099.51**	**83.17**	**1099.51**
Warm cache only ($n = 18$)	*108.49*	*107.57*	*159.25*	*83.17*	*159.25*

Note: values in milliseconds, $n = 20$ samples with Signed Cookies attached. Two samples were edge cache misses and eighteen were hits; the final row reports only the samples that hit at all three stages, which is the steady state a viewer of an already-watched video experiences. The P95 and maximum of the full sample coincide with the colder of the two misses, as at this sample size the 95th percentile is the single slowest observation.

Table B.3 adds, to the medians of Figure 6.8, the edge location that served each client and the median over the samples that hit the cache.

Table B.3: Time-to-First-Frame by client location, before and after `PriceClass_All`

	PRICECLASS_200		**PRICECLASS_ALL**		
CLIENT LOCATION	**EDGE**	**MEDIAN**	**EDGE**	**MEDIAN**	**WARM**
N. Virginia, United States	IAD61	55.23	n/a	n/a	n/a
Dublin, Ireland	DUB2	55.91	DUB2	55.68	55.68
Singapore	SIN3	56.89	SIN3	53.20	53.20
Tokyo, Japan	NRT12	59.68	n/a	n/a	n/a
Sydney, Australia	n/a	n/a	SYD3	52.30	43.36
São Paulo, Brazil	CPT51	**1196.10**	GRU1	**36.79**	**36.49**
Ho Chi Minh City (consumer)	SGN50	110.32	n/a	n/a	n/a

Note: values in milliseconds, $n = 20$ per location and per configuration. WARM reports the median over samples that hit the edge cache at all three stages. Entries marked n/a were not measured in that configuration: the first run predated the Sydney probe, and the second re-measured only the two locations that could change plus two controls. The Ho Chi Minh City row is the local measurement of Table B.2, taken from a consumer connection rather than from inside an AWS region, and is included for comparison rather than as a like-for-like sample.

B.2 Playback Quality

Table B.4 gives all five measures for each of the nine combinations of source and network profile, of which Figure 6.9 charts two.

Table B.4: Playback quality across three sources of differing visual complexity

METRIC	LOW	MEDIUM	HIGH
Unthrottled			
Rebuffering ratio, mean	0.00%	0.00%	0.00%
Rebuffering ratio, max	0.00%	0.00%	0.00%
Runs exhibiting a stall	0/5	0/5	0/5
Startup delay, median (s)	1.46	1.34	1.21
Average bitrate (kbit/s)	1644	1830	1664
Fast 3G			
Rebuffering ratio, mean	0.22%	2.30%	0.11%
Rebuffering ratio, max	0.60%	5.81%	0.55%
Runs exhibiting a stall	2/5	2/5	1/5
Startup delay, median (s)	1.22	1.15	1.12
Average bitrate (kbit/s)	1477	1316	1246
Slow 3G			
Rebuffering ratio, mean	4.91%	5.80%	4.84%
Rebuffering ratio, max	13.14%	13.73%	13.37%
Runs exhibiting a stall	3/5	3/5	3/5
Startup delay, median (s)	3.76	1.87	3.84
Average bitrate (kbit/s)	1070	1224	845

Note: five recorded runs per cell after two discarded warm-up runs, 120 s of playback each, at a device pixel ratio of one. Source durations are 4:23, 4:23, and 4:40, all longer than the measurement window.

B.3 Transcoding Latency and Capacity

Table B.5 lists the ten runs of the 25-minute video with the columns that Figure 6.10 does not show: capacity, slots and the length of the chunk phase.

Table B.5: A 25-minute video on the chunked path, by chunk configuration and capacity

N × VCPU	FARGATE	VCPU	SLOTS	PHASE (MIN)	READY (MIN)	VCPU-S
5 × 1	Spot	8	8	n/a	30.1	n/a
5 × 1†	Spot	8	8	32.6	36.3	7 117
23 × 1	Spot	8	8	19.2	23.6	7 270
12 × 2	Spot	8	4	16.3	20.1	6 816
6 × 4	Spot	8	2	16.8	20.4	6 792
6 × 4	Spot	8	2	14.4	18.0	6 363
6 × 4	On-Demand	8	2	21.7	26.0	10 182
6 × 4	On-Demand	8	2	18.2	22.5	7 852
12 × 4	Spot + OD	16	4	11.5	15.3	8 827
12 × 4	Spot + OD	16	4	10.2	14.7	8 159

Note: the first four rows were timed from the creation of the first job and the later rows from the end of the upload, which differ by about half a minute. The vCPU-seconds of the first five rows count chunk jobs only; the later rows count every job of the video. Rows are in chronological order of measurement. The first column gives the number of chunks and the vCPU of each; † *marks the run on the production environment. PHASE is the time from the first chunk start to the last chunk end, SLOTS the number of chunks encoded at once, VCPU-S the vCPU-seconds, and OD stands for On-Demand.*

Table B.6 gives the minutes of each step of the four-hour video, including the steps too short to be labelled in Figure 6.11, and the compute used.

Table B.6: A four-hour video at two capacities

STEP (MINUTES)	16 VCPU	8 VCPU
Planner (Lambda submit to child jobs)	0.4	0.4
Audio 64 kbit/s and 128 kbit/s (in parallel)	9.6	4.7
48 chunk jobs, 4 vCPU each	71.8	125.9
Finalizer (playlists, thumbnail, moderation, READY)	2.3	3.2
Upload complete to READY	**85.3**	**135.3**
Compute (vCPU-hours)	17.3	15.1

Table B.7 gives, for each of the four videos, the start and end of the chunk phase that Figure 6.12 draws.

Table B.7: Four 25-minute videos submitted together, by capacity

VCPU	ORDER	START (MIN)	END (MIN)	READY (MIN)
8	1	2.8	19.7	21.0
8	2	18.4	37.1	38.7
8	3	35.5	58.5	62.9
8	4	52.2	71.9	73.3
16	1	3.0	16.0	17.7
16	2	2.7	24.6	25.7
16	3	22.2	35.0	36.1
16	4	32.0	43.3	44.3

Note: in the 8 vCPU rows the four videos were uploaded together over one connection and timed from the first job creation. In the 16 vCPU rows the arrival was produced by server-side copies of one uploaded file, so all four events reached the pipeline within a second. START and END are the first chunk start and the last chunk end, and ORDER is the order of completion.